

尚硅谷大数据项目 之 数据治理考评平台

（作者：尚硅谷研究院）

版本：V2.0

第 1 章 数据治理考评平台介绍

1.1 为什么要数据治理

随着大数据技术的逐渐普及，越来越多的企业搭建了数据仓库、实时数仓、数据湖等等。但是由于企业各个系统平台数据源复杂，企业的组织结构的复杂，前后时期口径不统一等等原因，最终造成：

数据质量低：数据错误、不准确或不一致，导致决策依据不可靠，影响企业的运营效率和盈利能力。

数据孤岛：各部门和系统之间的数据无法有效整合，形成数据孤岛，降低数据的利用价值和沟通效率。

数据安全风险：缺乏有效的数据保护措施，可能导致敏感数据泄露，增加企业的法律和财务风险。

规范性问题：在数据使用、存储和传输过程中，可能无法满足规范性要求，增加数据的理解难度，降低使用率。

难以支持创新：由于数据质量、一致性和安全性问题，企业难以利用数据驱动创新和提升竞争力。

难以实现数据驱动：在没有良好数据治理的情况下，企业难以实现数据驱动的决策、市场营销和产品开发。

用户体验受损：数据问题可能导致客户和员工的不良体验，影响客户满意度和员工士

气。

1.2 什么是数据治理

数据治理是一种系统化的方法，旨在管理企业中数据的质量、一致性、安全性和完整性。它涉及一系列策略、流程、技术和工具，帮助组织有效地收集、存储、访问和利用数据。数据治理在以下几个方面具有重要意义：

数据质量：确保数据的准确性、一致性和可用性，降低错误和冗余，为决策提供可靠基础。

数据规范：通过标准化和数据整合，使不同部门和系统间的数据具有统一的标准规范，提高沟通效率和数据利用价值。

数据安全：保护敏感数据，防止未经授权的访问和泄露，确保合规性和隐私保护。

数据完整性：维护数据的完整性，确保不受损坏或误操作影响，提高数据恢复能力。

数据策略和流程：制定和执行数据治理政策，明确数据所有权和责任，促进组织内数据的有效管理。

1.3 数据治理方案之数据中台

数据治理本身是一个理念，必然需要具体的实施策略。

就像许多年前企业从纸质办公过渡到信息化建设，势必就搭建各种 OA、CRM、ERP 等各种企业管理系统，来实现企业的业务流程和管理制度。

数据治理本身也是一种管理制度，对应的落地解决方案中最常见的就是**数据中台**。

一个完善的数据中台项目，可谓是“一站式”的数据处理与治理平台，包含但不限于如下功能：

数据接入与集成：支持多种数据源的接入和集成，包括结构化、非结构化和半结构化数据，实现数据的统一收集和管理。

数据清洗与转换：提供数据清洗、转换和标准化功能，以提高数据质量和一致性，满足不同业务需求。

数据存储与管理：具备高效、可扩展的数据存储和管理能力，确保数据的完整性和安全性。

数据质量管理：监控和评估数据质量，包括准确性、完整性、一致性、及时性等方面，确保数据可靠性。

元数据管理与血缘管理：元数据管理以及数据血缘功能，方便用户发现、理解和使用

更多 [Java - 大数据 - 前端 - python 人工智能资料下载](#)，可[百度访问](#)：[尚硅谷官网](#)

数据。

数据安全性与合规：提供数据加密、访问控制、审计等功能，确保数据安全，同时满足相关法规和行业标准的合规要求。

数据分析与可视化：支持多维度的数据分析和可视化功能，帮助用户快速洞察数据背后的价值和趋势。

数据共享与服务：提供数据共享和 API 服务功能，便于跨部门和系统间的数据协同和价值创造。

数据治理策略与流程：制定和实施数据治理策略、流程和规范，明确数据所有权和责任，促进组织内的数据治理。

数据监控与运维：实时监控数据中台的性能和运行状况，确保数据的高可用性和稳定性，及时处理潜在问题。

总之，一个完善的数据中台应具备数据接入与集成、清洗与转换、存储与管理、质量管理、元数据管理、血缘管理、安全与合规、分析与可视化、共享与服务、治理策略与流程、监控与运维等多种功能，以支持企业实现高效的数据治理和价值创造。

1.4 数据治理方案之考评平台

相对于大而全的数据中台项目，本课程中数据治理考评平台是相对更轻量级的平台。

数据治理考评平台致力于从发现数据处理的各种问题入手，直击痛点，将问题量化，就像扫描电脑健康状态一样，扫描数据处理（目前以离线数仓为主）的全流程，把数据治理作为一张答卷，给出一个总评，并列出各个扣分项，以便逐步改善。

1.5 数据考评的指标

平台把考评的指标分为 5 个大类：

1) 规范

主要是考评数据是否合规、信息是否完整、责任划分是否明确。

2) 存储

主要考评数据存储规划是否合理。

3) 计算

主要考评计算逻辑规划是否合理

4) 质量

主要考评数据产出是否符合预期

更多 [Java - 大数据 - 前端 - python 人工智能资料下载](#)，可百度访问：[尚硅谷官网](#)

5) 安全

主要考评数据是否安全

1.6 指标列表

考评板块	考评指标	考评标准
规范	是否有技术 owner	有 则 10 分，无 则 0 分
	是否有业务 owner	有 则 10 分，无 则 0 分
	表名是否合规	参考建数仓表规范 ODS 层： 开头:ods 结尾 :inc/full 结构 ods_xx_(inc full) DIM 层： dim 开头 full/zip 结尾 结构: dim_xx_(zip full) DWD 层: dwd 开头 inc/full 结尾 结构: dwd_xx_xx_(inc full) DWS 层: dws 开头 结构 dws_xx_xx_xx_(1d/nd/td) ADS 层: ads 开头 结构 ads_xxx DM 层: dm 开头 结构: dm_xx OTHER: 未纳入分层 符合则 10 分，否则 0 分，未纳入分层给 5 分
	是否有表备注	有 则 10 分，无 则 0 分
	是否有字段备注	有备注字段/所有字段 *10 分
存储	生命周期是否合理	未设定周期类型的 给 0 分 周期类型为永久、拉链表 则给 10 分 周期类型为日分区： 无分区信息的给 0 分 没设生命周期给 0 分 周期长度超过建议周期天数{days}给 5 分
	是否空表	空表则 0 分，有数据则 10 分
	是否存在相似表	同层次两个表字段重复超过 {percent} %，则给 0 分，其余给 10 分

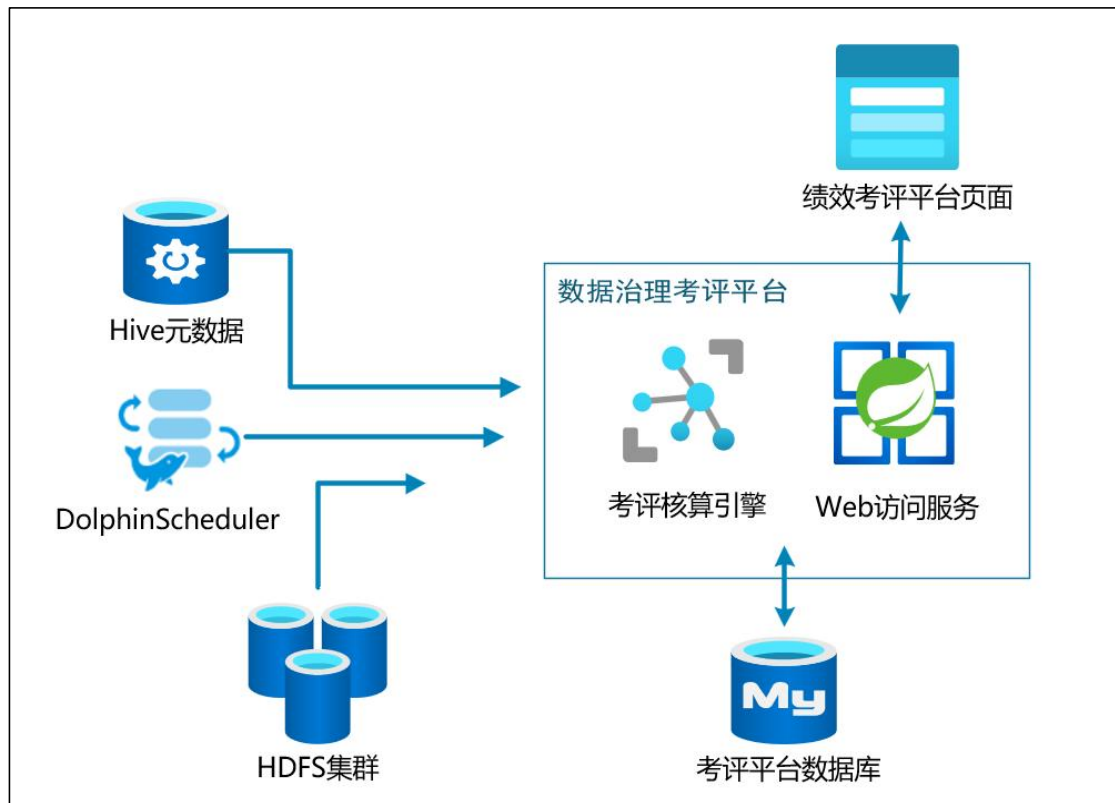
计算	是否长期无产出	一张表{days}天内没有产出数据 则给 0 分，其余给 10 分
	是否长期无访问	一张表{days}天内没有访问 则给 0 分，其余给 10 分
	计算中是否有报错	检查 DS 有报错 则给 0 分，其余给 10 分
	sql 是否包含 select *	sql 语句中包含 select * ，给 0 分，其余给 10 分
	sql 是否简单加工	sql 语句没有任何 join\groupby\union\函数的复杂计算，且 where 过滤字段皆为分区字段，则视为简单加工，给 0 分，其余给 10 分
质量	计算中是否存在数据倾斜	检查是否有数据倾斜，如果某个 stage 的最大任务耗时超过平均耗时任务耗时的 {percent} %，只检查耗时超过 {stage_dur_seconds} 秒的 stage。 存在倾斜给 0 分，不存在给 10 分。
	表产出时效监控	当日产出时效，超过前 {days} 天产出时效平均值 {percent} % 则给 0 分，其余 10 分
	表产出数据量监控	必须日分区表 当日产出的数据量，超过前 {days} 天平均产出量 {upper_limit} %，或低于 {lower_limit} %，则给 0 分，其余 10 分
安全	是否未明确安全等级	未设置 0 分 其余 10 分
	目录文件数据访问权限是否超过建议值	检查该表最高权限的目录或者文件，如果超过文件 超过 {file_permission} 或者 目录 超过 {dir_permission} 则给 0 分 其余给 10 分

第 2 章 数据治理考评平台的技术方案

考评平台内部主要由两部分组成。

一部分是最核心的“考评核算引擎”，主要功能是结合考评平台数据考核参数以及各个数据组件的相关信息，比如 Hive 元数据库、DolphinScheduler 中配置的任务信息、任务完成状况、HDFS 集群的相关信息，进行考评最终核算出分数，并进行保存。

另一部分是服务于外部访问的请求，比如查看考评、维护参数、补充元数据等等。



需要掌握技能：

利用 Springboot 搭建 web 服务。

利用 MyBatis 及 MyBatis-Plus 操作 MySQL 数据库。

从 Hive 中提取需要的元数据。

从 HDFS 中提取相关信息。

从 DolphinScheduler 的任务信息中提取相关信息。

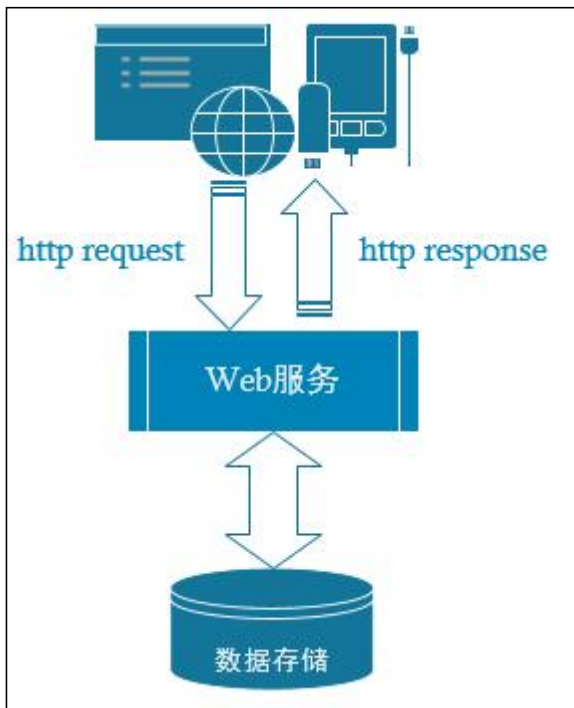
等等。

第 3 章 Springboot 入门

3.1 Web 服务

3.1.1 什么是 Web 服务

Web 服务（http 服务），接收网页或者 App 应用发起的 http 请求，然后通过程序进行处理、计算、查询、存储等，再将结果返回给页面或 App 应用。



3.1.2 传统的 Web 应用框架

为了方便 Web 应用程序的开发，从上个世纪各种 Web 应用框架层出不穷，不同的语言（java, perl, asp,php,C#,python），纷纷下场逐鹿，单就目前而言，在 Web 应用开发领域，Java 占据了最大的市场份额，而其中的 spring 家族又几乎统领了 Java 领域的 Web 框架。

在 Springboot 框架之前，最流行的框架组合就是所谓的“SSM”，即 SpringMVC+Spring+MyBatis

SpringMVC	负责处理 Web 请求
Spring	把整个业务处理组件化，控制整个项目的业务处理。
MyBatis	负责对接数据库

3.2 Springboot 介绍

3.2.1 Springboot 和 SSM 的关系

SSM 虽好，但是通过搭建一个 SSM 项目还是比较繁琐的，需要大量的 xml 配置文件。

springboot 整合了 springmvc , spring , MyBatis 等核心功能，相当于 springboot 单独包装了一层。也就是说本质上实现功能的还是原有的 spring ,springmvc, MyBatis 。

3.2.2 Springboot 的优势

更多 Java -大数据 -前端 -python 人工智能资料下载，可百度访问：[尚硅谷官网](#)

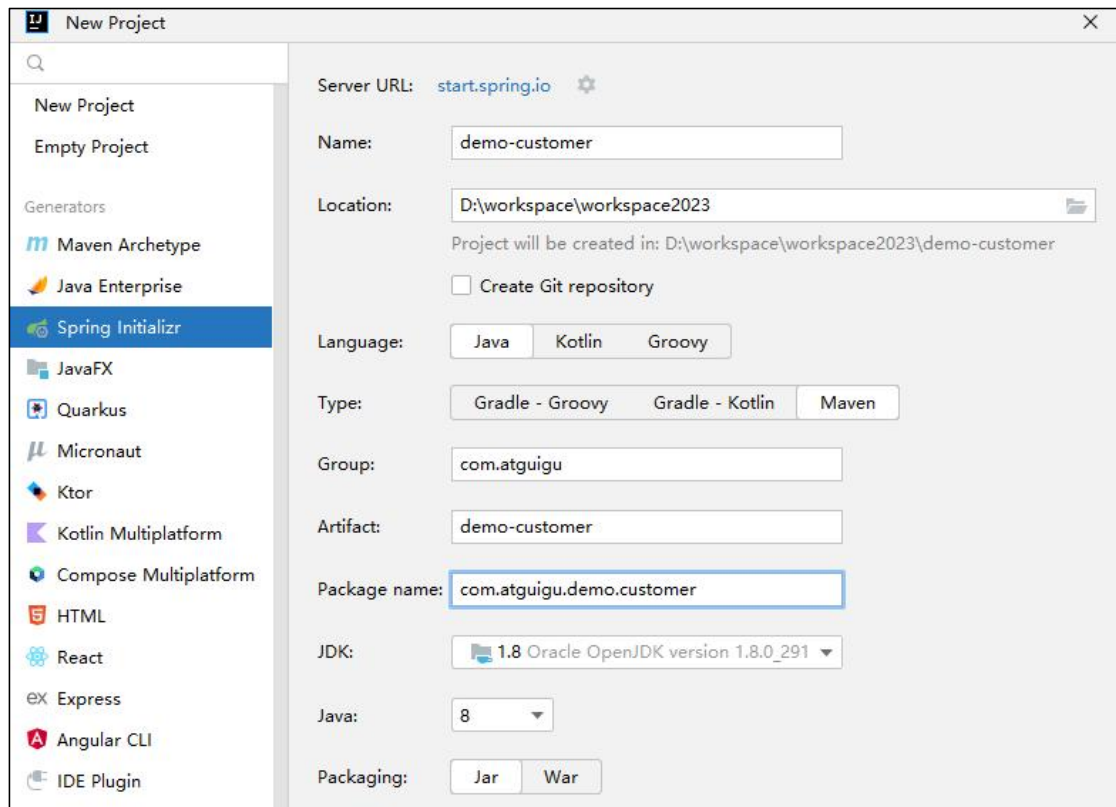
Spring Boot 是由 Pivotal 团队提供的全新框架，其设计目的是用来简化新 Spring 应用的初始搭建以及开发过程。该框架使用了特定的方式来进行配置，从而使开发人员不再需要定义样板化的配置。



- 不再需要那些千篇一律，繁琐的 xml 文件。
- 内嵌 Tomcat,不再需要外部的 Tomcat
- 更方便的和各个第三方工具（mysql,redis,elasticsearch,dubbo,kafka 等等整合），而只要维护一个配置文件即可。

3.3 创建模块

3.3.1 选择增加新模块



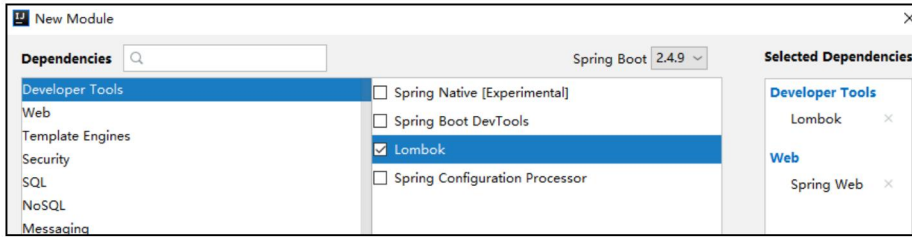
3.3.2 选择依赖

选择 spring web 包括 web 应用的常用注解，以及 web 容器（默认 tomcat）。

更多 Java –大数据 –前端 –python 人工智能资料下载，可百度访问：尚硅谷官网

选择 lombok，修饰实体类的注解插件。

这里不选也可以在之后去 pom.xml 里添加依赖。



pom.xml 中的依赖

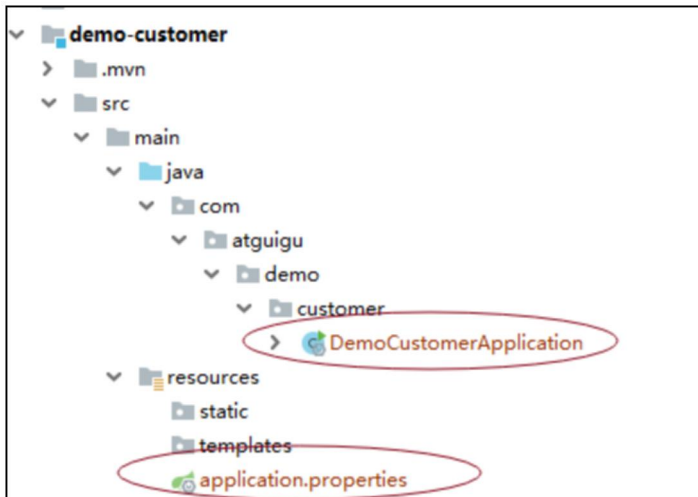
```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>
</dependencies>
```

3.3.3 自动产生的文件

创建好后的，springboot 的工程比 maven 的工程多了两个文件。其中：

DemoCustomerApplication 是整个程序的启动类。

application.properties 是整个工程的配置文件。



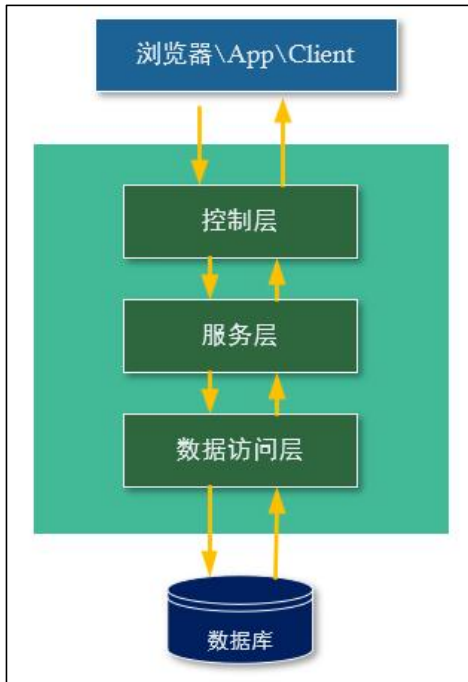
3.4 Springboot 应用开发

3.4.1 应用开发的分层

web 的应用开发，经历了数十年发展至今已经有一套比较成熟的分层结构，目的就是让程序更加的清晰，各层之间各司其职。任何一个 web 应用开发程序员接手一个新项目都

更多 Java –大数据 –前端 –python 人工智能资料下载，可百度访问：尚硅谷官网

可以很快上手，就是因为大家都在履行这种分层约定。



分层及各层职责

分层	名称	职责	涉及的标签
controller	控制层	1 接收请求和参数 2 调用服务层 3 返回响应和结果	@RestController @RequestMapping @RequestParam @RequestBody @PathVariable @GetMapping @PostMapping
service	服务层 (业务层)	1 接收控制层调用 2 处理业务 3 调用数据层 4 返回结果	@Service @Autowired
mapper	数据层	1 接收服务层调用 2 数据库的操作 (SQL) 3 返回结果	@Select @Insert @Delete @Update @Param

3.4.2 控制层(controller)

3.4.2.1 职责

- (1) 接收请求和参数
- (2) 调用服务层
- (3) 返回响应和结果

3.4.2.2 注解说明

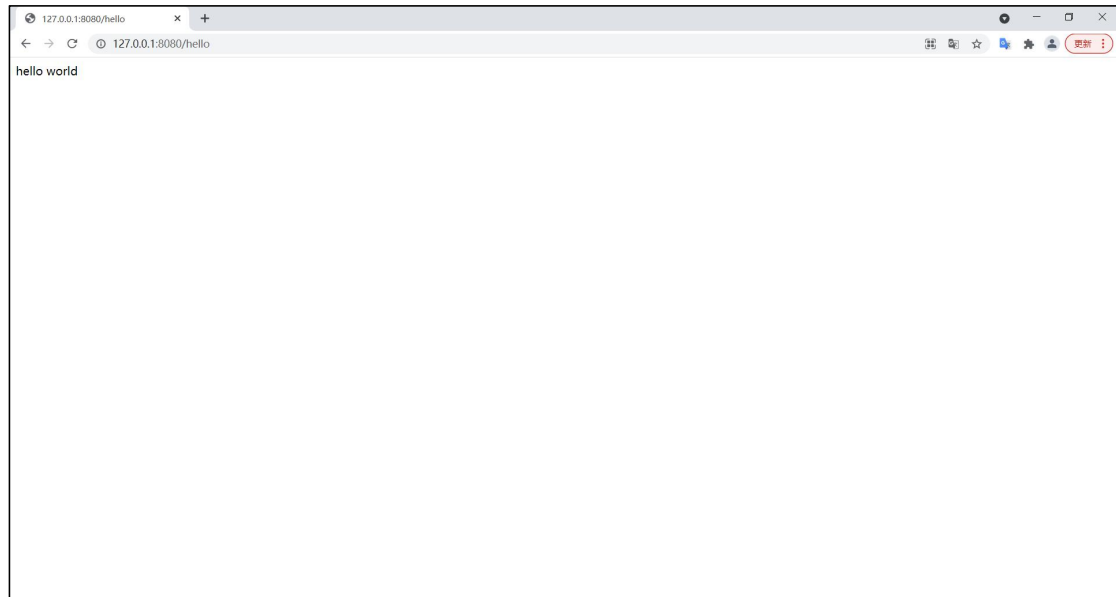
注解	位置	
<code>@RestController</code>	类	标识入口类
<code>@RequestMapping</code>	方法	标识入口方法
<code>@GetMapping</code>	方法	标识入口方法 GET 请求专用
<code>@PostMapping</code>	方法	标识入口方法 POST 请求专用
<code>@RequestParam</code>	参数	接收路径上的键值对参数 <code>http://xxxxx/xx?name=xxx&age=xxx</code>
<code>@PathVariable</code>	参数	接收路径上的值 <code>http://xxxxx/customer/123</code>
<code>@RequestBody</code>	参数	接收请求体(payload) 中的参数

3.4.2.3 HelloWorld

1) 请求打通

```
/**
 * @RestController 将类标识为 Controller ， 作为请求的入口类。
 */
@RestController
public class CustomerController {
    /**
     * 请求处理方法
     * 通过@RequestMapping 将请求与方法进行映射
     */
    @RequestMapping("/hello")
    public String helloController(){
        return "hello world" ;
    }
}
```

2) 浏览器地址栏输入 127.0.0.1:8080/hello 进行访问



3.4.2.4 请求参数

1) 请求路径中的键值对参数

例如: <http://127.0.0.1:8080/hello?name=zs&age=22>

```
/**
 * 请求处理方法
 *
 * @RequestParam 用来将请求路径中的键值对参数映射到方法的形参上
 */
@RequestMapping("/hello")
public String helloController(@RequestParam("name") String name , @RequestParam("age")
Integer age ){
    return "hello world  , name = " + name + " , age = "+ age  ;
}
```

2) 请求路径中嵌入的参数

例如: <http://127.0.0.1:8080/hello/lisi/33>

```
/**
 * 请求处理方法
 *
 * @PathVariable 用来将请求路径中嵌入的参数映射到方法的形参上
 */
@RequestMapping("/hello/{name}/{age}")
public String helloController(@PathVariable("name") String name , @PathVariable("age")
Integer age ){
    return "hello world  , name = " + name + " , age = "+ age  ;
}
```

3) 请求体中的参数, 参数直接映射到 bean

```
/**
 * 请求处理方法
 * @RequestBody 用来将请求体中的请求参数映射到 Bean 类中对应的属性上
 */
@PostMapping("/hello")
public String helloController(@RequestBody Customer customer){
    return "hello world  , name = " + customer.getName() + " , age = "+
customer.getAge()  ;
}
```

更多 Java –大数据 –前端 –python 人工智能资料下载, 可百度访问: 尚硅谷官网

}

3.4.3 实体类（bean 层）

1) Lombok 注解

注解	位置	说明
<code>@NoArgsConstructor</code>	类	增加无参构造函数
<code>@AllArgsConstructor</code>	类	增加全参构造函数
<code>@Data</code>	类	增加 getter setter 方法

2) 实体类

```

@Data
@AllArgsConstructor
@NoArgsConstructor
public class Customer {
    private Integer id ;
    private String name ;
    private Integer age ;
}

```

3.4.4 服务层 (Service 层)

3.4.4.1 职责

- (1) 接收控制层调用
- (2) 处理业务
- (3) 调用数据层
- (4) 返回结果

3.4.4.2 注解说明

注解	位置	说明
<code>@Service</code>	类	声明该类为 Service 层 Spring 容器组件
<code>@Component</code>	类	声明该类为 Spring 容器组件，功能上于 <code>@Service</code> 没有区别
<code>@Autowired</code>	类	自动将 Spring 容器中的组件，装配到引用上。

3.4.4.3 HelloWorld

1) Service 层由两个类组成，一个接口和一个实现类组成。

- (1) 服务层接口

```
/**
 * 服务层接口
 */
public interface CustomerService {
}
```

(2) 服务层实现类

```
/**
 * 服务层实现类
 */
public class CustomerServiceImpl implements CustomerService{
}
```

2) 在 CustomerService 中增加方法，例如通过 id 查询 Customer

```
public interface CustomerService {

    public Customer getCustomerById(Integer id ) ;
}
```

3) 在 CustomerServiceImpl 中实现方法

```
@Service
public class CustomerServiceImpl implements CustomerService{

    @Override
    public Customer getCustomerById(Integer id) {
        return new Customer(id,"zhangsan",22);
    }
}
```

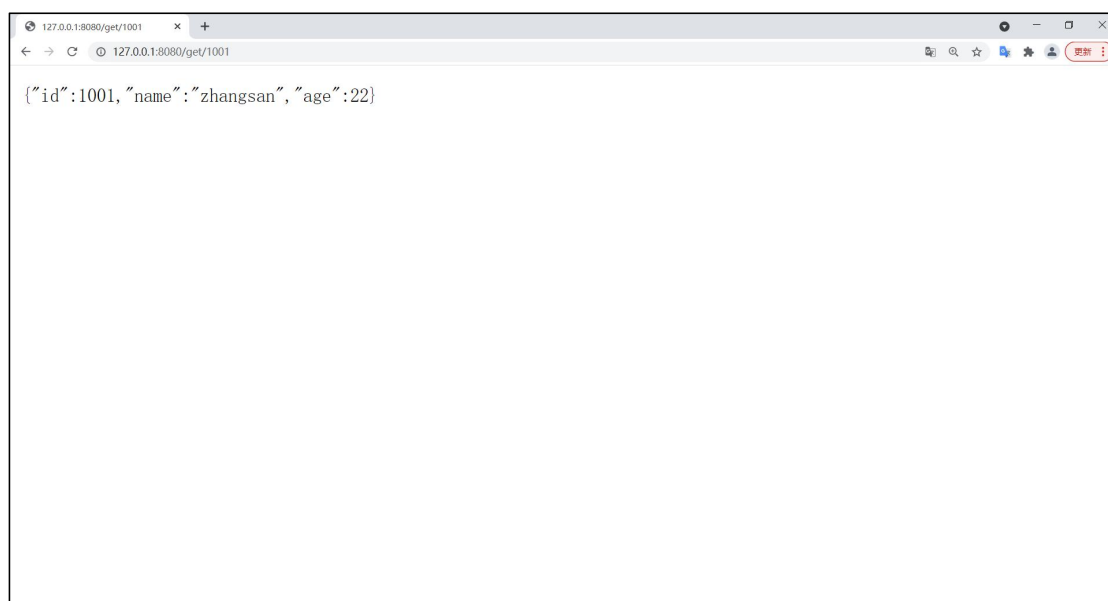
4) 在 Controller 层调用服务层

```
@RestController
public class CustomerController {

    @Autowired
    private CustomerService customerService ;

    @GetMapping("/get/{id}")
    public Customer getCustomer(@PathVariable("id") Integer id ){
        return customerService.getCustomerById(id) ;
    }
}
```

5) 在浏览器地址栏中输入 <http://127.0.0.1/get/1001> 进行访问



3.4.5 数据层（mapper）MyBatis

MyBatis 是一款优秀的持久层框架，它支持自定义 SQL。MyBatis 免除了几乎所有的 JDBC 代码以及设置参数和获取结果集的工作。MyBatis 可以通过简单的 **XML 或注解**来配置和映射。

3.4.5.1 职责

- （1）接收服务层调用
- （2）处理 jdbc 数据的操作
- （3）返回结果

3.4.5.2 注解说明

注解	位置	说明
@Mapper	类	标识 Mapper 接口
@Select	方法	查询语句
@Update	方法	修改语句
@Delete	方法	删除语句
@Insert	方法	插入语句
@Param	参数	标识 SQL 参数

关于注解中的参数替代符

注解	说明
#{}	会识别参数的类型会自动补全单引 本身字符串值中的特殊符号会被替换、转义

更多 Java –大数据 –前端 –python 人工智能资料下载，可百度访问：尚硅谷官网



参数原值（一般程序中动态组合 sql）

3.4.5.3 HelloWorld

1) 创建测试表

```
CREATE TABLE `customer` (  
  `id` bigint(20) NOT NULL AUTO_INCREMENT,  
  `name` varchar(200) DEFAULT NULL,  
  `age` bigint(20) DEFAULT NULL,  
  PRIMARY KEY (`id`)  
)  
ENGINE=InnoDB DEFAULT CHARSET=utf8
```

2) 引入依赖

```
<dependency>  
  <groupId>org.mybatis.spring.boot</groupId>  
  <artifactId>mybatis-spring-boot-starter</artifactId>  
  <version>2.3.1</version>  
</dependency>  
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-jdbc</artifactId>  
</dependency>  
<dependency>  
  <groupId>com.mysql</groupId>  
  <artifactId>mysql-connector-j</artifactId>  
  <scope>runtime</scope>  
</dependency>
```

3) application.properties

```
spring.datasource.url=jdbc:mysql://hadoop102:3306/test?characterEncoding=utf-8&useSSL=false  
spring.datasource.driver-class-name=com.mysql.jdbc.Driver  
spring.datasource.username=root  
spring.datasource.password=000000
```

4) 创建 Mapper 接口

```
@Mapper  
public interface CustomerMapper {  
  
    @Select("select * from customer ")  
    public List<Customer> getCustomerList();  
  
    @Select("select * from customer where id=#{id}")  
    public Customer getCustomerById(Long id);  
  
    @Select("select * from customer")  
    public List<Customer> selectCustomerList();  
  
    @Insert("insert into customer(name,age) values (#{customer.name},#{customer.age})")  
    public void insertCustomer(@Param("customer") Customer customer);  
  
    @Update("update customer set age= #{customer.age} where id=#{ customer.id }")  
    public void updateCustomer (@Param("customer")Customer customer );  
  
    @Delete("delete customer_info where id=#{id}")  
    public void deleteCustomerById (Long id );  
}
```

5) Service 层中加入调用方法

```
@Service
public class CustomerServiceImpl implements CustomerService {

    @Autowired
    CustomerMapper customerMapper;

    @Override
    public Customer getCustomerById(Integer id) {
        return customerMapper.getCustomerById(id);
    }
}
```

3.4.6 MyBatis-Plus

MyBatis-Plus（简称 MP）是一个 MyBatis 的增强工具，在 MyBatis 的基础上只做增强不做改变，完全去 SQL 化，封装好了大量的 CRUD 操作。甚至把 CRUD 操作封装到了 Service 层，可以直接在 controller 调用现成的 CRUD 服务层，极度舒适省心。

局限：只支持单表的简单的 CRUD 操作。不支持多表操作（join, union, 子查询）。

3.4.6.1 HelloWorld

1) 引入 MyBatis-Plus 依赖

```
<!--MyBatis-Plus-->
<dependency>
    <groupId>com.baomidou</groupId>
    <artifactId>mybatis-plus-boot-starter</artifactId>
    <version>3.4.1</version>
</dependency>
```

2) 增加 MyBatis-Plus 规范的服务层和数据层

Mapper 接口

```
import com.baomidou.mybatisplus.core.mapper.BaseMapper;

@Mapper
public interface CustomerMapper extends BaseMapper<Customer> {
}
```

Service 接口

```
public interface CustomerService extends IService<Customer> {
}
```

3) 重写 CustomerService 实现类

```
@Service
public class CustomerImpl extends ServiceImpl<CustomerMapper, Customer> implements
CustomerService {
}
```

只要加上这个三个继承。在 controller 中直接就可以使用 CRUD 命令。

4) 在 Bean 上加入对主键的标识

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Customer {
```

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：[尚硅谷官网](http://www.shangguigu.com)

```
@TableId(value = "id", type = IdType.AUTO) //标识根据数据库自增生成主键
String id;

String name;

int age;
}
```

5) Controller 直接调用封装方法

```
@GetMapping("/customer/list")
public List<Customer> getCustomerList(){
    List<Customer> list = customerService.list();
    return list ;
}

@GetMapping("/customer")
public Customer getCustomer(@RequestParam("id") String id){
    return customerService.getById(Integer.valueOf(id)); //直接返回会自动转为 json
}

@PostMapping("/customer")
public String saveCustomer(@RequestBody Customer customer){
    //会根据主键是否为空来决定 是插入还是修改
    customerService.saveOrUpdate(customer);
    return "success";
}

@DeleteMapping("/customer")
public String deleteCustomer(@RequestParam("id") String id){
    customerService.removeById(id);
    return "success";
}
```

3.4.7 MyBatis-Plus 的代码生成工具

虽然 MyBatis-Plus 已经节省了 sql 编写的大部分工作，但是当面对几十上百个表的模块处理时，仍然又大量的类模块要编写，其中最大的工作量就是实体类的编写，如果手工编写还很容易出错。

所以 MyBatis-Plus 还推出了代码生成工具。

1) 在 pom.xml 中增加代码生成工具

```
<dependency>
    <groupId>com.baomidou</groupId>
    <artifactId>mybatis-plus-generator</artifactId>
    <version>3.5.3.1</version>
</dependency>

<dependency>
    <groupId>org.apache.velocity</groupId>
    <artifactId>velocity-engine-core</artifactId>
    <version>2.3</version>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-freemarker</artifactId>
</dependency>
```

2) 增加一个代码生成的工具类

更多 Java –大数据–前端–python 人工智能资料下载，可百度访问：[尚硅谷官网](http://www.shangguigu.com)

参考:<https://github.com/baomidou/generator>

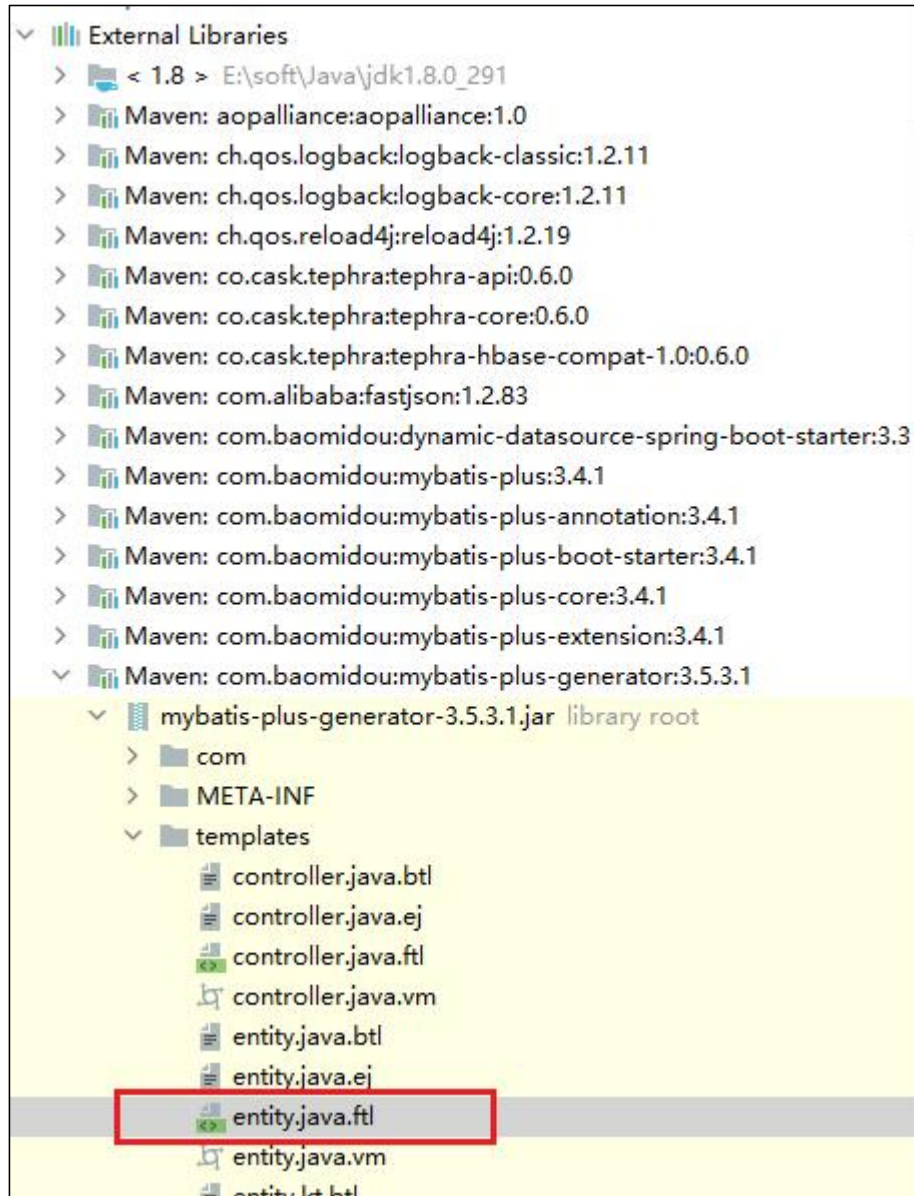
```
public class CodeGen3531 {
    public static void main(String[] args) {
        // String[]
        tables={ "governance_metric","governance_type","governance_assess_detail"};
        // String[] tables={ "t_ds_task_definition","t_ds_task_instance"};
        // String[]
        tables={ "governance_assess_table","governance_assess_tec_owner","governance_assess_global"};
        String[] tables={ "table_meta_info","table_meta_info_extra" };

        FastAutoGenerator.create("jdbc:mysql://hadoop102:3306/dga_1219","root","000000")
            .globalConfig(builder -> {
                builder.author("weiyunhui") //作者
                    .outputDir("D:\\workspace\\workspace2023\\dga_1219\\src\\main\\java") //输出路径(写到 java 目录)
                    .commentDate("yyyy-MM-dd")
                    .dateType(DateType.ONLY_DATE); //选择实体类中的日期类型 , Date
                //or LocalDateTime
            })
            .packageConfig(builder -> { //各个 package 名称
                builder.parent("com.atguigu.dga")
                    .moduleName("meta")
                    .entity("bean")
                    .service("service")
                    .serviceImpl("service.impl")
                    .controller("controller")
                    .mapper("mapper");
            })
            .strategyConfig(builder -> {
                builder.addInclude(tables)
                    .serviceBuilder()
                    .formatServiceFileName("%sService") //类后缀
                    .formatServiceImplFileName("%sServiceImpl") //类后缀
                    .entityBuilder()
                    .enableLombok() //允许使用 lombok
                    .controllerBuilder()
                    .formatFileName("%sController") //类后缀
                    .enableRestStyle() //生成@RestController 否则是@Controller
                    .mapperBuilder()
                    //生成通用的 resultMap 的 xml 映射
                    .enableBaseResultMap() //生成 xml 映射
                    .superClass(BaseMapper.class) //标配
                    .formatMapperFileName("%sMapper") //类后缀
                    .enableFileOverride() //生成代码覆盖已有文件 谨慎开启
                    .mapperAnnotation(Mapper.class) ; //生成代码 Mapper 上自带
                //@Mapper
            })
            .templateConfig(new Consumer<TemplateConfig.Builder>() {
                @Override
                public void accept(TemplateConfig.Builder builder) {
                    // 实体类使用我们自定义模板
                    builder.entity("templates/myentity.java");
                }
            })
            .templateEngine(new FreemarkerTemplateEngine()) // 使用 Freemarker
            //引擎模板, 默认的是 Velocity 引擎模板
            .execute();
    }
}
```

更多 Java-大数据-前端-python 人工智能资料下载, 可百度访问: 尚硅谷官网

3) 定制生成模板

以上生成代码主函数中有涉及自定义模板，从 idea 中的 jar 包目录中找到 jar 包的模板。



拷贝到 resources/template 目录下增加 myentity.java.ftl



把第 29、30 行的

```
...  
<#if entityLombokModel>  
@Getter  
@Setter  
...
```

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：[尚硅谷官网](http://www.shang硅谷.com)

改成

```
.....
<#if entityLombokModel>
@Data
.....
```

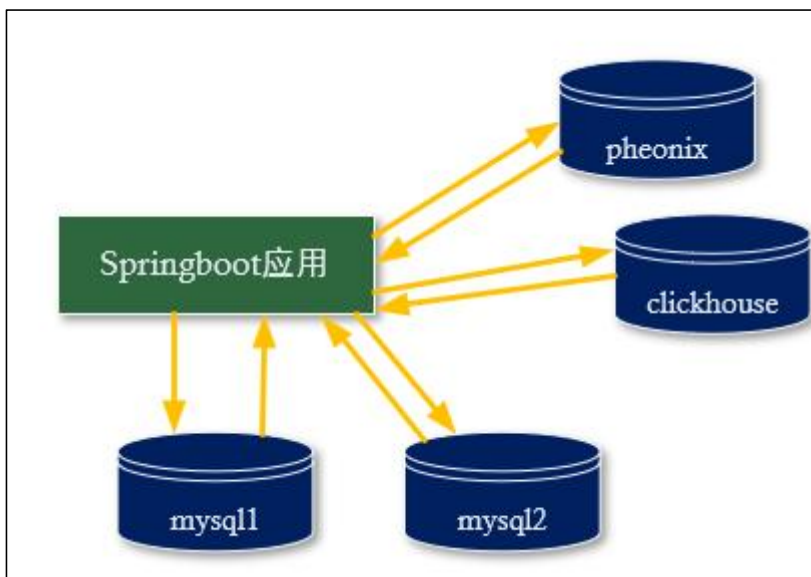
最后运行 CodeGen3531.java 的主函数就可以生成代码了。

3.4.8 动态数据源

在实际开发中经常有面临一个 Springboot 应用要对接多个数据源的情况，但是如果通过配置完成多数据源与一个 Springboot 应用的连接实际上是非常繁琐的工作。

这里介绍一个动态数据源的工具 dynamic-datasource。

这个工具只需要一个注解并配合简单的配置就可以实现一个 Springboot 对接多数据源。



3.4.8.1 相关注解

注解	位置	说明
@DS	类或者方法	@DS 可以注解在方法上或类上，同时存在就近原则 方法上注解 优先于类上注解。

3.4.8.2 HelloWorld

1) 添加依赖

```
<dependency>
  <groupId>com.baomidou</groupId>
  <artifactId>dynamic-datasource-spring-boot-starter</artifactId>
  <version>3.3.2</version>
</dependency>
```

2) 定义多个数据源

```
spring.datasource.dynamic.datasource.mysql0111.url=jdbc:mysql://hadoop102:3306/test?characterEncoding=utf-8&useSSL=false
spring.datasource.dynamic.datasource.mysql0111.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.dynamic.datasource.mysql0111.username=root
spring.datasource.dynamic.datasource.mysql0111.password=000000
```

更多 Java –大数据 –前端 –python 人工智能资料下载，可百度访问：尚硅谷官网

```
spring.datasource.dynamic.datasource.mysql0222.url=jdbc:mysql://hadoop102:3306/demo?characterEncoding=utf-8&useSSL=false
spring.datasource.dynamic.datasource.mysql0222.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.dynamic.datasource.mysql0222.username=root
spring.datasource.dynamic.datasource.mysql0222.password=000000
```

3) 在 service 层和 mapper 的类上增加默认数据源

```
@Service
@DS("mysql0111")
public class CustomerServiceImpl extends ServiceImpl<CustomerMapper, Customer> implements CustomerService {
    @Mapper
    @DS("mysql0111")
    public interface CustomerMapper extends BaseMapper<Customer> {}
```

4) 在特定的方法上增加特定数据源

```
@Insert("insert into customer(name,age) values (#{customer.name}, #{customer.age})")
@DS("mysql0222")
public void insertCustomer0111(@Param("customer") Customer customer);
```

3.4.9 整合 druid 连接池



数据连接池的主要目的是当应用反复访问数据库时，不会每次都重复创建连接。而是复用已有的连接。

3.4.9.1 HelloWorld

1) 添加依赖

```
<dependency>
    <groupId>com.alibaba</groupId>
    <artifactId>druid-spring-boot-starter</artifactId>
    <version>1.2.15</version>
</dependency>
```

2) 添加配置

```
spring.autoconfigure.exclude=com.alibaba.druid.spring.boot.autoconfigure.DruidDataSourceAutoConfigure

spring.datasource.dynamic.datasource.mysql0111.url=jdbc:mysql://hadoop102:3306/test?characterEncoding=utf-8&useSSL=false
spring.datasource.dynamic.datasource.mysql0111.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.dynamic.datasource.mysql0111.username=root
spring.datasource.dynamic.datasource.mysql0111.password=000000

spring.datasource.dynamic.datasource.mysql0111.druid.initial-size=5
spring.datasource.dynamic.datasource.mysql0111.druid.min-idle=5
spring.datasource.dynamic.datasource.mysql0111.druid.max-active=20
spring.datasource.dynamic.datasource.mysql0111.druid.max-wait=60000
spring.datasource.dynamic.datasource.mysql0111.druid.test-on-borrow=true
spring.datasource.dynamic.datasource.mysql0111.druid.test-while-idle=true
spring.datasource.dynamic.datasource.mysql0111.druid.test-on-return=false
```

更多 Java – 大数据 – 前端 – python 人工智能资料下载，可百度访问：尚硅谷官网

```
spring.datasource.dynamic.datasource.mysql0222.url=jdbc:mysql://hadoop102:3306/demo?characterEncoding=utf-8&useSSL=false
spring.datasource.dynamic.datasource.mysql0222.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.dynamic.datasource.mysql0222.username=root
spring.datasource.dynamic.datasource.mysql0222.password=000000
spring.datasource.dynamic.datasource.mysql0222.druid.initial-size=5
spring.datasource.dynamic.datasource.mysql0222.druid.min-idle=5
spring.datasource.dynamic.datasource.mysql0222.druid.max-active=20
spring.datasource.dynamic.datasource.mysql0222.druid.max-wait=60000
spring.datasource.dynamic.datasource.mysql0222.druid.test-on-borrow=true
spring.datasource.dynamic.datasource.mysql0222.druid.test-while-idle=true
spring.datasource.dynamic.datasource.mysql0222.druid.test-on-return=false
```

连接池参数解释

参数	
initial-size	初始连接数
min-idle	最少保持空闲连接数
max-active	最大可活动的连接数
max-wait	最长等待时长
test-on-borrow	借走时测试
test-while-idle	空闲时测试
test-on-return	归还时测试

更丰富的连接池配置参数参考官网说明：

[DruidDataSource 配置属性列表](#) · [alibaba/druid Wiki](#) · [GitHub](#)

3.5 总结

Springboot 作为一个 web 应用快速开发框架，整合了 SpringMVC、Spring、MyBatis 等框架。其中：

SpringMVC 负责接收返回 web 请求。

Spring 负责把业务处理类的组件化，管理生命周期。

MyBatis 负责连接数据库、封装参数、封装结果。

MyBatis-Plus 是 MyBatis 的第三方扩展依赖，可以方便的对数据单表进行插删改查操作。

MyBatis-Plus 提供了代码生成器 generator，辅助我们快速生成基础代码。

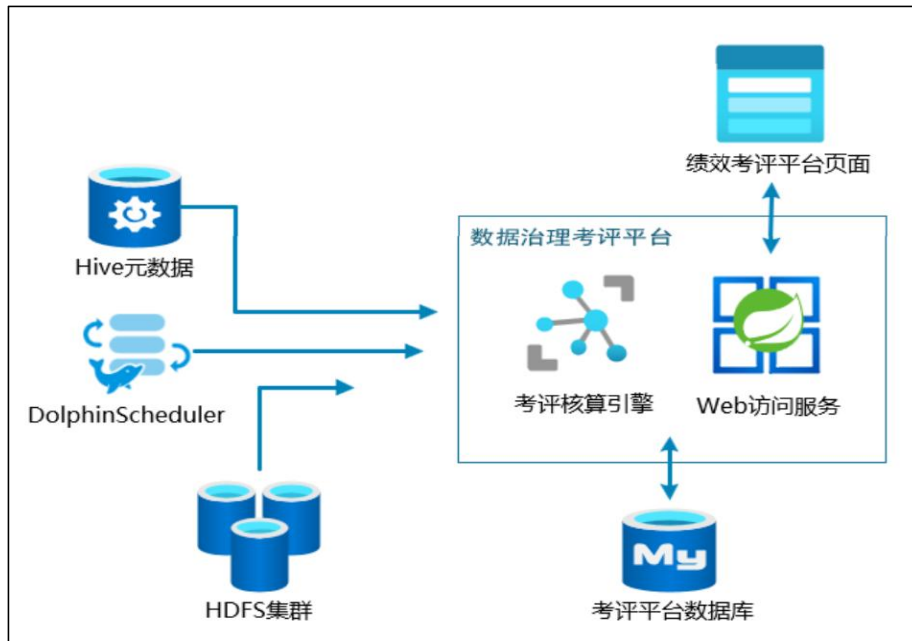
DynamicDatasource 是用来对接多个 jdbc 数据源的工具。

Druid 是一个非常好用的数据连接池，可以使得数据连接循环利用减少开销。

第 4 章 数据治理考评项目基础搭建

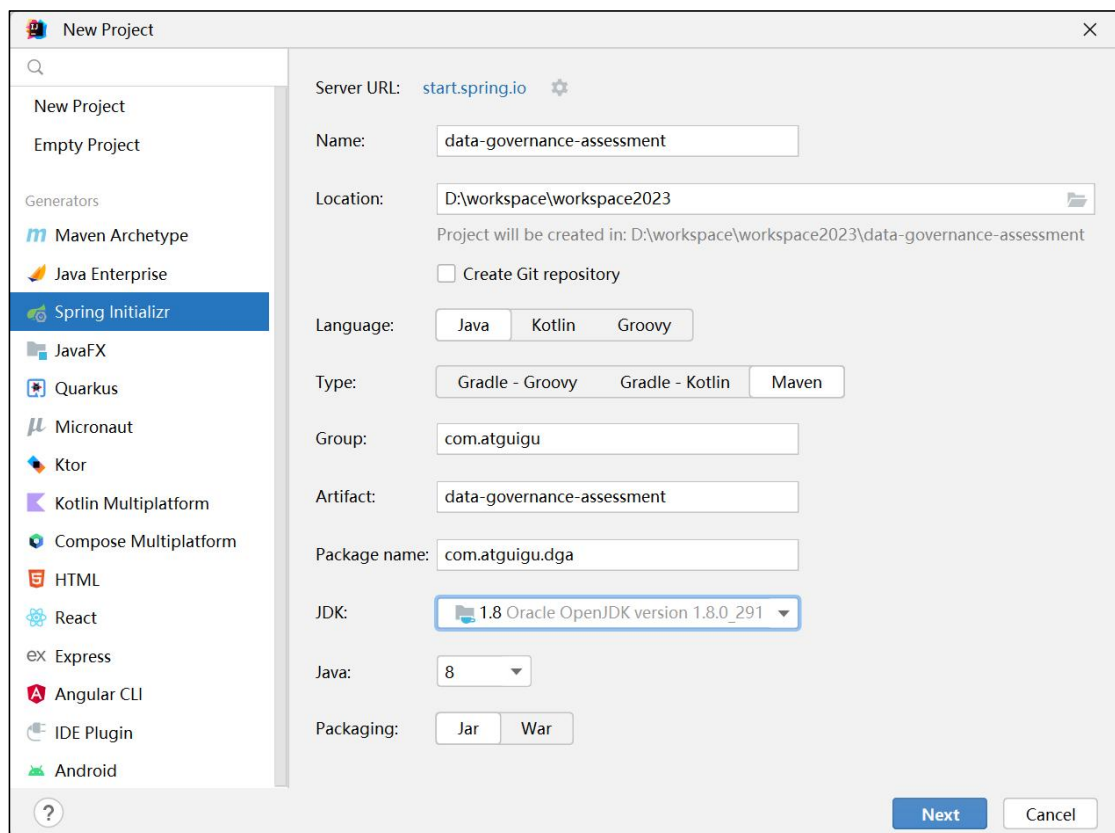
更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网

整个考评项目其实就是一个 Springboot 应用，接下来搭建应用。



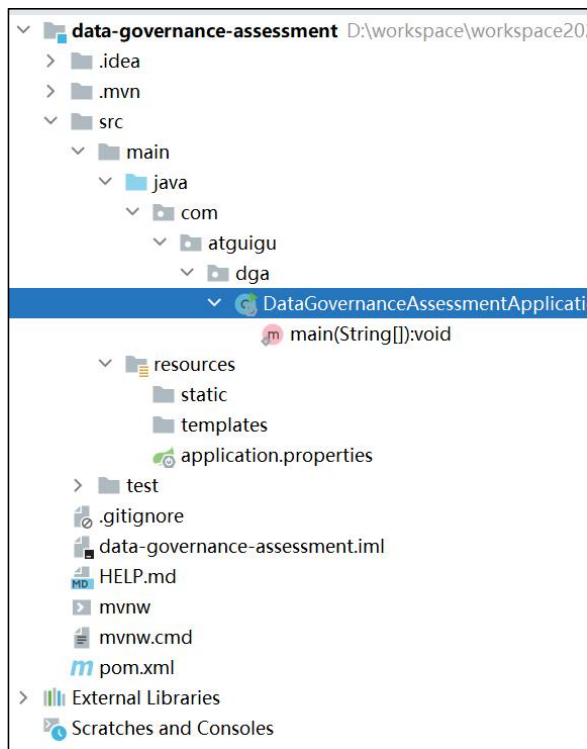
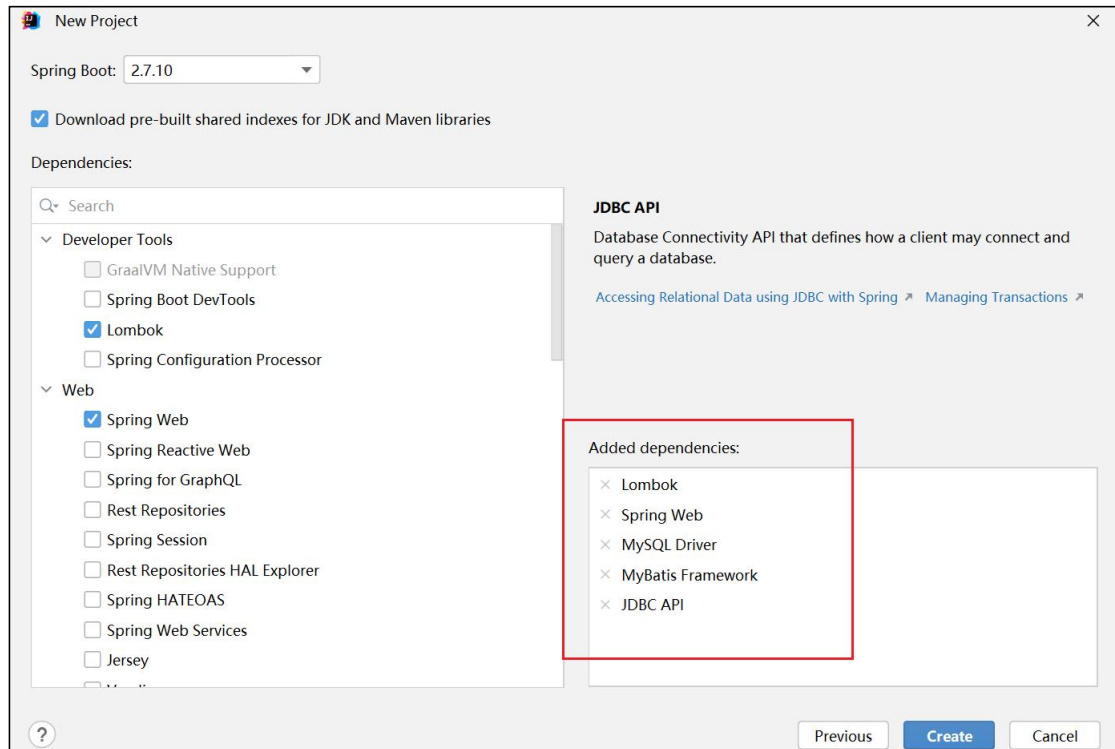
4.1 建立工程项目

1) 填写工程信息



2) 选择依赖

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网



4.2 pom.xml 文件中的依赖

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jdbc</artifactId>
```

更多 Java – 大数据 – 前端 – python 人工智能资料下载，可百度访问：尚硅谷官网

```
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
    <groupId>org.mybatis.spring.boot</groupId>
    <artifactId>mybatis-spring-boot-starter</artifactId>
    <version>2.3.0</version>
</dependency>

<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>

<dependency>
    <groupId>com.baomidou</groupId>
    <artifactId>mybatis-plus-boot-starter</artifactId>
    <version>3.4.1</version>
</dependency>

<dependency>
    <groupId>org.apache.commons</groupId>
    <artifactId>commons-lang3</artifactId>
</dependency>

<dependency>
    <groupId>com.baomidou</groupId>
    <artifactId>dynamic-datasource-spring-boot-starter</artifactId>
    <version>3.3.2</version>
    <scope>compile</scope>
</dependency>

<dependency>
    <groupId>com.alibaba</groupId>
    <artifactId>fastjson</artifactId>
    <version>1.2.83</version>
</dependency>

<dependency>
    <groupId>com.alibaba</groupId>
    <artifactId>druid-spring-boot-starter</artifactId>
    <version>1.2.15</version>
</dependency>
</dependencies>
```

4.3 application.properties

```
server.port=80
```

```
spring.datasource.dynamic.datasource.dga.driver-class-name=com.mysql.cj.jdbc.Driver
```

更多 Java –大数据–前端–python 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
spring.datasource.dynamic.datasource.dga.url=jdbc:mysql://hadoop102:3306/dga?serverTimezone=
UTC&useUnicode=true&characterEncoding=utf-8&useSSL=false&allowPublicKeyRetrieval=true
spring.datasource.dynamic.datasource.dga.username=root
spring.datasource.dynamic.datasource.dga.password=000000
```

4.4 创建数据库

```
CREATE DATABASE `dga` /*!40100 DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci */
```

第 5 章 整体任务分析

对考评的需求功能划分为 5 个任务。

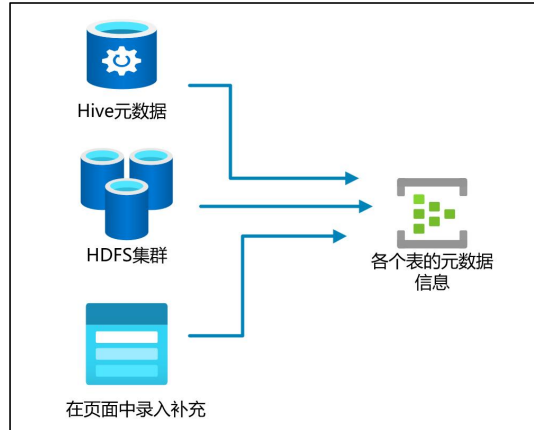


第 6 章 任务一：元数据的加载与处理

6.1 分析

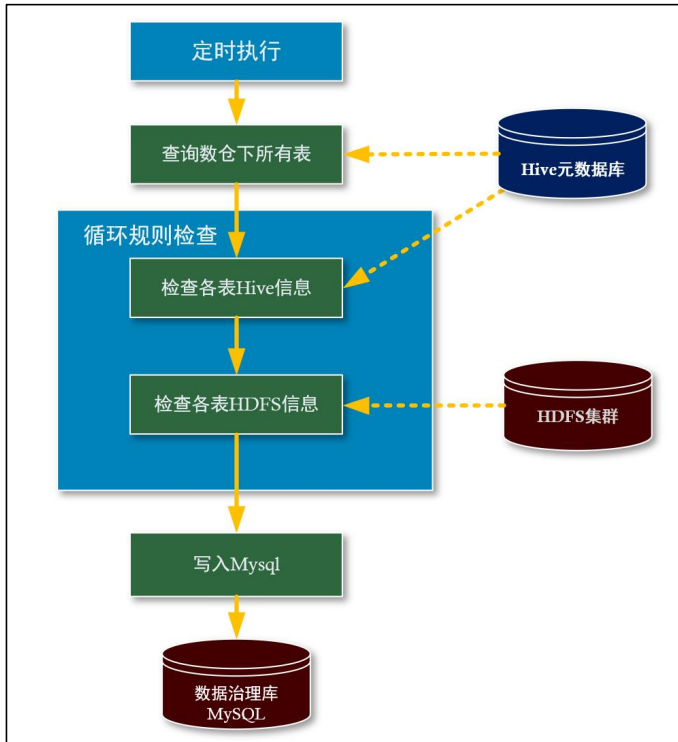
在整个数据治理最核心的对象就是“表”，也是数据资产的基本单元。那进行考评前，我们需要从 hive 以及 hdfs 中获取相关信息。这些信息作为考评的最基本依据，同时可以让大数据开发人员直观的从平台页面中查看平台当前状况。

那么这些表的元数据从何而来呢？



主要是三个来源：Hive 的元数据信息、HDFS 集群，另外还需要由用户通过界面补充录入一些辅助信息。

6.2 程序结构设计



6.3 表结构

元数据表表结构如下：

```

create table table_meta_info
(
    id                bigint primary key auto_increment comment '表id',
    table_name        varchar(200) null comment '表名',
    schema_name       varchar(200) null comment '库名',
    col_name_json     varchar(2000) null comment '字段名 json (来源:hive)',
    partition_col_name_json varchar(4000) null comment '分区字段名 json(来源:hive)',
    table_fs_owner     varchar(200) null comment 'hdfs 所属人 (来源:hive)',
    table_parameters_json varchar(2000) null comment '参数信息 (来源:hive)',
    table_comment     varchar(200) null comment '表备注 (来源:hive)',
)
  
```

更多 Java –大数据 –前端 –python 人工智能资料下载，可百度访问：尚硅谷官网

```

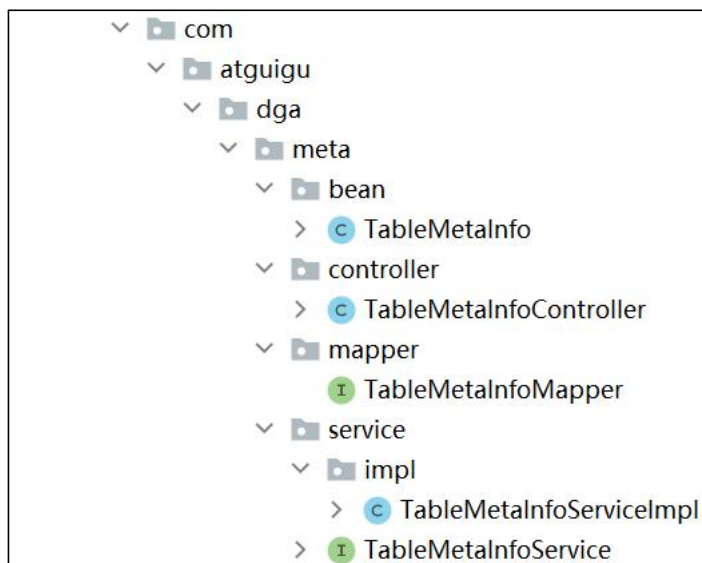
table_fs_path          varchar(200)  null comment 'hdfs 路径 ( 来源:hive)',
table_input_format     varchar(200)  null comment '输入格式 ( 来源:hive)',
table_output_format    varchar(200)  null comment '输出格式 ( 来源:hive)',
table_row_format_serde varchar(200)  null comment '行格式 ( 来源:hive)',
table_create_time      datetime      null comment '表创建时间 ( 来源:hive)',
table_type             varchar(200)  null comment '表类型 ( 来源:hive)',
table_bucket_cols_json varchar(200)  null comment '分桶列 ( 来源:hive)',
table_bucket_num       bigint        null comment '分桶个数 ( 来源:hive)',
table_sort_cols_json   varchar(200)  null comment '排序列 ( 来源:hive)',
table_size             bigint        null comment '数据量大小 ( 来源:hdfs)',
table_total_size       bigint        null comment '所有副本数据总量大小 ( 来源:hdfs)',
table_last_modify_time  datetime      null comment '最后修改时间 ( 来源:hdfs)',
table_last_access_time  datetime      null comment '最后访问时间 ( 来源:hdfs)',
fs_capacity_size       bigint        null comment '当前文件系统容量 ( 来源:hdfs)',
fs_used_size           bigint        null comment '当前文件系统使用量 ( 来源:hdfs)',
fs_remain_size         bigint        null comment '当前文件系统剩余量 ( 来源:hdfs)',
assess_date            varchar(10)    null comment '考评日期 ',
create_time            datetime      null comment '创建时间 (自动生成)',
update_time            datetime      null comment '更新时间 (自动生成)',
constraint table_meta_info_pk
    unique (table_name, schema_name, assess_date)
)
comment '元数据表';

```

说明： 该表设置了唯一键，即每天每张表元数据只保留一份，同一日期同一张表的元数据后面的要覆盖前面的。

6.4 搭建模块基本结构

类名	说明
TableMetaInfoServiceImpl	服务层，主要逻辑
TableMetaInfoService	服务层接口
TableMetaInfo	实体类
TableMetaInfoMapper	数据层
TableMetaInfoController	控制层，web 接口



更多 Java –大数据 –前端 –python 人工智能资料下载，可百度访问：尚硅谷官网

以上类可以利用 MyBatis-Plus-generator 自动生成。

6.4.1 生成器生成代码

需添加的依赖如下：

```
<dependency>
  <groupId>com.baomidou</groupId>
  <artifactId>mybatis-plus-boot-starter</artifactId>
  <version>3.4.1</version>
</dependency>

<dependency>
  <groupId>com.baomidou</groupId>
  <artifactId>mybatis-plus-generator</artifactId>
  <version>3.5.3.1</version>
</dependency>

<dependency>
  <groupId>org.apache.velocity</groupId>
  <artifactId>velocity-engine-core</artifactId>
  <version>2.3</version>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-freemarker</artifactId>
</dependency>
```

生成器代码如下：

```
public class CodeGen3531 {
    public static void main(String[] args) {
        // String[]
        tables={ "governance_metric","governance_type","governance_assess_detail"};
        // String[] tables={ "t_ds_task_definition","t_ds_task_instance"};
        // String[]
        tables={ "governance_assess_table","governance_assess_tec_owner","governance_assess_global"};
        String[] tables={ "table_meta_info","table_meta_info_extra" };

        FastAutoGenerator.create("jdbc:mysql://hadoop102:3306/dga","root","000000")
            .globalConfig(builder -> {
                builder.author("weiyunhui") //作者
                    .outputDir("D:\\workspace\\workspace2023\\dga\\src\\main\\java") //输出路径(写到 java 目录)
                    .commentDate("yyyy-MM-dd")
                    .dateType(DateType.ONLY_DATE); //选择实体类中的日期类型 , Date or LocalDateTime
            })
            .packageConfig(builder -> { //各个 package 名称
                builder.parent("com.atguigu.dga")
                    .moduleName("meta")
                    .entity("bean")
                    .service("service")
                    .serviceImpl("service.impl")
                    .controller("controller")
                    .mapper("mapper");
            })
            .strategyConfig(builder -> {
                builder.addInclude(tables)
                    .serviceBuilder()
                    .formatServiceFileName("%sService") //类后缀
                    .formatServiceImplFileName("%sServiceImpl") //类后缀
            })
    }
}
```

更多 Java-大数据-前端-python 人工智能资料下载，可百度访问：[尚硅谷官网](http://www.shangguigu.com)

```

        .entityBuilder()
        .enableLombok() //允许使用 lombok
        .controllerBuilder()
        .formatFileName("%sController") //类后缀
        .enableRestStyle() //生成@RestController 否则是@Controller
        .mapperBuilder()
        //生成通用的 resultMap 的 xml 映射
        .enableBaseResultMap() //生成 xml 映射
        .superClass(BaseMapper.class) //标配
        .formatMapperFileName("%sMapper") //类后缀
        .enableFileOverride() //生成代码覆盖已有文件 谨慎开启
        .mapperAnnotation(Mapper.class) ; //生成代码 Mapper 上自带

    @Mapper

    })
    .templateConfig(new Consumer<TemplateConfig.Builder>() {
        @Override
        public void accept(TemplateConfig.Builder builder) {
            // 实体类使用我们自定义模板
            builder.entity("templates/myentity.java");
        }
    })
    .templateEngine(new FreemarkerTemplateEngine()) // 使用 Freemarker 引擎模板,
    默认的是 Velocity 引擎模板
    .execute();
}
}

```

生成的代码如下

6.4.2 bean 对象

```

/**
 * <p>
 * 元数据表
 * </p>
 *
 * @author weiyunhui
 * @since 2023-04-12
 */
@Data
public class TableMetaInfo implements Serializable {

    private static final long serialVersionUID = 1L;

    /**
     * 表 id
     */
    @TableId(value = "id", type = IdType.AUTO)
    private Long id;

    /**
     * 表名
     */
    private String tableName;

    /**
     * 库名
     */
    private String schemaName;

    /**
     * 字段名 json ( 来源:hive)

```

更多 Java –大数据 –前端 –python 人工智能资料下载，可百度访问：尚硅谷官网


```
    */
    private String colNameJson;

    /**
     * 分区字段名 json( 来源:hive)
     */
    private String partitionColNameJson;

    /**
     * hdfs 所属人 ( 来源:hive)
     */
    private String tableFsOwner;

    /**
     * 参数信息 ( 来源:hive)
     */
    private String tableParametersJson;

    /**
     * 表备注 ( 来源:hive)
     */
    private String tableComment;

    /**
     * hdfs 路径 ( 来源:hive)
     */
    private String tableFsPath;

    /**
     * 输入格式( 来源:hive)
     */
    private String tableInputFormat;

    /**
     * 输出格式 ( 来源:hive)
     */
    private String tableOutputFormat;

    /**
     * 行格式 ( 来源:hive)
     */
    private String tableRowFormatSerde;

    /**
     * 表创建时间 ( 来源:hive)
     */
    private Date tableCreateTime;

    /**
     * 表类型 ( 来源:hive)
     */
    private String tableType;

    /**
     * 分桶列 ( 来源:hive)
     */
    private String tableBucketColsJson;

    /**
     * 分桶个数 ( 来源:hive)
     */
    private Long tableBucketNum;
```

```
/**
 * 排序列 (来源:hive)
 */
private String tableSortColsJson;

/**
 * 数据量大小 (来源:hdfs)
 */
private Long tableSize=0L;

/**
 * 所有副本数据总量大小 (来源:hdfs)
 */
private Long tableTotalSize=0L;

/**
 * 最后修改时间 (来源:hdfs)
 */
private Date tableLastModifyTime;

/**
 * 最后访问时间
 */
private Date tableLastAccessTime;

/**
 * 当前文件系统容量 (来源:hdfs)
 */
private Long fsCapacitySize;

/**
 * 当前文件系统使用量 (来源:hdfs)
 */
private Long fsUsedSize;

/**
 * 当前文件系统剩余量 (来源:hdfs)
 */
private Long fsRemainSize;

/**
 * 考评日期
 */
private String assessDate;

/**
 * 创建时间 (自动生成)
 */
private Date createTime;

/**
 * 更新时间 (自动生成)
 */
private Date updateTime;
}
```

6.4.3 Mapper 接口

```
package com.atguigu.dga.meta.mapper;

import com.atguigu.dga.meta.bean.TableMetaInfo;
import com.baomidou.MyBatisplus.core.mapper.BaseMapper;
import org.apache.ibatis.annotations.Mapper;

/**
 * <p>
 * 元数据表 Mapper 接口
 * </p>
 *
 * @author weiyunhui
 * @since 2023-04-12
 */
@Mapper
public interface TableMetaInfoMapper extends BaseMapper<TableMetaInfo> {

}
```

6.4.4 服务实现类

```
/**
 * <p>
 * 元数据表 服务实现类
 * </p>
 *
 * @author weiyunhui
 * @since 2023-04-12
 */
@Service
@DS("dga")
public class TableMetaInfoServiceImpl extends ServiceImpl<TableMetaInfoMapper,
TableMetaInfo> implements TableMetaInfoService {

}
```

6.4.5 服务实现接口类

```
/**
 * <p>
 * 元数据表 服务类
 * </p>
 *
 * @author weiyunhui
 * @since 2023-04-12
 */
public interface TableMetaInfoService extends IService<TableMetaInfo> {

}
```

6.4.6 控制类

```
@RestController
@RequestMapping("/tableMetaInfo")
public class TableMetaInfoController {}
```

6.5 从 Hive 中查询表名列表

在 java 程序中读取 Hive 元数据，有两种办法一种是直接通过 jdbc 查询 hive 的 metastore 元数据库。另一种办法是使用 HiveClient 通过方法获得 Hive 的 Java 表对象。

本项目推荐使用 HiveClient，主要是原因是 HiveClient 的访问形式更加友好，而直接

[更多 Java-大数据-前端-python 人工智能资料下载，可百度访问：尚硅谷官网](#)

查询 metastore 更加晦涩。

6.5.1 pom.xml 加载依赖

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jdbc</artifactId>
  </dependency>
  <dependency>
    <groupId>org.mybatis.spring.boot</groupId>
    <artifactId>mybatis-spring-boot-starter</artifactId>
    <version>3.0.0</version>
  </dependency>

  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>com.baomidou</groupId>
    <artifactId>mybatis-plus-boot-starter</artifactId>
    <version>3.4.1</version>
  </dependency>

  <dependency>
    <groupId>org.apache.commons</groupId>
    <artifactId>commons-lang3</artifactId>
  </dependency>

  <dependency>
    <groupId>com.baomidou</groupId>
    <artifactId>dynamic-datasource-spring-boot-starter</artifactId>
    <version>3.3.2</version>
    <scope>compile</scope>
  </dependency>

  <dependency>
    <groupId>com.alibaba</groupId>
    <artifactId>fastjson</artifactId>
    <version>1.2.83</version>
  </dependency>

  <dependency>
    <groupId>com.baomidou</groupId>
```

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
<artifactId>mybatis-plus-generator</artifactId>
<version>3.5.3.1</version>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-freemarker</artifactId>
</dependency>

<!-- https://mvnrepository.com/artifact/com.alibaba/druid-spring-boot-starter -->
<dependency>
    <groupId>com.alibaba</groupId>
    <artifactId>druid-spring-boot-starter</artifactId>
    <version>1.2.15</version>
</dependency>

<dependency>
    <groupId>org.apache.hive</groupId>
    <artifactId>hive-metastore</artifactId>
    <version>3.1.3</version>
</dependency>
<dependency>
    <groupId>org.apache.hive</groupId>
    <artifactId>hive-common</artifactId>
    <version>3.1.3</version>
</dependency>

<dependency>
    <groupId>org.apache.hadoop</groupId>
    <artifactId>hadoop-mapreduce-client-core</artifactId>
    <version>3.3.4</version>
</dependency>

<dependency>
    <groupId>org.apache.hadoop</groupId>
    <artifactId>hadoop-common</artifactId>
    <version>3.3.4</version>
</dependency>

<dependency>
    <groupId>commons-beanutils</groupId>
    <artifactId>commons-beanutils</artifactId>
    <version>1.9.4</version>
</dependency>

<dependency>
    <groupId>org.apache.hive</groupId>
    <artifactId>hive-exec</artifactId>
    <version>3.1.3</version>
    <exclusions>
        <exclusion>
            <artifactId>hadoop-annotations</artifactId>
            <groupId>org.apache.hadoop</groupId>
        </exclusion>
        <exclusion>
            <artifactId>hadoop-yarn-server-common</artifactId>
            <groupId>org.apache.hadoop</groupId>
        </exclusion>
        <exclusion>
            <artifactId>hadoop-yarn-server-resourcemanager</artifactId>
            <groupId>org.apache.hadoop</groupId>
        </exclusion>
    </exclusions>
</dependency>
```

```
</exclusion>
<exclusion>
    <artifactId>hadoop-yarn-server-applicationhistoryservice</artifactId>
    <groupId>org.apache.hadoop</groupId>
</exclusion>
<exclusion>
    <artifactId>hadoop-yarn-server-web-proxy</artifactId>
    <groupId>org.apache.hadoop</groupId>
</exclusion>
<exclusion>
    <artifactId>hadoop-yarn-common</artifactId>
    <groupId>org.apache.hadoop</groupId>
</exclusion>
<exclusion>
    <artifactId>jackson-core-asl</artifactId>
    <groupId>org.codehaus.jackson</groupId>
</exclusion>
<exclusion>
    <artifactId>jackson-mapper-asl</artifactId>
    <groupId>org.codehaus.jackson</groupId>
</exclusion>
<exclusion>
    <artifactId>jersey-core</artifactId>
    <groupId>com.sun.jersey</groupId>
</exclusion>
<exclusion>
    <artifactId>jersey-server</artifactId>
    <groupId>com.sun.jersey</groupId>
</exclusion>
<exclusion>
    <artifactId>zookeeper</artifactId>
    <groupId>org.apache.zookeeper</groupId>
</exclusion>
</exclusions>
</dependency>
</dependencies>
```

6.5.2 在 hadoop 集群中启动 metastore server

```
nohup hive --service metastore &
```

6.5.3 在 application.properties 增加配置

```
hive.metastore.server.url=thrift://hadoop102:9083
```

6.5.4 创建 HiveClient

```
@Service
@DS("dga")
public class TableMetaInfoServiceImpl extends ServiceImpl<TableMetaInfoMapper,
TableMetaInfo> implements TableMetaInfoService {
    IMetaStoreClient hiveClient = null ;
    //从配置文件中读取配置，赋值到属性上
    @Value("${hive.metastore.server.url}")
    String metaStoreUrl = null ;

    /**
     * 创建 MetaStoreClient
     */
    @PostConstruct
    private void getHiveClient(){
        HiveConf hiveConf = new HiveConf();
        MetastoreConf.setVar(hiveConf, MetastoreConf.ConfVars.THRIFT_URIS, metaStoreUrl);
    }
}
```

```
try {
    hiveClient = new HiveMetaStoreClient(hiveConf) ;
} catch (MetaException e) {
    throw new RuntimeException(e);
}
}
```

6.6 从 Hive 查询各表的元数据

主要负责提取 hive 元数据的一些字段

```
create table table_meta_info
(
    id                bigint primary key auto_increment comment '表id',
    table_name        varchar(200) null comment '表名',
    schema_name       varchar(200) null comment '库名',
    col_name_json     varchar(2000) null comment '字段名 json (来源:hive)',
    partition_col_name_json varchar(4000) null comment '分区字段名 json(来源:hive)',
    table_fs_owner     varchar(200) null comment 'hdfs 所属人 (来源:hive)',
    table_parameters_json varchar(2000) null comment '参数信息 (来源:hive)',
    table_comment      varchar(200) null comment '表备注 (来源:hive)',
    table_fs_path      varchar(200) null comment 'hdfs 路径 (来源:hive)',
    table_input_format varchar(200) null comment '输入格式 (来源:hive)',
    table_output_format varchar(200) null comment '输出格式 (来源:hive)',
    table_row_format_serde varchar(200) null comment '行格式 (来源:hive)',
    table_create_time  varchar(200) null comment '表创建时间 (来源:hive)',
    table_type         varchar(200) null comment '表类型 (来源:hive)',
    table_bucket_cols_json varchar(200) null comment '分桶列 (来源:hive)',
    table_bucket_num   bigint null comment '分桶个数 (来源:hive)',
    table_sort_cols_json varchar(200) null comment '排序列 (来源:hive)',
    table_size         bigint null comment '数据量大小 (来源:hdfs)',
    table_total_size   bigint null comment '所有副本数据总量大小 (来源:hdfs)',
    table_last_modify_time datetime null comment '最后修改时间 (来源:hdfs)',
    table_last_access_time datetime null comment '最后访问时间 (来源:hdfs)',
    fs_capacity_size   bigint null comment '当前文件系统容量 (来源:hdfs)',
    fs_used_size       bigint null comment '当前文件系统使用量 (来源:hdfs)',
    fs_remain_size     bigint null comment '当前文件系统剩余量 (来源:hdfs)',
    assess_date        varchar(10) null comment '考评日期 ',
    create_time        datetime null comment '创建时间 (自动生成)',
    update_time        datetime null comment '更新时间 (自动生成)',
    constraint table_meta_info_pk
        unique (table_name, schema_name, assess_date)
)
comment '元数据表';
```

6.6.1 service 层接口

```
public interface TableMetaInfoService extends IService<TableMetaInfo> {
    public void initTableMeta(String databaseName);
}
```

6.6.2 服务层的查询各表元数据的方法

```
@Service
@DS("dga")
public class TableMetaInfoServiceImpl extends ServiceImpl<TableMetaInfoMapper,
TableMetaInfo> implements TableMetaInfoService {

    public void initTableMeta(String databaseName , String assessDate ) throws Exception {

        //1.2 通过 api 方法获取 hive 元数据
        // 1) 根据库名查询表名列表
```

```
List<String> allTableNameList = hiveClient.getAllTables(databaseName);
List<TableMetaInfo> tableMetaInfoList = new ArrayList<>(allTableNameList.size());
for (String tableName : allTableNameList) {
    Table table = hiveClient.getTable(databaseName, tableName);
    // 2) 提取每个表的hive 元数据
    TableMetaInfo tableMetaInfo = getTableMetaFromHive(table);
    //System.out.println("table =" + tableMetaInfo);

    //2.提取 hdfs 的元数据
    addHdfsInfo(tableMetaInfo);

    //考评时间
    tableMetaInfo.setAssessDate( assessDate);
    //创建时间
    tableMetaInfo.setCreateTime(new Date());

    //保存到集合, 稍后批写
    tableMetaInfoList.add(tableMetaInfo);
}

// 3. 批写入数据库
//写入数据之前, 先清理当日数据
remove(new QueryWrapper<TableMetaInfo>().eq("assess_date", assessDate));
//写入
saveOrUpdateBatch(tableMetaInfoList);
}

/**
 * 取 hive 中的元数据信息
 * @param table
 * @return
 */
private TableMetaInfo getTableMetaFromHive(Table table ){
    TableMetaInfo tableMetaInfo = new TableMetaInfo();
    tableMetaInfo.setTableName(table.getTableName());
    tableMetaInfo.setSchemaName(table.getDbName());
    //json 过滤
    PropertyPreFilters.MySimplePropertyPreFilter propertyFilter
        = new PropertyPreFilters().addFilter("comment", "name", "type");

    tableMetaInfo.setColNameJson(JSON.toJSONString(table.getSd().getCols(),propertyFilter));

    tableMetaInfo.setPartitionColNameJson(JSON.toJSONString(table.getPartitionKeys(),propertyFilter));

    tableMetaInfo.setTableFsOwner(table.getOwner());
    tableMetaInfo.setTableParametersJson(JSON.toJSONString(table.getParameters()));
    tableMetaInfo.setTableComment(table.getParameters().get("comment"));
    tableMetaInfo.setTableFsPath(table.getSd().getLocation());
    tableMetaInfo.setTableInputFormat(table.getSd().getInputFormat());
    tableMetaInfo.setTableOutputFormat(table.getSd().getOutputFormat());

    tableMetaInfo.setTableRowFormatSerde(table.getSd().getSerdeInfo().getSerializationLib());
    tableMetaInfo.setTableCreateTime(new Date(table.getCreateTime() * 1000));
    tableMetaInfo.setTableBucketNum(table.getSd().getNumBuckets() + 0L);
    if(tableMetaInfo.getTableBucketNum() > 0 ){
        List<String> bucketCols = table.getSd().getBucketCols();
        tableMetaInfo.setTableBucketColsJson(JSON.toJSONString(bucketCols));

    }
    tableMetaInfo.setTableSortColsJson(JSON.toJSONString(table.getSd().getSortCols()));
    tableMetaInfo.setTableType(table.getTableType());
    return tableMetaInfo;
}
```


6.7 补充 HDFS 相关信息

需要从 hdfs 提取一些数据信息

```
create table table_meta_info
(
    id                bigint primary key auto_increment comment '表id',
    table_name        varchar(200) null comment '表名',
    schema_name       varchar(200) null comment '库名',
    col_name_json     varchar(2000) null comment '字段名 json (来源:hive)',
    partition_col_name_json varchar(4000) null comment '分区字段名 json(来源:hive)',
    table_fs_owner     varchar(200) null comment 'hdfs 所属人 (来源:hive)',
    table_parameters_json varchar(2000) null comment '参数信息 (来源:hive)',
    table_comment      varchar(200) null comment '表备注 (来源:hive)',
    table_fs_path      varchar(200) null comment 'hdfs 路径 (来源:hive)',
    table_input_format varchar(200) null comment '输入格式 (来源:hive)',
    table_output_format varchar(200) null comment '输出格式 (来源:hive)',
    table_row_format_serde varchar(200) null comment '行格式 (来源:hive)',
    table_create_time  datetime null comment '表创建时间 (来源:hive)',
    table_type         varchar(200) null comment '表类型 (来源:hive)',
    table_bucket_cols_json varchar(200) null comment '分桶列 (来源:hive)',
    table_bucket_num   bigint null comment '分桶个数 (来源:hive)',
    table_sort_cols_json varchar(200) null comment '排序列 (来源:hive)',
    table_size         bigint null comment '数据量大小 (来源:hdfs)',
    table_total_size   bigint null comment '所有副本数据总量大小 (来源:hdfs)',
    table_last_modify_time datetime null comment '最后修改时间 (来源:hdfs)',
    table_last_access_time datetime null comment '最后访问时间 (来源:hdfs)',
    fs_capacity_size   bigint null comment '当前文件系统容量 (来源:hdfs)',
    fs_used_size       bigint null comment '当前文件系统使用量 (来源:hdfs)',
    fs_remain_size     bigint null comment '当前文件系统剩余量 (来源:hdfs)',
    assess_date        varchar(10) null comment '考评日期 ',
    create_time        datetime null comment '创建时间 (自动生成)',
    update_time        datetime null comment '更新时间 (自动生成)',
    constraint table_meta_info_pk
        unique (table_name, schema_name, assess_date)
)
comment '元数据表';
```

难点：这里要统计表的大小、以及数据的最后修改和访问日期，不能以文件夹为准，需要遍历文件夹以及其下的子文件夹和文件，最终汇总或者比较得出。

而文件夹这种树形结构的访问非常适合使用递归来操作遍历。

6.7.1 在服务层添加如下代码

```
/**
 * 取 hdfs 的元数据信息
 * @param tableMetaInfo
 */
private void addHdfsInfo(TableMetaInfo tableMetaInfo) throws Exception {
    // 1. 连接 hdfs 客户端 FileSystem
    FileSystem fs =
        FileSystem.get(new URI(tableMetaInfo.getTableFsPath()), new Configuration(),
            tableMetaInfo.getTableFsOwner());
    // 获取表路径下所有的文件及目录
    FileStatus[] listStatus = fs.listStatus(new Path(tableMetaInfo.getTableFsPath()));
    // 递归补充文件相关信息
    addFileInfo(listStatus, tableMetaInfo, fs);
}
```

更多 Java-大数据-前端-python 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
//System.out.println(tableMetaInfo);

//补充环境信息
tableMetaInfo.setFsCapacitySize(fs.getStatus().getCapacity());
tableMetaInfo.setFsUsedSize(fs.getStatus().getUsed());
tableMetaInfo.setFsRemainSize(fs.getStatus().getRemaining());
}
```

6.7.2 递归补充文件信息

```
/**
 * 递归
 *
 * // table_total_size    table_size * 副本系数
 *
 * // last_modify 遍历文件夹及其子目录下的所有文件最近修改时间
 *
 * // last_access 遍历文件夹及其子目录下的所有文件最近访问时间
 *
 * // 文件系统总容量 取现成
 *
 * @param listStatus
 * @param tableMetaInfo
 * @param fs
 */
private void addFileInfo(FileStatus[] listStatus, TableMetaInfo tableMetaInfo, FileSystem fs)
throws Exception {
    for (FileStatus fileStatus : listStatus) {
        if (fileStatus.isFile()) {
            //处理 , 如果是文件, 取文件的元数据信息进行累加统计
            tableMetaInfo.setTableSize(tableMetaInfo.getTableSize() +
fileStatus.getLen());

            tableMetaInfo.setTableTotalSize(tableMetaInfo.getTableTotalSize() +
fileStatus.getLen() * fileStatus.getReplication());

            long lastModifyTimeMs = Math.max(tableMetaInfo.getTableLastModifyTime() ==
null? 0:tableMetaInfo.getTableLastModifyTime().getTime(), fileStatus.getModificationTime());
            tableMetaInfo.setTableLastModifyTime( new Date(lastModifyTimeMs) );

            long lastAccessTimeMs = Math.max(tableMetaInfo.getTableLastAccessTime() ==
null? 0:tableMetaInfo.getTableLastAccessTime().getTime(), fileStatus.getAccessTime());
            tableMetaInfo.setTableLastAccessTime( new Date(lastAccessTimeMs) );

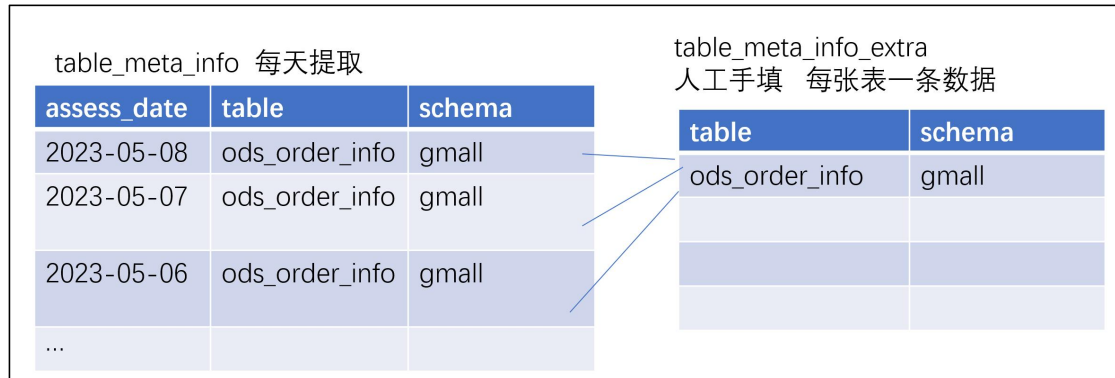
        }else{
            //下探 , 如果是目录, 继续递归,
            addFileInfo(fs.listStatus(fileStatus.getPath()) , tableMetaInfo , fs);
        }
    }
}
```

6.8 生成辅助信息

辅助信息本身也是一种数据表的元数据, 但是与从 hive 和 hdfs 提取的信息不同, 辅助信息一般是由数据开发人员手工维护的。

在从 hive 或者 hdfs 提取数据后, 如果该表是首次提取, 可以初始化写如一条大部分为空值的辅助信息, 以便日后由数据开发人员进行补充。

更多 Java –大数据 –前端 –python 人工智能资料下载, 可百度访问: [尚硅谷官网](http://www.shang硅谷.com)



table_meta_info 与 table_meta_info_extra 是一对多的关系

6.8.1 辅助信息的表结构

```
create table if not exists dga.table_meta_info_extra
(
    id                bigint primary key auto_increment comment '表 id',
    table_name        varchar(200) null comment '表名',
    schema_name       varchar(200) null comment '库名',
    tec_owner_user_name varchar(20) null comment '技术负责人',
    busi_owner_user_name varchar(20) null comment '业务负责人',
    lifecycle_type     varchar(20) null comment '存储周期类型',
    lifecycle_days     bigint null comment '生命周期(天)',
    security_level     varchar(20) null comment '安全级别',
    dw_level          varchar(20) null comment '数仓所在层级(ODS DWD DIM DWS ADS)',
    create_time        datetime null comment '创建时间 (自动生成)',
    update_time        datetime null comment '更新时间 (自动生成)',
    constraint table_meta_info_extra_pk
        unique (table_name, schema_name)
)
comment '元数据表附加信息';
```

6.8.2 实体 bean

```
package com.atguigu.dga.meta.bean;

import com.baomidou.MyBatisplus.annotation.IdType;
import java.util.Date;
import com.baomidou.MyBatisplus.annotation.TableId;
import lombok.Data;

import java.io.Serializable;

/**
 * <p>
 * 元数据表附加信息
 * </p>
 *
 * @author weiyunhui
 * @since 2023-04-14
 */
@Data
public class TableMetaInfoExtra implements Serializable {

    private static final long serialVersionUID = 1L;

    /**
     * 表 id
     */
    @TableId
    @IdType(IdType.AUTO)
```

更多 Java-大数据-前端-python 人工智能资料下载，可百度访问：尚硅谷官网

```
    */
    @TableId(value = "id", type = IdType.AUTO)
    private Long id;

    /**
     * 表名
     */
    private String tableName;

    /**
     * 库名
     */
    private String schemaName;

    /**
     * 技术负责人 （来源：附加）
     */
    private String tecOwnerUserName;

    /**
     * 业务负责人 （来源：附加）
     */
    private String busiOwnerUserName;

    /**
     * 生命周期类型（来源：附加）
     */
    private String lifecycleType;

    /**
     * 生命周期(天)（来源：附加）
     */
    private Long lifecycleDays;

    /**
     * 安全级别（来源：附加）
     */
    private String securityLevel;

    /**
     * 数仓所在层级(ODSDWDDIMDWSADS)（来源：附加）
     */
    private String dwLevel;

    /**
     * 创建时间（自动生成）
     */
    private Date createTime;

    /**
     * 更新时间（自动生成）
     */
    private Date updateTime;
}
```

6.8.3 服务层增加类

6.8.3.1 增加接口类 TableMetaInfoExtraService

```
public interface TableMetaInfoExtraService extends IService<TableMetaInfoExtra> {
```

更多 Java-大数据-前端-python 人工智能资料下载，可百度访问：[尚硅谷官网](http://www.shang硅谷.com)

```
public void initTableMetaExtra(List<TableMetaInfo> tableMetaInfoList);  
}
```

6.8.3.2 增加实现类 TableMetaInfoExtraServiceImpl

```
/**  
 * <p>  
 * 元数据表附加信息 服务实现类  
 * </p>  
 *  
 * @author weiyunhui  
 * @since 2023-07-28  
 */  
@Service  
@DS("dga")  
public class TableMetaInfoExtraServiceImpl extends ServiceImpl<TableMetaInfoExtraMapper,  
TableMetaInfoExtra> implements TableMetaInfoExtraService {  
  
    /**  
     * 初始化表辅助信息  
     */  
    @Override  
    public void initTableMetaExtra(List<TableMetaInfo> tableMetaInfoList) {  
        List<TableMetaInfoExtra> tableMetaInfoExtraList = new  
ArrayList<>(tableMetaInfoList.size());  
  
        for (TableMetaInfo tableMetaInfo : tableMetaInfoList) {  
            // 先查询表辅助信息是否存在  
            TableMetaInfoExtra tableMetaInfoExtra = getOne(  
                new QueryWrapper<TableMetaInfoExtra>()  
                    .eq("schema_name", tableMetaInfo.getSchemaName())  
                    .eq("table_name", tableMetaInfo.getTableName());  
            // 如果不存在, 则进行初始化  
            if (tableMetaInfoExtra == null) {  
                tableMetaInfoExtra = new TableMetaInfoExtra();  
                tableMetaInfoExtra.setSchemaName(tableMetaInfo.getSchemaName());  
                tableMetaInfoExtra.setTableName(tableMetaInfo.getTableName());  
                tableMetaInfoExtra.setSecurityLevel(MetaConst.SECURITY_LEVEL_UNSET);  
                tableMetaInfoExtra.setLifecycleType(MetaConst.LIFECYCLE_TYPE_UNSET);  
  
                // 正常系统中, 需要给 用户的 ID  
                tableMetaInfoExtra.setBusiOwnerUserName(null);  
                tableMetaInfoExtra.setTecOwnerUserName(null);  
  
                tableMetaInfoExtra.setLifecycleDays(-1L);  
  
                tableMetaInfoExtra.setDwLevel(getTableLevelByName(tableMetaInfoExtra.getTableName()));  
  
                tableMetaInfoExtra.setCreateTime(new Date());  
                // 保存到集合中, 做批写  
                tableMetaInfoExtraList.add(tableMetaInfoExtra);  
            }  
  
            // 存储  
            saveOrUpdateBatch(tableMetaInfoExtraList);  
        }  
    }  
  
    private String getTableLevelByName(String tableName) {  
        if (tableName.startsWith("ods")) {  
            return "ODS";  
        } else if (tableName.startsWith("dwd")) {  

```

```
        return "DWD";
    }else if (tableName.startsWith("dim")) {
        return "DIM";
    }else if (tableName.startsWith("dws")) {
        return "DWS";
    }else if (tableName.startsWith("ads")) {
        return "ADS";
    }else if (tableName.startsWith("dm")) {
        return "DM";
    }else {
        return "OTHER";
    }
}
}
```

6.8.3.3 增加常量类

```
public class MetaConst {

    public static final String LIFECYCLE_TYPE_PERM="PERM";    //永久
    public static final String LIFECYCLE_TYPE_ZIP="ZIP";      //拉链
    public static final String LIFECYCLE_TYPE_DAY="DAY";      //日分区
    public static final String LIFECYCLE_TYPE_OTHER="OTHER";  //其他
    public static final String LIFECYCLE_TYPE_UNSET="UNSET";  //未设置

    public static final String SECURITY_LEVEL_UNSET="UNSET";  //未设置
    public static final String SECURITY_LEVEL_PUBLIC="PUBLIC"; //公开
    public static final String SECURITY_LEVEL_INTERNAL="INTERNAL"; //内部
    public static final String SECURITY_LEVEL_SECRET="SECRET"; //保密
    public static final String SECURITY_LEVEL_HIGH="HIGH";    //高度机密
}
```

6.8.4 数据层增加类

数据层不用额外增加，需要的方法 MyBatis-Plus 都有。

```
package com.atguigu.dga.meta.mapper;

import com.atguigu.dga.meta.bean.TableMetaInfoExtra;
import com.baomidou.MyBatisplus.core.mapper.BaseMapper;
import org.apache.ibatis.annotations.Mapper;

/**
 * <p>
 * 元数据表附加信息 Mapper 接口
 * </p>
 *
 * @author weiyunhui
 * @since 2023-04-14
 */
@Mapper
public interface TableMetaInfoExtraMapper extends BaseMapper<TableMetaInfoExtra> {

}
```

6.8.5 在 TableMetaInfoServiceImpl 中调用

```
public void initTableMeta(String databaseName , String assessDate ) throws Exception {

    //1.2 通过 api 方法获取 hive 元数据
    // 1) 根据库名查询表名列表
    List<String> allTableNameList = hiveClient.getAllTables(databaseName);
    List<TableMetaInfo> tableMetaInfoList = new ArrayList<>(allTableNameList.size()) ;
```

更多 Java –大数据 –前端 –python 人工智能资料下载，可百度访问：[尚硅谷官网](http://www.shangguigu.com)

```

for (String tableName : allTableNameList) {
    Table table = hiveClient.getTable(databaseName , tableName) ;
    // 2) 提取每个表的hive 元数据
    TableMetaInfo tableMetaInfo = getTableMetaFromHive(table) ;
    //System.out.println("table =" + tableMetaInfo);

    //2.提取 hdfs 的元数据
    addHdfsInfo(tableMetaInfo) ;

    //考评时间
    tableMetaInfo.setAssessDate( assessDate);
    //创建时间
    tableMetaInfo.setCreateTime(new Date());

    //保存到集合，稍后批写
    tableMetaInfoList.add(tableMetaInfo);
}

// 3. 批写入数据库
//写入数据之前， 先清理当日数据
remove(new QueryWrapper<TableMetaInfo>().eq("assess_date" , assessDate)) ;
//写入
saveOrUpdateBatch(tableMetaInfoList) ;
//4. 初始化元数据辅助信息
tableMetaInfoExtraService.initTableMetaExtra(tableMetaInfoList);
}

```

第 7 章 任务二：各表元数据的页面接口

7.1 接口梳理

数据治理平台的一个要务就是要将后台的各种数据表，整齐的直观呈现出来，方便数据开发人员了解每张表的内容和定位。

页面可视化是必备的功能，作为数据开发人员，至少要掌握如何提供数据 web 服务接口提供给前端页面。

接口名称	接口路径	请求方式
表信息查询列表	/tableMetaInfo/table-list	GET
单表信息详情	/tableMetaInfo/table/{tableMetaInfoId}	GET
单表辅助信息保存	/tableMetaInfo/tableExtra	POST
手动更新全表元数据	/tableMetaInfo/init-tables/{database}/{assessDate}	POST

7.2 表信息查询列表

7.2.1 需求

为前端提供页面查询的数据接口，根据给定的条件及分页参数查询列表。

库名

表名

层级

ads层

查询

清空

手动更新全库元数据

表名	库名	表备注	单副本数据量	全部副本总数据量	技术负责人	业务负责人	最后访问时间	数据最后更新时间
ads_traffic_stats_by_channel	gmail	各渠道流量统计	2.2KB	6.6KB	张晨	张晨	2023-02-07 17:02	2023-02-07 17:02
ads_page_path	gmail	页面浏览路径分析	8.0KB	24.0KB			2023-04-11 15:04	2023-02-07 17:02
ads_user_change	gmail	用户变动统计	45B	0.1KB			2023-02-07 17:02	2023-02-07 17:02
ads_user_retention	gmail	用户留存率	0.5KB	1.5KB			2023-02-07 17:02	2023-02-07 17:02
ads_user_stats	gmail	用户新增活跃统计	0.2KB	0.5KB			2023-02-07 17:02	2023-02-07 17:02

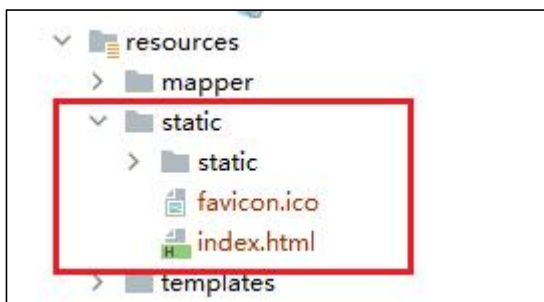
5条/页

< 1 2 3 4 >

前往 1 页

7.2.2 前端页面使用

1) 拷贝资料中的【前端页面】到 resource 目录中。



2) 修改 windows 的 hosts 增加地址映射

```
127.0.0.1 dga.gmall.com
```

3) application.properties 注意要设 80 端口

```
server.port=80
```

7.2.3 接口具体要求

梳理前端页面需要的接口

接口名称	表信息查询
接口路径	/tableMetaInfo/table-list
请求方式	GET
接口参数	pageNo, pageSize, tableName, schemaName, dwLevel
返回值	{ "total": 79, "list": [{ "id": 1186, "tableName": "ads_traffic_stats_by_channel", "schemaName": "gmall", "tableSize": 2254,


```
"tableTotalSize": 6762,
"tableComment": "各渠道流量统计",
"tecOwnerUserName": "weiyunhui",
"busiOwnerUserName": "weiyunhui",
"tableLastModifyTime": "2023-02-07T09:52:58.000+00:00",
"tableLastAccessTime": "2023-02-07T09:52:58.000+00:00"
},
{ "id": 1187,
  ....
}
]
}
```

7.2.4 代码清单

类名	说明
TableMetaInfoForQuery	用户传递页面的条件参数及分页参数
TableMetaInfoVO	用于返回页面列表结果
TableMetaInfoServiceImpl	服务层，主要逻辑，增加查询方法
TableMetaInfoService	服务层接口，增加查询方法声明
TableMetaInfoMapper	数据层，增加查询列表的方法
TableMetaInfoController	控制层，列表查询 web 接口

7.2.5 增加实体参数类

TableMetaInfoForQuery

```
@Data
public class TableMetaInfoQuery {
    private String schemaName ;
    private String tableName;
    private String dwLevel ;
    private Integer pageSize ;
    private Integer pageNo ;
}
```

7.2.6 增加实体列表结果类

VO 类主要用于返回页面的数据对象,即 View Object

```
@Data
public class TableMetaInfoVO {
    /**
     * 表 id
     */
    private Long id;

    /**
     * 表名
     */
}
```

```
private String tableName;

/**
 * 库名
 */
private String schemaName;

/**
 * 数据量大小 (来源:hdfs)
 */
private Long tableSize=0L;

/**
 * 所有副本数据总量大小 (来源:hdfs)
 */
private Long tableTotalSize=0L;

/**
 * 表备注 (来源:hive)
 */
private String tableComment;

/**
 * 技术负责人 (来源: 附加)
 */
private String tecOwnerUserName;

/**
 * 业务负责人 (来源: 附加)
 */
private String busiOwnerUserName;

/**
 * 最后修改时间 (来源:hdfs)
 */
private Date tableLastModifyTime;

/**
 * 最后访问时间
 */
private Date tableLastAccessTime;
}
```

7.2.7 控制层查询方法

```
@GetMapping("/table-list")
public String getTableMetaList(TableMetaInfoQuery tableMetaInfoQuery){
    System.out.println(tableMetaInfoQuery);
    // 查询分页明细
    List<TableMetaInfoVO> tableMetaInfoVOList =
tableMetaInfoService.getTableMetaListForPage(tableMetaInfoQuery);
    // 查询总数
    Integer total = tableMetaInfoService.getTableMetaListCount(tableMetaInfoQuery) ;

    //封装返回数据
    JSONObject jsonObject = new JSONObject();
    jsonObject.put("total" , total) ;
    jsonObject.put("list" , tableMetaInfoVOList) ;
}
```

```
return jsonObject.toJSONString() ;  
}
```

7.2.8 服务层增加方法

该层需要增加两个方法一个是根据条件查询列表，另一个是查询总数。

1) 接口类 TableMetaInfoService 增加

```
List<TableMetaInfoVO> getTableMetaListForPage(TableMetaInfoQuery tableMetaInfoQuery);  
  
Integer getTableMetaListCount(TableMetaInfoQuery tableMetaInfoQuery);
```

2) 实现类 TableMetaInfoServiceImpl 增加方法:

根据条件查询列表

```
/**  
 * 根据条件和分页信息查询 TableMetaInfo  
 * @param tableMetaInfoQuery  
 * @return  
 */  
public List<TableMetaInfoVO> getTableMetaListForPage(TableMetaInfoQuery tableMetaInfoQuery)  
{  
    // 拼接 SQL  
    // select from table_meta_info tm join table_meta_info_extra te  
    // on tm.schema_name = te.schema_name and tm.table_name = te.table_name  
    // and assess_date = (select max(assess_date) from table_meta_info)  
    // and schema_name like '%xxx%'  
    // and tm.table_name like '%xx%'  
    // and te.dw_level = 'xx'  
    // limit 1, pageSize 利用 pageNo pageSize 换算  
  
    StringBuilder sql = new StringBuilder(500);  
    sql.append(" select tm.id , tm.table_name , tm.schema_name , table_comment ,  
table_size , " +  
        " table_total_size , tec_owner_user_name ,  
busi_owner_user_name ,table_last_access_time , " +  
        " table_last_modify_time from table_meta_info tm join table_meta_info_extra  
te " +  
        " on tm.schema_name = te.schema_name and tm.table_name = te.table_name " +  
        " and assess_date = (select max(assess_date) from table_meta_info) ");  
    // 判断是否有 schema_name 条件  
    if(tableMetaInfoQuery.getSchemaName() !=null &&  
tableMetaInfoQuery.getSchemaName().length() > 0 ){  
        sql.append(" and tm.schema_name like '%" +  
SqlUtil.filterUnsafeSql(tableMetaInfoQuery.getSchemaName()) + "%' ");  
    }  
    //判断是否有 table_name 条件  
    if(tableMetaInfoQuery.getTableName() !=null &&  
tableMetaInfoQuery.getTableName().length() > 0 ){  
        sql.append(" and tm.table_name like '%" +  
SqlUtil.filterUnsafeSql(tableMetaInfoQuery.getTableName()) + "%' ");  
    }  
    //判断是否有 dw_level 条件  
    if(tableMetaInfoQuery.getDwLevel() !=null &&  
tableMetaInfoQuery.getDwLevel().length() > 0 ){  
        sql.append(" and te.dw_level = '" +  
SqlUtil.filterUnsafeSql(tableMetaInfoQuery.getDwLevel()) + "' ");  
    }  
  
    //分页  
    //计算开始行
```

更多 Java-大数据-前端-python 人工智能资料下载，可百度访问：尚硅谷官网

```
Integer pageStart = (tableMetaInfoQuery.getPageNo() - 1 ) *
tableMetaInfoQuery.getPageSize() ;
sql.append(" limit " + pageStart + " , " + tableMetaInfoQuery.getPageSize() ) ;

System.out.println(sql.toString());

List<TableMetaInfoVO> tableMetaInfoVOList =
getBaseMapper().selectTableMetaInfoForPage(sql.toString());

return tableMetaInfoVOList ;
}
```

根据条件获取总记录数

```
/**
 * 根据条件查询 TableMetaInfo 总数
 * @param tableMetaInfoQuery
 * @return
 */
@Override
public Integer getTableMetaListCount(TableMetaInfoQuery tableMetaInfoQuery) {

    StringBuilder sql = new StringBuilder(500);
    sql.append(" select count(*) cnt from table_meta_info tm join table_meta_info_extra
te " +
        " on tm.schema_name = te.schema_name and tm.table_name = te.table_name " +
        " and assess_date = (select max(assess_date) from table_meta_info) ");
    // 判断是否有 schema_name 条件
    if(tableMetaInfoQuery.getSchemaName() !=null &&
tableMetaInfoQuery.getSchemaName().length() > 0 ){
        sql.append(" and tm.schema_name like '%" +
SqlUtil.filterUnsafeSql(tableMetaInfoQuery.getSchemaName()) + "%' ");
    }
    //判断是否有 table_name 条件
    if(tableMetaInfoQuery.getTableName() !=null &&
tableMetaInfoQuery.getTableName().length() > 0 ){
        sql.append(" and tm.table_name like '%" +
SqlUtil.filterUnsafeSql(tableMetaInfoQuery.getTableName()) + "%' ");
    }
    //判断是否有 dw_level 条件
    if(tableMetaInfoQuery.getDwLevel() !=null &&
tableMetaInfoQuery.getDwLevel().length() > 0 ){
        sql.append(" and te.dw_level = '" +
SqlUtil.filterUnsafeSql(tableMetaInfoQuery.getDwLevel()) + "' ");
    }

    Integer count = getBaseMapper().selectTableMetaInfoCount(sql.toString());

    return count;
}
```

为避免 SQL 注入问题在 util 包下增加工具类

```
public class SqlUtil {

    public static String filterUnsafeSql(String input) {
        if (input == null) {
            return null;
        }

        // 替换 MySQL 中可能导致 SQL 注入的特殊字符
        return input.replace("\\", "\\")
            .replace("'", "\\'")
            .replace("\"", "\\\"")
    }
}
```

更多 Java-大数据-前端-python 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```

        .replace("\\b", "\\b")
        .replace("\\n", "\\n")
        .replace("\\r", "\\r")
        .replace("\\t", "\\t")
        .replace("\\u001A", "\\Z")
        .replace("%", "\\%")
        .replace("_", "\\_");
    }
}

```

7.2.9 增加数据层方法

把服务拼接好的 sql 传入，因为\${} 有 sql 注入风险，需要在拼接 sql 时处理。

```

@Mapper
@DS("dga")
public interface TableMetaInfoMapper extends BaseMapper<TableMetaInfo> {

    @Select("${sql}")
    public List<TableMetaInfoVO> selectTableMetaInfoForPage(@Param("sql") String sql );

    @Select("${sql}")
    public Integer selectTableMetaInfoCount(@Param("sql") String sql );
}

```

7.3 单表信息详情

7.3.1 需求分析

为某张表提供元数据的详情展示

任务信息	
表名	ads_traffic_stats_by_channel
库名	gmall
辅助信息	
技术负责人	张磊
业务负责人	张磊
生命周期(天)	12
安全级别	1
数仓所在层级	ADS
元数据抓取时间	2023-04-15 15:04
表信息	
字段名json	[{"comment":"统计日期","name":"dt","type":"string"}, {"comment":"最近天数,1:最近1天,7:最近7天,30:最近30天","name":"recent_days","type":"bigint"}, {"comment":"渠道","name":"channel","type":"string"}, {"comment":"访客id","name":"visitor_id","type":"string"}]
分区字段名	dt
hdfs所属人	atguigu
参数信息	{"totalSize":"2254","EXTERNAL":"TRUE","numFiles":"2","transient_lastDdlTime":"1680235106","bucketing_version":"2","comment":"各渠道流量统计"}

7.3.2 接口具体要求

接口名称	单表信息详情
接口路径	/tableMetaInfo/table/{tableMetaInfoId}
请求方式	GET
接口参数	请求路径中的 tableMetaInfoId
返回值	{ "id":1186,"tableName":"ads_traffic_stats_by_channel","schemaName":"gmall","colNameJson":'[{"comment":"统计日期","name":"dt","type":"string"}, {"comment":"最近天数,1:最近1天,7:最近7天,30:最近30天","name":"recent_days","type":"bigint"}, {"comment":"渠道","name":"channel","type":"string"}, {"comment":"访客id","name":"visitor_id","type":"string"}]', "partitionField":"dt", "owner":"atguigu", "parameters":{"totalSize":"2254","EXTERNAL":"TRUE","numFiles":"2","transient_lastDdlTime":"1680235106","bucketing_version":"2","comment":"各渠道流量统计"} }

更多 Java –大数据 –前端 –python 人工智能资料下载，可百度访问：尚硅谷官网

```

{"name":"channel","type":"string"}, {"comment":"访客人数"}
{"name":"uv_count","type":"bigint"}, {"comment":"会话平均停留时长，单位为秒"}
{"name":"avg_duration_sec","type":"bigint"}, {"comment":"会话平均浏览页面数"}
{"name":"avg_page_count","type":"bigint"}, {"comment":"会话数"}
{"name":"sv_count","type":"bigint"}, {"comment":"跳出率"}
{"name":"bounce_rate","type":"decimal(16,2)"}], "partitionColNameJson": "[]", "tableFsOwner": "atguigu", "tableParametersJson": {"totalSize": "2254", "EXTERNAL": "TRUE", "numFiles": "2", "transient_lastDdlTime": "1680235106", "bucketing_version": "2", "comment": "各渠道流量统计"}, "tableComment": "各渠道流量统计", "tableFsPath": "hdfs://hadoop102:8020/warehouse/gmall/ads/ads_traffic_stats_by_channel", "tableInputFormat": "org.apache.hadoop.mapred.TextInputFormat", "tableOutputFormat": "org.apache.hadoop.mapred.TextInputFormat", "tableRowFormatSerde": "org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe", "tableCreateTime": "2023-03-31T03:58:26.000+00:00", "tableType": "EXTERNAL_TABLE", "tableBucketColsJson": null, "tableBucketNum": -1, "tableSortColsJson": null, "tableSize": 2254, "tableTotalSize": 6762, "tableLastModifyTime": "2023-02-07T09:52:58.000+00:00", "tableLastAccessTime": "2023-02-07T09:52:58.000+00:00", "fsCapacitySize": 141652144128, "fsUsedSize": 10058940416, "fsRemainSize": 84828000256, "assessDate": "2023-04-15", "createTime": "2023-04-15T07:35:05.000+00:00", "updateTime": null, "tableMetaInfoExtra": {"id": 1, "tableName": "ads_traffic_stats_by_channel", "schemaName": "gmall", "tecOwnerUserName": "weiyunhui", "busiOwnerUserName": "weiyunhui", "lifecycleDays": 12, "securityLevel": "1", "dwLevel": "ADS", "createTime": "2023-04-15T07:35:09.000+00:00", "updateTime": null}}

```

7.3.3 代码清单

类名	说明
TableMetaInfoController	控制层，单表查询 web 接口

7.3.4 实体类增加属性

```

@Data
public class TableMetaInfo implements Serializable {

    private static final long serialVersionUID = 1L;

    /**
     * 表 id
     */
    @TableId(value = "id", type = IdType.AUTO)
    private Long id;

    .....

    @TableField(exist = false)
    private TableMetaInfoExtra tableMetaInfoExtra;
}

```

注意：此处增加了注解 `@TableField(exist = false)`，目的是避免 MyBatis-Plus 与数据库字段映射时发生误解，因为该属性只是用来传递参数，数据库中并没有该字段，特此声明。

7.3.5 控制层查询方法

```

@GetMapping("/table/{id}")
public String getTableMetaInfoById(@PathVariable("id") Long id) {

```

更多 Java – 大数据 – 前端 – python 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```

TableMetaInfo tableMetaInfo = tableMetaInfoService.getById(id);
TableMetaInfoExtra tableMetaInfoExtra = tableMetaInfoExtraService.getOne(
    new QueryWrapper<TableMetaInfoExtra>()
        .eq("schema_name", tableMetaInfo.getSchemaName())
        .eq("table_name", tableMetaInfo.getTableInfo().getTableName())
);
tableMetaInfo.setTableMetaInfoExtra(tableMetaInfoExtra);

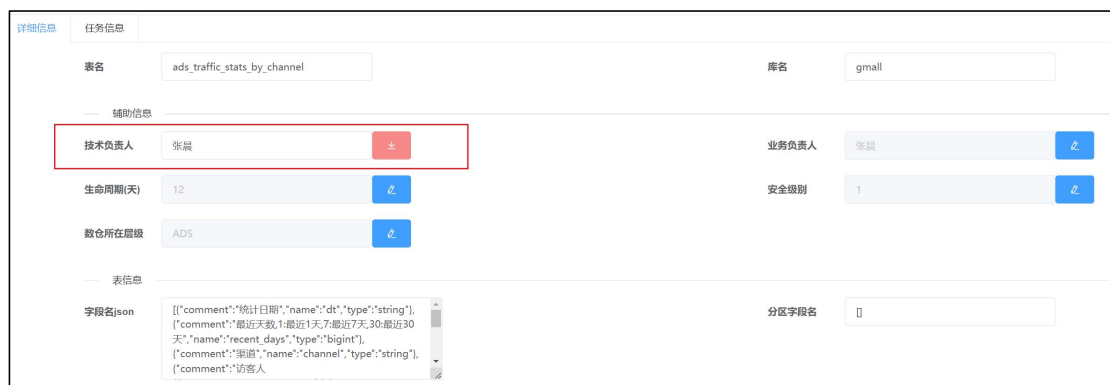
return JSON.toJSONString(tableMetaInfo);
}

```

7.4 辅助信息保存接口

7.4.1 需求分析

在页面中需要用户动手维护辅助信息，点击保存按钮后把辅助信息提交到后台保存。



7.4.2 接口具体需求

接口名称	辅助信息保存接口
接口路径	/tableMetaInfo/tableExtra
请求方式	POST
接口参数	<pre> { "id": 1, "tableName": "ads_traffic_stats_by_channel", "schemaName": "gmail", "tecOwnerUserName": "weiyunhui", "busiOwnerUserName": "weiyunhui", "lifecycleDays": 12, "securityLevel": "1", "dwLevel": "ADS", "createTime": "2023-04-15T07:35:09.000+00:00", "updateTime": null } </pre>
返回值	保存成功后返回 "success" 即可。

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网

7.4.3 代码清单

类名	说明
TableMetaInfoController	控制层，单表查询 web 接口

7.4.4 控制层保存辅助信息方法

在 TableMetaInfoController

```
@PostMapping("/tableExtra")
public String saveTableMetaInfoExtra(@RequestBody TableMetaInfoExtra tableMetaInfoExtra) {
    tableMetaInfoExtraService.saveOrUpdate(tableMetaInfoExtra) ;
    return "success";
}
```

7.5 手动更新全库元数据

7.5.1 需求分析

在页面中支持用户手动更新全表元数据，输入库名和考评日期点击确定完成更新。

7.5.2 接口具体需求

接口名称	辅助信息保存接口
接口路径	/tableMetaInfo/init-tables/{database}/{assessdate}
请求方式	POST
接口参数	无
返回值	保存成功后返回 "success" 即可。

7.5.3 代码清单

类名	说明
TableMetaInfoController	控制层，手动更新全表元数据方法

7.5.4 控制层手动更新全表元数据方法

在 TableMetaInfoController 中添加方法

```
@PostMapping("init-tables/{schemaName}/{dt}")
public String initTableMeta(@PathVariable("schemaName") String schemaName ,
    @PathVariable("dt") String dt ) throws Exception {
    tableMetaInfoService.initTableMeta(schemaName , dt);
    return "success" ;
}
```


第 8 章 任务三：数据治理考评

8.1 考评指标分析

考评板块	考评指标	考评标准
规范	是否有技术 owner	有 则 10 分，无 则 0 分
	是否有业务 owner	有 则 10 分，无 则 0 分
	表名是否合规	参考建数仓表规范 ODS 层： 开头:ods 结尾 :inc/full 结构 ods_xx_(inc full) DIM 层： dim 开头 full/zip 结尾 结构: dim_xx_(zip full) DWD 层: dwd 开头 inc/full 结尾 结构: dwd_xx_xx_(inc full) DWS 层: dws 开头 结构 dws_xx_xx_xx_(1d/nd/td) ADS 层: ads 开头 结构 ads_xxx DM 层: dm 开头 结构: dm_xx OTHER: 未纳入分层 符合则 10 分，否则 0 分，未纳入分层给 5 分
	是否有表备注	有 则 10 分，无 则 0 分
存储	是否有字段备注	有备注字段/所有字段 *10 分
	生命周期是否合理	未设定周期类型的 给 0 分 周期类型为永久、拉链表 则给 10 分 周期类型为日分区： 无分区信息的给 0 分 没设生命周期给 0 分 周期长度超过建议周期天数{days}给 5 分
	是否空表	空表则 0 分，有数据则 10 分
	是否存在相似表	同层次两个表字段重复超过 {percent} %，则给 0 分，其余给 10 分

计算	是否长期无产出	一张表{days}天内没有产出数据 则给 0 分，其余给 10 分
	是否长期无访问	一张表{days}天内没有访问 则给 0 分，其余给 10 分
	计算中是否有报错	检查 DS 有报错 则给 0 分，其余给 10 分
	sql 是否包含 select *	sql 语句中包含 select * ，给 0 分，其余给 10 分
	sql 是否简单加工	sql 语句没有任何 join\groupby\union\函数的复杂计算，且 where 过滤字段皆为分区字段，则视为简单加工，给 0 分，其余给 10 分
	计算中是否存在数据倾斜	检查是否有数据倾斜，如果某个 stage 的最大任务耗时超过平均耗时任务耗时的{percent}%, 只检查耗时超过 {stage_dur_seconds}秒的 stage。 存在倾斜给 0 分，不存在给 10 分。
质量	表产出时效监控	当日产出时效，超过前 {days} 天产出时效平均值 {percent}% 则给 0 分，其余 10 分
	表产出数据量监控	必须日分区表 当日产出的数据量，超过前 {days} 天平均产出量 {upper_limit}%, 或低于 {lower_limit}%, 则给 0 分，其余 10 分
安全	是否未明确安全等级	未设置 0 分 其余 10 分
	目录文件数据访问权限是否超过建议值	检查该表最高权限的目录或者文件，如果超过文件超过 {file_permission} 或者目录超过 {dir_permission} 则给 0 分 其余给 10 分

8.2 健康分计算规则

计算表的健康分（仅参考）

数据表健康分公式= （ 规范得分/规范总分 * 规范权重

+ 存储得分/存储总分*存储权重

+ 计算得分/计算总分*计算权重

+ 质量得分/质量总分*质量权重

+ 安全得分/存储总分*安全权重

更多 Java –大数据 –前端 –python 人工智能资料下载，可百度访问：尚硅谷官网

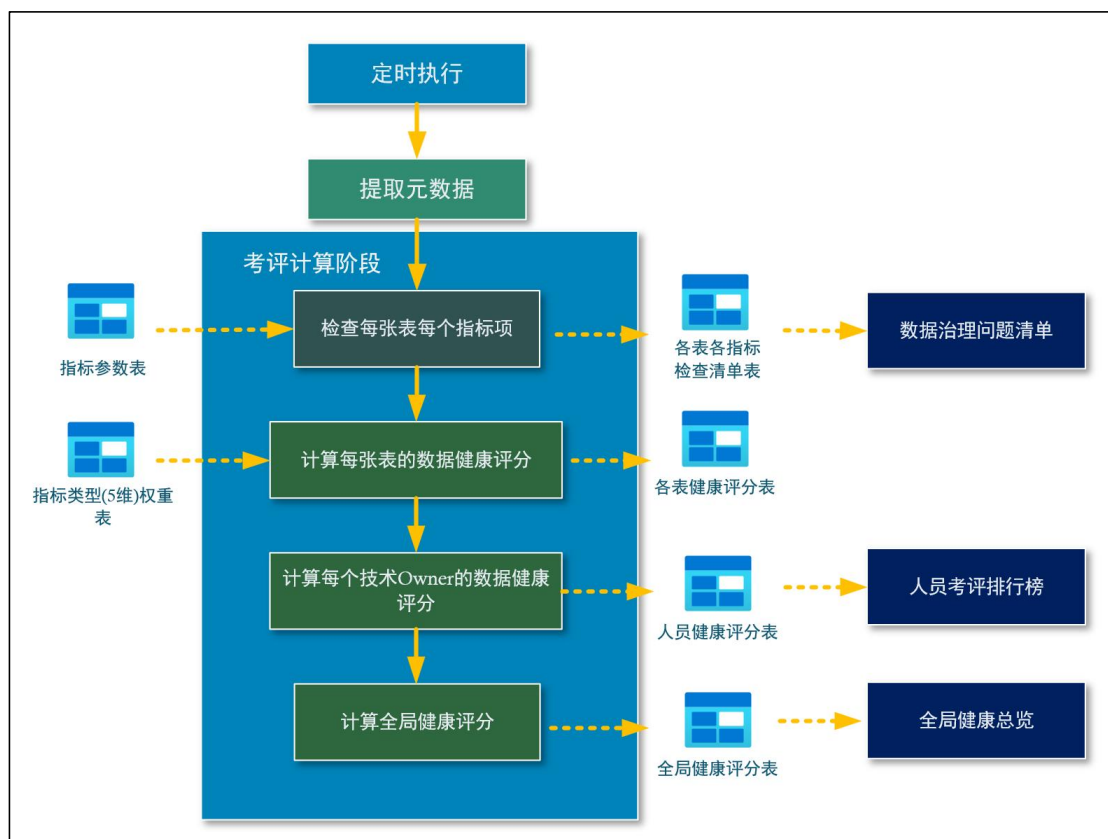
) * 数据表权重

各个板块权重

板块	权重
规范	15
存储	30
计算	30
质量	15
安全	10

要把每张表的每个指标项进行计算，再按照公式计算数据表的健康分，同时核算到每个技术 owner 的负责表的总健康分，最终核算为全数仓的总健康分。

8.3 执行步骤分析



8.4 表结构设计

考评指标参数表

```
create table dga.governance_metric
(
    id                bigint NOT NULL AUTO_INCREMENT comment 'id',
    metric_name       varchar(200) null comment '指标名称',
    metric_code       varchar(200) null comment '指标编码',
```

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网

```
metric_desc      varchar(2000) null comment '指标描述',
governance_type  varchar(20)   null comment '治理类型',
metric_params_json varchar(2000) null comment '指标参数',
governance_url   varchar(500)  null comment '治理连接',
skip_assess_tables varchar(2000) null comment '跳过考评的表名(多表逗号分割)',
is_disabled      varchar(1)    null comment '是否启用',
PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=1 DEFAULT CHARSET=utf8mb4 COMMENT '考评指标参数表';
```

考评指标类别权重表

```
CREATE TABLE `governance_type` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT 'id',
  `type_code` varchar(200) DEFAULT NULL COMMENT '治理项类型编码',
  `type_desc` varchar(2000) DEFAULT NULL COMMENT '治理项类型描述',
  `type_weight` decimal(10,2) DEFAULT NULL COMMENT '治理类型权重',
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=1 DEFAULT CHARSET=utf8mb4 COMMENT='治理考评类别权重表';
```

治理考评结果明细

```
CREATE TABLE `governance_assess_detail` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT 'id',
  `assess_date` varchar(20) DEFAULT NULL COMMENT '考评日期',
  `table_name` varchar(200) DEFAULT NULL COMMENT '表名',
  `schema_name` varchar(200) DEFAULT NULL COMMENT '库名',
  `metric_id` varchar(200) DEFAULT NULL COMMENT '指标项 id',
  `metric_name` varchar(200) DEFAULT NULL COMMENT '指标项名称',
  `governance_type` varchar(200) DEFAULT NULL COMMENT '治理类型',
  `tec_owner` varchar(200) DEFAULT NULL COMMENT '技术负责人',
  `assess_score` decimal(10,2) DEFAULT NULL COMMENT '考评得分',
  `assess_problem` varchar(2000) DEFAULT NULL COMMENT '考评问题项',
  `assess_comment` varchar(2000) DEFAULT NULL COMMENT '考评备注',
  `is_assess_exception` varchar(1) DEFAULT '0' COMMENT '考评是否异常',
  `assess_exception_msg` varchar(2000) DEFAULT NULL COMMENT '异常信息',
  `governance_url` varchar(2000) DEFAULT NULL COMMENT '治理处理路径',
  `create_time` datetime DEFAULT NULL COMMENT '创建日期',
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='治理考评结果明细';
```

8.5 代码类清单

package		说明
bean	GovernanceType	治理类型
(代码生成器生成)	GovernanceMetric	治理指标
	GovernanceAssessDetail	考评明细
mapper	GovernanceMetricMapper	治理指标数据访问接口
(代码生成器生成)	GovernanceAssessDetailMapper	考评业务数据访问接口
	GovernanceTypeMapper	治理类型数据访问接口
service	GovernanceAssessDetailService	考评业务接口
(代码生成器生成)	GovernanceMetricService	治理指标接口
	GovernanceTypeService	治理类型
service.impl	GovernanceAssessDetailServiceImpl	考评业务实现类
(代码生成器生成)	GovernanceMetricServiceImpl	治理指标实现类

更多 Java – 大数据 – 前端 – python 人工智能资料下载，可百度访问：[尚硅谷官网](#)

	GovernanceTypeServiceImpl	治理类型业务实现类
assessor	Assessor	考评器抽象类
需要手写	BusiOwnerAssessor TecOwnerAssessor FieldCommentAssessor TableCommentAssessor TableDwLevelAssessor TableNameStandardAssessor	根据考评指标定义各种考评器，每个考评指标一个考评器，继承考评器抽象类。

8.6 自动生成代码

以上代码除了 assessor 都可以通过 MyBatis-Plus 的代码生成工具以数据库表为基础生成最基础的代码。

8.7 各个指标项考评业务逻辑

8.7.1 分析

目标：依次对所有表的所有考评指标进行判断

步骤：

- (1) 取得所有待考评表的列表 (List<TableMetaInfo>)
- (2) 取得所有待考评的指标项列表(List<GovernanceMetric>)
- (3) 两层循环迭代两个列表，取得每个指标项的考评器，把考评器需要的参数传递给 考评器，考评打分，得到各个表各个指标项的评分结果(List<GovernanceAssessDetail>)
- (4) 把考评结果保存起来

8.7.2 步骤一：取得所有待考评表的列表

虽然我们需要取得主要的内容是 table_meta_info，但是 table_meta_info_extra 也是我们考评的内容，所以我们需要把这两个表关联起来查询。并且查询结果写到 List<TableMetaInfo>中。

首先可以利用 MyBatis-Plus 先查询出一个 List<TableMetaInfo>再根据每个 TableMetaInfo 中的表名、库名查询各个 TableMetaInfoExtra。但是这种方式会造成循环查询数据库，这种操作经常会成为性能炸弹，不推荐。

所以对表关联查询最好的方式还是使用 sql 语句中的 join 操作，然后把 join 后的结果封装到一个组合关系的对象中。

但是进行 join 操作最麻烦的就是把 join 后的结果封装到一个组合关系的对象中，尤其是当字段非常多的时候。

8.7.2.1 mapper 层 查询 table_meta_info 和 table_meta_info_extra 的 join 结果

在 TableMetaInfoMapper 中增加方法

```
@Select(" select tm.id as tm_id , te.id as te_id ,  tm.* , te.*  from table_meta_info tm  
join table_meta_info_extra te " +  
        " on tm.schema_name = te.schema_name and tm.table_name = te.table_name " +  
        " where tm.assess_date = #{assessDate}")  
@ResultMap("meta_result_map")  
public List<TableMetaInfo> selectTableMetaInfoList(@Param("assessDate") String  
assessDate) ;
```

这里@Select 里包含了一个 join 方法，除了 join 操作以外 where 条件是对考评日期进行过滤，取最近一次的元数据提取。

另外麻烦的是要把这两个表的字段封装到一个对象中。在 TableMetaInfo 中 TableMetaInfoExtra 是作为一个属性存在的。

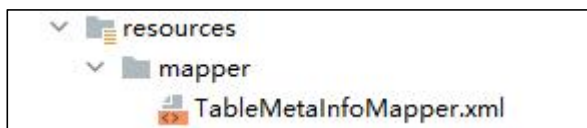
8.7.2.2 关于封装属性

首先 select 后面的字段推荐把所有字段都写清楚，如果主子表的字段有重复的，请利用别名区分开。

然后可以利用 MyBatis 的 xml 的自动映射机制，把字段映射到类的属性上。

这里使用了@ResultMap("tableMetaInfoMap") 这其实对应了一个 xml 的映射配置文件。

在 Resource/mapper/下增加映射文件



```
<?xml version="1.0" encoding="UTF-8"?>  
<!DOCTYPE mapper PUBLIC "-//MyBatis.org//DTD Mapper 3.0//EN"  
"http://MyBatis.org/dtd/MyBatis-3-mapper.dtd">  
<mapper namespace="com.atguigu.dga.meta.mapper.TableMetaInfoMapper">  
    <!-- 通用查询映射结果 -->  
    <resultMap id="meta_result_map" type="com.atguigu.dga.meta.bean.TableMetaInfo"  
autoMapping="true">  
        <id column="tm_id" property="id" />  
        <association property="tableMetaInfoExtra"  
javaType="com.atguigu.dga.meta.bean.TableMetaInfoExtra" autoMapping="true">  
            <id column="te_id" property="id"/>  
        </association>  
    </resultMap>  
</mapper>
```

分解说明：

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网

其实 MyBatis 对应 sql 语句的定义有两种方式 xml 和注解方式，绝大多数场景注解方式都可以替代 xml，但是极个别情况目前没有办法替代，比如关于 join 结果的自动映射。

以上 xml 的映射

```
<mapper namespace="com.atguigu.dga.meta.mapper.TableMetaInfoMapper">
```

namespace 标识该 xml 关联的 mapper 接口类。

```
<resultMap id="tableMetaInfoMap" type="com.atguigu.dga.meta.bean.TableMetaInfo"
autoMapping="true">
```

type 定义了查询结果要封装到哪个类中，id 为该映射做了唯一标识可以在其他地方通过 resultMap 进行引用，autoMapping 表示需要按照下划线对驼峰的方式进行自动映射。

```
<result column="id" property="id"></result>
```

<result>标签用于明确哪些字段要封装到哪些属性中，但是因为有设置 autoMapping，所以只考虑主子类中有重复命名的字段，特别声明就可以了，其他字段会自动映射。

```
<association property="tableMetaInfoExtra"
javaType="com.atguigu.dga.meta.bean.TableMetaInfoExtra" autoMapping="true">
  <result column="te_id" property="id"></result>
</association>
```

此处声明，哪些字段应该映射到主类下 tableMetaInfoExtra 这个属性中，javaType 声明这个属性是什么 java 类型的，autoMapping 可以进行自动映射。

<result>也是和主类声明一样，只声明主子类同名字段即可，其他的交 automapping。这样如果有特别多的字段时，可以减少很多工作量。

另外这个 xml 所存放的位置需要在 application.properties 特别声明。

```
#设定自动映射行为：FULL 标识对象存在嵌套的情况
mybatis.configuration.auto-mapping-behavior=FULL
#映射文件所在位置
mybatis.mapper-locations=classpath:mapper/*.xml
```

8.7.2.3 Meta 服务层代码

TableMetaInfoService 增加方法：

```
List<TableMetaInfo> getTableMetaList(String assessDate);
```

TableMetaInfoServiceImpl 增加方法：

```
public List<TableMetaInfo> getTableMetaList(String assessDate) {
    List<TableMetaInfo> tableMetaInfoList =
getBaseMapper().selectTableMetaInfoList(assessDate);
    return tableMetaInfoList;
}
```

8.7.2.4 governance 服务层增加代码

GovernanceAssessDetailService 接口中增加方法

```
public interface GovernanceAssessDetailService extends IService<GovernanceAssessDetail> {
    public void mainAssess(String assessDate);
}
```

GovernanceAssessDetailServiceImpl 类中实现方法

```
@Service
```

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网

```
@DS("dga")
public class GovernanceAssessDetailServiceImpl extends
    ServiceImpl<GovernanceAssessDetailMapper, GovernanceAssessDetail> implements
    GovernanceAssessDetailService {

    public void mainAssess( String assessDate){
        // 0 幂等性处理 , 把考评当日的考评结果清理
        remove( new QueryWrapper<GovernanceAssessDetail>().eq("assess_date" , assessDate));

        // 1. 提取表元数据信息, 提取到一个 List<>
        // 1.1 元数据信息 + 辅助信息
        // 1.2 结合考评日期
        List<TableMetaInfo> tableMetaList = tableMetaInfoService.getTableMetaList(assessDate);
    }
}
```

8.7.3 步骤二：查询所有指标

查询全部指标，只需要去掉被禁用的指标即可。

在 GovernanceAssessDetailServiceImpl 的 mainAssess 方法增加如下即可

```
// 2. 提取需要考评的指标, 提取到一个 List<>
// 2.1 启用状态
List<GovernanceMetric> governanceMetricList = governanceMetricService.list(
    new QueryWrapper<GovernanceMetric>()
        .eq("is_disabled", "0")
);
```

8.7.4 步骤三：对每张表每个指标进行考评

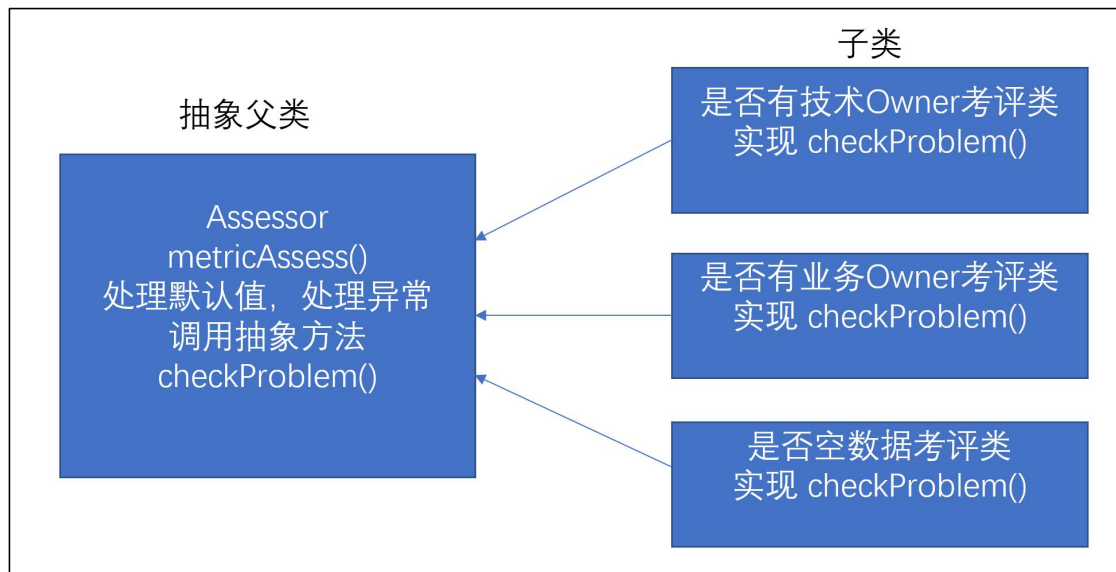
用之前的表元数据列表和指标项列表进行两层嵌套循环。然后依次进行考评操作，产生考评结果，并进行考评。

8.7.4.1 模板模式

在模板模式（Template Pattern）中，一个抽象类公开定义了执行它的方法的方式/模板。它的子类可以按需要重写方法实现，但调用将以抽象类中定义的方式进行。

特点：由父抽象类负责控制调用，由不同子类负责核心功能。

好处：符合开闭原则，即对修改封闭，对扩展开发代码责任清晰易于维护。



参考网址: <https://www.w3cschool.cn/shejimoshi/>

8.7.4.2 创建抽象类

```

public abstract class Assessor {

    // 主方法, 控制, 被外部调用
    // 参数 1 考评时间
    // 参数 2 某张表 元数据 TableMetaInfo
    // 参数 3 什么指标 GovernanceMetric

    public final GovernanceAssessDetail metricAssess(AssessParam assessParam){
        System.out.println("表: " + assessParam.getTableMetaInfo().getTableName() + " , 指标: " + assessParam.getGovernanceMetric().getMetricCode() + " 父类进行考评...");
        // 0.考评结果默认信息 时间 表名 指标 .... 默认给满分
        GovernanceAssessDetail governanceAssessDetail = new GovernanceAssessDetail();
        governanceAssessDetail.setAssessDate(assessParam.getAssessDate());
        governanceAssessDetail.setTableName(assessParam.getTableMetaInfo().getTableName());
        governanceAssessDetail.setSchemaName(assessParam.getTableMetaInfo().getSchemaName());
        governanceAssessDetail.setMetricId(assessParam.getGovernanceMetric().getId()+"");

        governanceAssessDetail.setMetricName(assessParam.getGovernanceMetric().getMetricName());

        governanceAssessDetail.setGovernanceType(assessParam.getGovernanceMetric().getGovernanceType());

        String tecOwner =
        assessParam.getTableMetaInfo().getTableMetaInfoExtra().getTecOwnerUserName() == null ? "未填写" : assessParam.getTableMetaInfo().getTableMetaInfoExtra().getTecOwnerUserName();
        governanceAssessDetail.setTecOwner(tecOwner);
        governanceAssessDetail.setAssessScore(BigDecimal.TEN);
        governanceAssessDetail.setCreateTime(new Date());
        // 1.调用抽象方法 检查问题项
        //checkProblem(governanceAssessDetail , assessParam);
        // 2. 捕获异常 , 写入指定字段
        try {
            // 1.调用抽象方法 检查问题项
            checkProblem(governanceAssessDetail , assessParam);
        }catch (Exception e){
            governanceAssessDetail.setIsAssessException("1");
            StringWriter stringWriter = new StringWriter();
            PrintWriter printWriter = new PrintWriter(stringWriter);
            e.printStackTrace(printWriter);
        }
    }
}
    
```

更多 Java –大数据 –前端 –python 人工智能资料下载, 可百度访问: 尚硅谷官网

```
String expMsg = stringWriter.toString();
governanceAssessDetail.setAssessExceptionMsg(expMsg.substring(0, Math.min(2000,
expMsg.length())));
}
// 3. 返回结果
return governanceAssessDetail ;
}

// 抽象方法 1 个或者多个
protected abstract void checkProblem(GovernanceAssessDetail governanceAssessDetail ,
AssessParam assessParam ) throws Exception;
}
```

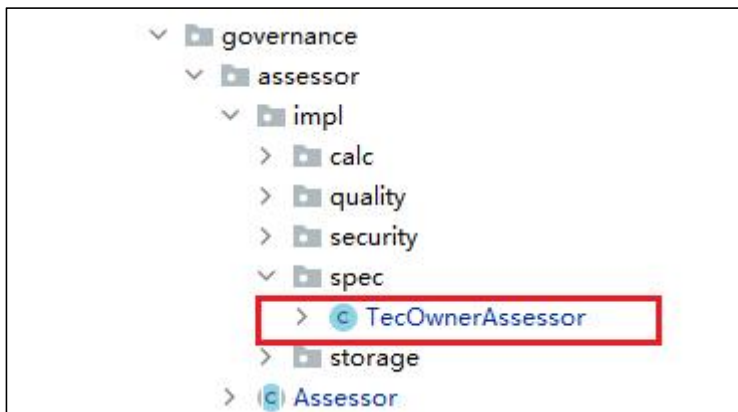
8.7.4.3 创建考评器之一：是否有技术 Owner

考评板块	考评指标	考评标准	需要信息
规范	有技术 owner	有则 10 分，无则 0 分	元数据

由于已经有抽象父类了所以，子类必须做的工作就只是：判断然后打分。

在 assessor 目录下增加 impl 目录，并且为了区分不同的治理类型，建立五个 package: spec, storage, quality, calc, security。

由于是否有技术 Owner 属于规范的一种，所以在 spec 下创建考评器 TecOwnerAssessor



```
@Component("TEC_OWNER")
public class TecOwnerAssessor extends Assessor {
    @Override
    protected void checkProblem(GovernanceAssessDetail governanceAssessDetail, AssessParam
assessParam) {
        // 考评，如果检查出问题，给 0 分，写问题描述
        System.out.println("表: " + assessParam.getTableMetaInfo().getTableName() + " , 指标:
" + assessParam.getGovernanceMetric().getMetricCode() + " 子类检查问题项.....");

        String tecOwnerUserName =
assessParam.getTableMetaInfo().getTableMetaInfoExtra().getTecOwnerUserName();
        if(tecOwnerUserName == null || tecOwnerUserName.trim().length() == 0 ){
            governanceAssessDetail.setAssessScore(BigDecimal.ZERO);
            governanceAssessDetail.setAssessProblem("未填写技术 OWNER");
            String governanceUrl = assessParam.getGovernanceMetric().getGovernanceUrl();
            //替换{tableId}
            governanceUrl = governanceUrl.replace("{tableId}",
assessParam.getTableMetaInfo().getId() + "");
            governanceAssessDetail.setGovernanceUrl(governanceUrl);
        }
    }
}
```

更多 Java-大数据-前端-python 人工智能资料下载，可百度访问：[尚硅谷官网](http://www.shangguigu.com)

```
}  
}
```

其中 `@Component` 标识该类会被创建为容器组件，同时给组件赋予了名字“TEC_OWNER”，这个标识名称用于从容器中获得对应的指标考评器。

8.7.4.4 获得组件工具类

增加一个工具类用于从 Spring 容器中获得组件

```
@Component  
public class SpringBeanProvider implements ApplicationContextAware {  
  
    ApplicationContext applicationContext ;  
  
    @Override  
    public void setApplicationContext(ApplicationContext applicationContext) throws  
BeansException {  
        this.applicationContext = applicationContext ;  
    }  
  
    public <T> T getBeanByName(String beanName , Class<T> tClass){  
        T bean = applicationContext.getBean(beanName, tClass);  
        return bean ;  
    }  
}
```

8.7.4.5 增加一条考评指标数据

```
INSERT INTO `governance_metric` VALUES (1,'是否有技术 Owner','TEC_OWNER','辅助信息是否填写技术  
Owner 有 则10分， 无则0分','SPEC','','0')
```

8.7.4.6 增加考核参数对象

```
@Data  
public class AssessParam {  
    String assessDate ;  
  
    TableMetaInfo tableMetaInfo ;  
  
    GovernanceMetric governanceMetric ;  
}
```

8.7.4.7 对所有表所有指标进行考评

在 `GovernanceAssessDetailServiceImpl` 的 `mainAssess` 方法增加如下代码：

```
//准备考评结果集合  
List<GovernanceAssessDetail> governanceAssessDetailList = new  
ArrayList<>( tableMetaList.size() * governanceMetricList.size());  
for (TableMetaInfo tableMetaInfo : tableMetaList) {  
    for (GovernanceMetric governanceMetric : governanceMetricList) {  
        if(governanceMetric.getSkipAssessTables() != null &&  
governanceMetric.getSkipAssessTables().trim().length() > 0 ){  
            String[] skipTableArr =  
governanceMetric.getSkipAssessTables().split(",");  
            //检查是否需要跳过检查的表  
            boolean isSkip = false ;  
            for (String skipTable: skipTableArr) {  
                if(tableMetaInfo.getTableName().equals(skipTable)){  
                    isSkip = true ;  
                    break ;  
                }  
            }  
        }  
    }  
}
```

更多 Java-大数据-前端-python 人工智能资料下载，可百度访问：尚硅谷官网

```

    }
}
//是跳过表，本次检查跳过
if(isSkip){
    continue;
}
}
//某张表某个指标的考评
String metricCode = governanceMetric.getMetricCode();
//通过 metricCode 从容器中获取对应的考评器
Assessor assessor = springBeanProvider.getBeanByName(metricCode,
Assessor.class);
// 封装考评需要的参数
AssessParam assessParam = new AssessParam();
assessParam.setAssessDate(assessDate);
assessParam.setTableMetaInfo(tableMetaInfo);
assessParam.setGovernanceMetric(governanceMetric);
assessParam.setTableMetaInfoList(tableMetaList);
//开始考评
GovernanceAssessDetail governanceAssessDetail =
assessor.metricAssess(assessParam);
//将考评结果存入到考评结果集合中
governanceAssessDetailList.add(governanceAssessDetail) ;
}
}

```

8.7.5 步骤四：保存考评结果

在 GovernanceAssessDetailServiceImpl 的 mainAccess 方法增加如下代码：

```

// 4. 把考评结果写到 Mysql
saveBatch(governanceAssessDetailList) ;

```

8.8 总结

任务三，以一个最简单的考评指标为例，打通一次考评流程。

考评的流程一共四步：

- (1) 获得待考评表的列表
- (2) 获得所有指标的列表
- (3) 考评计算，打分
- (4) 保存结果

同时为了提高考评器的可维护性，以及编程的开闭原则，我们可以根据维护的考评的数据库表动态的使用 Spring 容器中的组件对象。这种结构实际上是设计模式中**策略模式**的结构。

第 9 章 其他比较简单的指标项

9.1 是否有业务 Owner

考评板块	考评指标	考评标准	需要信息
------	------	------	------

规范	是否有业务 owner	有则 10 分，无则 0 分	元数据
----	-------------	----------------	-----

```
@Component("BUSI_OWNER")
public class BusiOwnerAssessor extends Assessor {

    public void checkProblem(GovernanceAssessDetail governanceAssessDetail, AssessParam
assessParam ) {
        TableMetaInfo tableMetaInfo = assessParam.getTableMetaInfo();
        GovernanceMetric governanceMetric = assessParam.getGovernanceMetric();

        if(tableMetaInfo.getTableMetaInfoExtra().getBusiOwnerUserName()==null||tableMetaInfo.getTabl
eMetaInfoExtra().getBusiOwnerUserName().length()==0){
            governanceAssessDetail.setAssessScore(BigDecimal.valueOf(0));
            governanceAssessDetail.setAssessProblem("缺少业务 OWNER");

        }

        governanceAssessDetail.setGovernanceUrl(governanceMetric.getGovernanceUrl()+tableMetaInfo.ge
tId());
    }

}
```

9.2 是否有表备注

考评板块	考评指标	考评标准	需要信息
规范	是否有表备注	有则 10 分，无则 0 分	元数据

```
@Component("TABLE_COMMENT")
public class TableCommentAssessor extends Assessor {

    public void checkProblem(GovernanceAssessDetail governanceAssessDetail, AssessParam
assessParam ) {
        TableMetaInfo tableMetaInfo = assessParam.getTableMetaInfo();

        if(tableMetaInfo.getTableComment()==null||tableMetaInfo.getTableComment().length()==0){

            governanceAssessDetail.setAssessScore(BigDecimal.valueOf(0));
            governanceAssessDetail.setAssessProblem("缺少表备注");

        }

    }

}
```

9.3 是否缺失字段备注

考评板块	考评指标	考评标准	需要信息
规范	是否有字段备注	有备注字段/所有字段 *10 分	元数据

```
@Component("FIELD_COMMENT")
public class FieldCommentAssessor extends Assessor {

    public void checkProblem(GovernanceAssessDetail governanceAssessDetail, AssessParam
assessParam ) {
        TableMetaInfo tableMetaInfo = assessParam.getTableMetaInfo();

        if(tableMetaInfo.getColNameJson()!=null&&tableMetaInfo.getColNameJson().length()>0){
```

```
//获得字段列表
List<JSONObject> coljsonList=
JSON.parseArray(tableMetaInfo.getColNameJson(),JSONObject.class);

Set<String> colMissCommentSet= new HashSet<>();
for (JSONObject jsonObject : coljsonList) {
    String comment = jsonObject.getString("comment");
    if(comment==null || comment.length()==0){ //判断哪些缺失
        colMissCommentSet.add(comment); //缺失备注的字段
    }
}
if(colMissCommentSet.size()>0) {
    if(colMissCommentSet.size()==coljsonList.size()){ //全部缺失, 给 0 分
        governanceAssessDetail.setAssessScore(BigDecimal.valueOf(0));
    }else {
        int score = (coljsonList.size() - colMissCommentSet.size()) * 100 /
coljsonList.size(); //根据缺失的占比给分数
        governanceAssessDetail.setAssessScore(BigDecimal.valueOf(score));
    }
    governanceAssessDetail.setAssessProblem("缺少字段备
注:"+JSON.toJSONString(colMissCommentSet));
}
}
}
```

9.4 是否空表

考评板块	考评指标	考评标准	需要信息
存储	是否空表	空表则 0 分，有数据则 10 分	元数据

```
@Component("TABLE_EMPTY")
public class TableEmptyAssessor extends Assessor {

    public void checkProblem(GovernanceAssessDetail governanceAssessDetail, AssessParam
assessParam ) {
        TableMetaInfo tableMetaInfo = assessParam.getTableMetaInfo();
        GovernanceMetric governanceMetric = assessParam.getGovernanceMetric();
        if(tableMetaInfo.getTableSize()==0L){
            governanceAssessDetail.setAssessScore(BigDecimal.valueOf(0));
            governanceAssessDetail.setAssessProblem("数据为空");

governanceAssessDetail.setGovernanceUrl(governanceMetric.getGovernanceUrl()+"/"+tableMetaInf
o.getId());
        }
    }
}
```

9.5 是否设置了安全级别

考评板块	考评指标	考评标准	需要信息
安全	是否明确安全等级	未设置 0 分 其余 10 分	元数据

```
@Component("SECURITY_LEVEL")
public class SecurityLevelAssessor extends Assessor {
```

```

@Override
protected void checkProblem(GovernanceAssessDetail governanceAssessDetail, AssessParam
assessParam) throws Exception {
    TableMetaInfo tableMetaInfo = assessParam.getTableMetaInfo();
    GovernanceMetric governanceMetric = assessParam.getGovernanceMetric();
    if (tableMetaInfo.getTableMetaInfoExtra().getSecurityLevel() == null ||
tableMetaInfo.getTableMetaInfoExtra().getSecurityLevel().equals("UNSET")) {
        governanceAssessDetail.setAssessScore(BigDecimal.valueOf(0));
        governanceAssessDetail.setAssessProblem("没有设置安全级别");
        governanceAssessDetail.setGovernanceUrl(governanceMetric.getGovernanceUrl() +
tableMetaInfo.getId());
    }
}
}

```

9.6 长期未被访问表

考评板块	考评指标	考评标准	需要信息
计算	是否长期无访问	一张表 {days} 天内没有访问 则给 0 分，其余给 10	长期无访问

```

@Component("NO_ACCESS")
public class NoAccessAssessor extends Assessor {
    @Override
    protected void checkProblem(GovernanceAssessDetail governanceAssessDetail, AssessParam
assessParam) throws ParseException {
        //根据元数据中的最后访问时间 与考评时间进行差值计算，获得的天数与参数中的 days 比较，大于 days 给 0
        分，记录问题
        Date tableLastAccessTime = assessParam.getTableMetaInfo().getTableLastAccessTime();
        String assessDateString = assessParam.getAssessDate();

        // apache lang3      google guava
        // 字符串转日期
        Date assessDate = DateUtils.parseDate(assessDateString, "yyyy-MM-dd");
        // 截取到天
        Date lastTableAssessDate = DateUtils.truncate(tableLastAccessTime, Calendar.DATE);

        //计算差值
        long diffMils = Math.abs(assessDate.getTime() - lastTableAssessDate.getTime());

        long diffDays = TimeUnit.DAYS.convert(diffMils, TimeUnit.MILLISECONDS);

        //获取考评指标参数
        String metricParamsJson = assessParam.getGovernanceMetric().getMetricParamsJson();
        JSONObject paramObj = JSON.parseObject(metricParamsJson);
        Integer days = paramObj.getInteger("days");
        if (diffDays >= days) {
            governanceAssessDetail.setAssessScore(BigDecimal.ZERO);
            governanceAssessDetail.setAssessProblem("长期未访问");
        }
    }
}

```

9.7 长期未产出表

考评板块	考评指标	考评标准	需要信息
计算	是否长期无产出	一张表 {days} 天内没有产出数据 则 检查 hdfs	

给 0 分，其余给 10

```
@Component("NO_PRODUCE")
public class NoProduceAssessor extends Assessor {

    @Override
    protected void assess(GovernanceAssessDetail governanceAssessDetail, AssessParam
    assessParam) throws Exception {
        TableMetaInfo tableMetaInfo = assessParam.getTableMetaInfo();

        String metricParamsJson = assessParam.getGovernanceMetric().getMetricParamsJson();
        JSONObject metricParamsJsonObj = JSON.parseObject(metricParamsJson);
        Integer days = metricParamsJsonObj.getInteger("days");

        Date assessDate = DateUtils.parseDate(assessParam.getAssessDate(), "yyyy-MM-dd");
        Date lastModifyDate = DateUtils.truncate(tableMetaInfo.getTableLastModifyTime(),
        java.util.Calendar.DATE);

        long diffInMillis = Math.abs(assessDate.getTime() - lastModifyDate.getTime());
        long diffDays = TimeUnit.DAYS.convert(diffInMillis, TimeUnit.MILLISECONDS);

        //如果差值天数大于 阈值天数
        if(diffDays >= days ){
            governanceAssessDetail.setAssessScore(BigDecimal.ZERO);
            governanceAssessDetail.setAssessProblem("长期没有写入操作");
        }
    }
}
```

第 10 章 一般难度的指标项

10.1 表名是否合规

考评板块	考评指标	考评标准	需要信息
规范	表名是否合规	参考建数仓表规范 ODS 层： 开头:ods 结尾 :inc/full 结构 ods_xx_(inc full) DIM 层： dim 开头 full/zip 结尾 结构: dim_xx_(zip full) DWD 层: dwd 开头 inc/full 结尾 结构: dwd_xx_xx_(inc full) DWS 层: dws 开头 结构 dws_xx_xx_xx_(1d/nd/td) ADS 层: ads 开头 结构 ads_xxx DM 层: dm 开头	元数据

结构: dm_xx

OTHER: 未纳入分层

符合则 10 分, 否则 0 分,

未纳入分层给 5 分。

本身来说这指标其实就是字符串检查并不复杂, 不过这个指标可以通过正则表达式使代码变得更加清晰简洁, 而减少字符串切割或字符串匹配的繁琐。

正则表达式的主要目的: 判断一个字符串是否符合某种范式。

正则表达式学习: <https://www.runoob.com/regexp/regexp-tutorial.html>

```
@Component("TABLE_NAME_STANDARD")
public class TableNameStandardAssessor extends Assessor {

    Pattern odsPattern = Pattern.compile("^ods_\\w+_ (inc|full)$") ;
    Pattern dimPattern = Pattern.compile("^dim_\\w+_ (zip|full)$") ;
    Pattern dwPattern = Pattern.compile("^dwd_\\w+_\\w+_ (inc|full)$") ;
    Pattern dwsPattern = Pattern.compile("^dws_\\w+_\\w+_\\w+_ (\\d{1,2}d|nd|td)$") ;
    Pattern adsPattern = Pattern.compile("^ads_\\w+$") ;
    Pattern dmPattern = Pattern.compile("^dm_\\w+$") ;

    @Override
    protected void checkProblem(GovernanceAssessDetail governanceAssessDetail, AssessParam
assessParam) throws Exception {
        String tableName = assessParam.getTableMetaInfo().getTableName() ;
        String dwLevel = assessParam.getTableMetaInfo().getTableMetaInfoExtra().getDwLevel() ;
        Matcher matcher = null ;
        if(dwLevel.equals("ODS")){
            matcher =odsPattern.matcher(tableName) ;
        }else if(dwLevel.equals("DWD")){
            matcher =dwdPattern.matcher(tableName);
        }else if(dwLevel.equals("DIM")){
            matcher =dimPattern.matcher(tableName);
        }else if(dwLevel.equals("DWS")){
            matcher =dwsPattern.matcher(tableName);
        }else if(dwLevel.equals("ADS")){
            matcher =adsPattern.matcher(tableName);
        }else if(dwLevel.equals("DM")){
            matcher =dmPattern.matcher(tableName);
        }else{
            governanceAssessDetail.setAssessScore(BigDecimal.valueOf(5));
            governanceAssessDetail.setAssessProblem("未纳入分层");
            return ;
        }

        if(!matcher.matches()){
            governanceAssessDetail.setAssessScore(BigDecimal.ZERO);
            governanceAssessDetail.setAssessProblem("表名不符合规范");
        }
    }
}
```

10.2 生命周期是否合理

考评板块	考评指标	考评标准	需要信息
存储	生命周期是否合理	未设定周期类型的 给 0 分	元数据

周期类型为永久、拉链表 则给 10 分

周期类型为日分区：

无分区信息的给 0 分

没设生命周期给 0 分

周期长度超过建议周期天数

{days}，则给（建议周期天数 / 设置周期天数）*10

```
@Component("LIFECYCLE")
public class LifecycleAssessor extends Assessor {
    @Override
    public void checkProblem(GovernanceAssessDetail governanceAssessDetail, AssessParam
assessParam ) {
        TableMetaInfo tableMetaInfo = assessParam.getTableMetaInfo();
        GovernanceMetric governanceMetric = assessParam.getGovernanceMetric();
        JSONObject paramJson = JSON.parseObject(governanceMetric.getMetricParamsJson());
        Integer lifecycleSuggestDays = paramJson.getInteger("days");

        if(tableMetaInfo.getTableMetaInfoExtra().getLifecycleType()==null||tableMetaInfo.getTableMet
aInfoExtra().getLifecycleType().equals("UNSET")){
            governanceAssessDetail.setAssessScore(BigDecimal.valueOf(0));
            governanceAssessDetail.setAssessProblem("未设置生命周期类型");

            governanceAssessDetail.setGovernanceUrl(governanceMetric.getGovernanceUrl()+"-"+tableMetaInf
o.getId());

        }else if(tableMetaInfo.getTableMetaInfoExtra().getLifecycleType().equals(MetaConst.
LIFECYCLE_TYPE_DAY)){
            if(tableMetaInfo.getPartitionColNameJson()!=null){
                List<JSONObject> partitionColList =
JSON.parseArray(tableMetaInfo.getPartitionColNameJson(), JSONObject.class);
                if(partitionColList==null||partitionColList.size()==0){
                    governanceAssessDetail.setAssessScore(BigDecimal.valueOf(0));
                    governanceAssessDetail.setAssessProblem("缺少分区信息");
                }else
            if(tableMetaInfo.getTableMetaInfoExtra()==null||tableMetaInfo.getTableMetaInfoExtra().getLif
ecycleDays()==0L){
                governanceAssessDetail.setAssessScore(BigDecimal.valueOf(0));

                governanceAssessDetail.setGovernanceUrl(governanceMetric.getGovernanceUrl()
+tableMetaInfo.getId());
                governanceAssessDetail.setAssessProblem("缺少生命周期");
            } else
            if(tableMetaInfo.getTableMetaInfoExtra().getLifecycleDays()>lifecycleSuggestDays){

                long score = lifecycleSuggestDays * 10 /
tableMetaInfo.getTableMetaInfoExtra().getLifecycleDays();
                governanceAssessDetail.setAssessScore(BigDecimal.valueOf(score));
                governanceAssessDetail.setAssessProblem("生命周期超过建议值");

                governanceAssessDetail.setGovernanceUrl(governanceMetric.getGovernanceUrl()
+tableMetaInfo.getId());
            }
        }
    }
}
```

```

    }
    }
}

```

10.3 是否为相似表

考评板块	考评指标	考评标准	需要信息
存储	是否存在相似表	同层次两个表字段重复超过 {percent}%, 则给 0 分, 其余给 10 分	元数据

```

@Component("TABLE_SIMILAR")
public class TableSimilarAssessor extends Assessor {
    @Override
    protected void checkProblem(GovernanceAssessDetail governanceAssessDetail, AssessParam
assessParam) throws Exception {
        //0.提前查询数据表的全集合, 作为参数
        Map<String, TableMetaInfo> tableMetaInfoMap = assessParam.getTableMetaInfoMap();
        TableMetaInfo currentTableMetaInfo = assessParam.getTableMetaInfo();
        String metricParamsJson = assessParam.getGovernanceMetric().getMetricParamsJson();
        JSONObject paramJsonObj = JSON.parseObject(metricParamsJson);
        Integer paramPercent = paramJsonObj.getInteger("percent");

        List<String> similarTableNameList = new ArrayList<>();
        //1.用当前考评表和其他所有表进行比较
        for (TableMetaInfo otherTableMetaInfo : tableMetaInfoMap.values()) {
            //排除自己

            if(currentTableMetaInfo.getTableInfo().equals(otherTableMetaInfo.getTableInfo())){
                continue;
            }
            //排除不同层次的

            if(!currentTableMetaInfo.getTableMetaInfoExtra().getDwLevel().equals( otherTableMetaInfo.get
TableMetaInfoExtra().getDwLevel())){
                continue;
            }
            //2.比较两张表的字段(2层循环) 获得相同字段名的个数
            Integer similarCount = 0 ;
            List<JSONObject> currentJsonObjList =
JSON.parseArray(currentTableMetaInfo.getColNameJson(), JSONObject.class);
            List<JSONObject> otherJsonObjList =
JSON.parseArray(otherTableMetaInfo.getColNameJson(), JSONObject.class);
            for (JSONObject currentColJsonObj : currentJsonObjList) {
                for (JSONObject otherColJsonObj : otherJsonObjList) {
                    String currentColName = currentColJsonObj.getString("name");
                    String otherColName = otherColJsonObj.getString("name");
                    if(currentColName.equals(otherColName)){
                        similarCount ++ ;
                    }
                }
            }

            //3. 相同字段名个数 / 总字段数 = 相似比率
            int similarPercent = similarCount * 100 / currentJsonObjList.size();
            //4. 相似比率和percent 比较, 如果大于加入相似表集合
            if(similarPercent > paramPercent){
                similarTableNameList.add(otherTableMetaInfo.getTableInfo());
            }
        }
    }
}

```

```

    }
}

//5.循环结束， 如果相似表集合 > 0， 给 0 分， 那些相似表列入考评结果
if(similarTableNameList.size() > 0){
    governanceAssessDetail.setAssessScore(BigDecimal.ZERO);
    governanceAssessDetail.setAssessProblem("存在相似表: " + similarTableNameList);
}
}
}

```

10.4 表产出数据量监控

考评板块	考评指标	考评标准	需要信息
质量	表产出数据量监控	必须日分区表	检查 hdfs
		当日产出的数据量，超过前 {days} 天 平均 产出量 {upper_limit}%，或低于 {lower_limit}%，则给 0 分，其 余 10 分	

```

@Component("PRODUCE_DATA_SIZE")
public class ProduceDataSizeAssessor extends Assessor {
    @Override
    protected void checkProblem(GovernanceAssessDetail governanceAssessDetail, AssessParam
assessParam) throws Exception {

        String assessDate = assessParam.getAssessDate();
        String metricParamsJson = assessParam.getGovernanceMetric().getMetricParamsJson();
        String tableFsOwner = assessParam.getTableMetaInfo().getTableFsOwner();

        JSONObject paramsJsonObj = JSON.parseObject(metricParamsJson);
        BigDecimal upperLimitPercent = paramsJsonObj.getBigDecimal("upper_limit");
        BigDecimal lowerLimitPercent = paramsJsonObj.getBigDecimal("lower_limit");
        Integer days = paramsJsonObj.getInteger("days");

        String tableFsPath = assessParam.getTableMetaInfo().getTableFsPath();

        if(MetaConst.LIFECYCLE_TYPE_DAY.equals(assessParam.getTableMetaInfo().getTableMetaInfoExtra
().getLifecycleType())){ //按日分区

            //计算前 n 天的分区目录名
            String partitionColNameJson =
assessParam.getTableMetaInfo().getPartitionColNameJson();
            JSONArray partColArr = JSON.parseArray(partitionColNameJson);
            String partName = partColArr.getJSONObject(0).getString("name");

            Date partitionDate = DateUtils.addDays(DateUtils.parseDate(assessDate, "yyyy-MM-
dd"), -1);
            String partitionDt = DateFormatUtils.format(partitionDate, "yyyy-MM-dd");

```

```
String curPartDirName=tableFsPath+"/"+partName+"="+partitionDt;
//当天日期分区的大小
Long curDataSize = getDataSizeByPath(curPartDirName, tableFsOwner);
governanceAssessDetail.addLog("(当前分区:"+curPartDirName +",数据
量:"+curDataSize );
//n 天前每日分区
int partCountWithData=0;
Long partTotalDateSize=0L;

//求前 n 天的总大小
for (int i = 1; i <= days ; i++) {
    Date date = DateUtils.addDays(partitionDate, 0 - i );
    String partDt = DateFormatUtils.format(date, "yyyy-MM-dd");
    String partDirName=tableFsPath+"/"+partName+"="+partDt;
    Long partDataSize = getDataSizeByPath(partDirName, tableFsOwner);
    if(partDataSize>0){
        partCountWithData++;
        partTotalDateSize+=partDataSize;
        governanceAssessDetail.addLog("(分区:"+partDirName +",数据
量:"+partDataSize );
    }

}
governanceAssessDetail.addLog("( 总数据量:"+partTotalDateSize+")" );

//进行比较
if(partCountWithData>0){
    Long avgPartDataSize=partTotalDateSize/partCountWithData;
    if(curDataSize > avgPartDataSize * ( upperLimitPercent.longValue()+100L)
/100L ){
        governanceAssessDetail.setAssessScore(BigDecimal.ZERO);
        governanceAssessDetail.setAssessProblem("产出 高于现有
"+partCountWithData+"日平均数据量的"+upperLimitPercent.longValue()+"%");
    }else if(curDataSize < avgPartDataSize * lowerLimitPercent.longValue()
/100L){
        governanceAssessDetail.setAssessScore(BigDecimal.ZERO);
        governanceAssessDetail.setAssessProblem("产出 低于现有
"+partCountWithData+"日平均数据量的"+lowerLimitPercent.longValue()+"%");
    }
}

}

}

//创建文件系统客户端，并调用递归方法获得文件大小
public Long getDataSizeByPath(String path,String fsUser) throws Exception{
    //初始化文件服务器客户端
    FileSystem fs = FileSystem.get(new URI(path), new Configuration(), fsUser);
    Long dataSize=0L;
    if(fs.exists(new Path(path))){
        FileStatus[] fileStatuses = fs.listStatus(new Path(path));
        dataSize = getFileDataSize(fileStatuses, 0L, fs);
    }

    return dataSize;
}

//通过递归获得目录及子目录中文件大小
public Long getFileDataSize (FileStatus[] curPathFileStatuses, Long curDateSize,
FileSystem fs) throws Exception{
```

```

        for (FileStatus curPathFileStatus : curPathFileStatuses) {
            if (curPathFileStatus.isDirectory()) {
                FileStatus[] fileStatuses = fs.listStatus(curPathFileStatus.getPath());
                curDateSize = getFileDataSize(fileStatuses, curDateSize, fs);
            } else {
                curDateSize += curPathFileStatus.getLen();
            }
        }
        return curDateSize;
    }
}

```

10.5 目录文件数据访问权限超过建议值

考评板块	考评指标	考评标准	需要信息
安全	目录文件数据访问权限超过建议值	检查该表最高权限的目录或者文件，如果超过文件超过 {file_permission} 或者目录超过 {dir_permission} 则给 0 分 其余给 10 分	检查 hdfs

```

@Component("FILE_ACCESS_PERMISSION")
public class FileAccessPermissionAssessor extends Assessor {

    @Override
    protected void checkProblem(GovernanceAssessDetail governanceAssessDetail, AssessParam assessParam) throws Exception {
        //1. 获得判断 hdfs 路径
        String tableFsPath = assessParam.getTableMetaInfo().getTableFsPath();
        String tableOwner = assessParam.getTableMetaInfo().getTableFsOwner();
        String metricParamsJson = assessParam.getGovernanceMetric().getMetricParamsJson();
        JSONObject jsonObj = JSON.parseObject(metricParamsJson);
        String filePermission = jsonObj.getString("file_permission");
        String dirPermission = jsonObj.getString("dir_permission");

        boolean isBeyondPermission = checkBeyondPermission(tableFsPath, filePermission, dirPermission, tableOwner);
        //2. 遍历所有的文件和文件夹 递归
        //3. 文件和文件夹分别和建议权限进行比较，如果超过 返回标识

        //4. 是否超过权限，如果是 0 分+问题描述
        if (isBeyondPermission) {
            governanceAssessDetail.setAssessScore(BigDecimal.ZERO);
            governanceAssessDetail.setAssessProblem("存在或目录超过建议权限");
        }
    }

    private boolean checkBeyondPermission(String tableFsPath, String filePermission, String dirPermission, String tableOwner) throws Exception {
        FileSystem fs = FileSystem.get(new URI(tableFsPath), new Configuration(), tableOwner);

        FileStatus[] fileStatuses = fs.listStatus(new Path(tableFsPath));
    }
}

```

```
        boolean isBeyondPermission = checkBeyondPermissionRec(fileStatuses , fs ,
filePermission , dirPermission) ;

        return isBeyondPermission ;
    }

    private boolean checkBeyondPermissionRec(FileStatus[] fileStatuses, FileSystem fs ,
String filePermission , String dirPermission) throws IOException {
        for (FileStatus fileStatus : fileStatuses) {
            if(fileStatus.isFile()){
                //1. 叶子节点（文件） , 处理结论 或者 return
                FsPermission currentFilePermission = fileStatus.getPermission();
                boolean isBeyondPermission =
checkPermissionByFileOrDir(currentFilePermission , filePermission) ;
                if(isBeyondPermission){
                    return true ;
                }
            }else{
                //2.非叶子节点(目录) , 2.1 判断 2.2 调用自己
                FsPermission currentDirPermission = fileStatus.getPermission();
                boolean isBeyondPermission = checkPermissionByFileOrDir(currentDirPermission,
dirPermission);
                if(isBeyondPermission){
                    return true ;
                }else{
                    FileStatus[] subFileStatus = fs.listStatus(fileStatus.getPath());
                    boolean isSubBeyondPermission = checkBeyondPermissionRec(subFileStatus,
fs, filePermission, dirPermission);
                    if(isSubBeyondPermission){
                        return true ;
                    }
                }
            }
        }

        return false;
    }

    private boolean checkPermissionByFileOrDir(FsPermission currentFilePermission, String
fileOrDirPermission) {
        char ownerP = fileOrDirPermission.charAt(0);
        char groupP = fileOrDirPermission.charAt(1);
        char otherP = fileOrDirPermission.charAt(2);

        if(currentFilePermission.getUserAction().ordinal() > Integer.parseInt(ownerP + "")){
            return true;
        }
        if(currentFilePermission.getGroupAction().ordinal() > Integer.parseInt( groupP +
"")){
            return true ;
        }
        if((currentFilePermission.getOtherAction().ordinal() > Integer.parseInt(otherP +
""))){
            return true ;
        }

        return false;
    }
}
```

第 11 章 对接 dolphinscheduler 的指标

更多 Java –大数据 –前端 –python 人工智能资料下载，可百度访问：[尚硅谷官网](http://www.shang硅谷.com)

(1) 流程中的每个节点不能以层为单位进行配置，要以 SQL 为单位进行配置，即每个 SQL 一个节点。

(3) 为了方便其他应用通过表定位到任务，所以每个任务定义的名称要以“库名.表名”构命名。

11.2 了解 dolphinscheduler 的任务的数据表

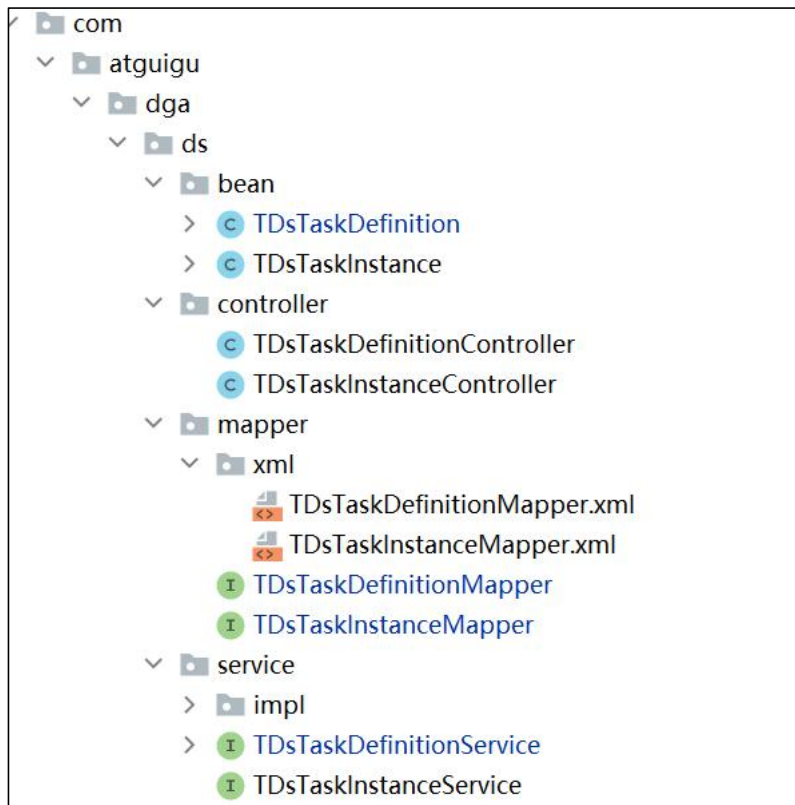


t_ds_task_definition 保存了任务的定义比如每个节点表名和执行 sql 或脚本，以及设定超时的时间，重试次数等等。

t_ds_task_instance 保存了具体某次该任务被运行的情况，包括本次运行时长、运行状态。

11.3 生成 t_ds_task_definition 和 t_ds_task_instance 的基本代码

确定了我们要对这两个表进行查询等操作，可以利用 MyBatis-Plus 代码生成工具直接生成这两个表的代码。



其中 controller 和 mapper 的 xml 部分不适用可以删除掉。

11.4 将任务定义加载到评估参数中

1) 修改 application.properties 添加 ds 动态数据源

```
spring.datasource.dynamic.datasource.ds.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.dynamic.datasource.ds.url=jdbc:mysql://hadoop102:3306/dolphinscheduler?serverTimezone=UTC&useUnicode=true&characterEncoding=utf-8&useSSL=false&allowPublicKeyRetrieval=true
spring.datasource.dynamic.datasource.ds.username=root
spring.datasource.dynamic.datasource.ds.password=000000
```

2) 修改评估参数类

```
@Data
public class AssessParam {
    String assessDate ;
}
```

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
TableMetaInfo tableMetaInfo ;

GovernanceMetric governanceMetric ;

List<TableMetaInfo> tableMetaInfoList ;

// 该指标需要的任务定义
TDsTaskDefinition tDsTaskDefinition ;
// 该指标需要的任务状态信息(当日)
TDsTaskInstance tDsTaskInstance ;
}
```

3) 查询 DS 中的任务实例

TdsTaskInstanceService 接口中添加方法

```
public interface TDsTaskInstanceService extends IService<TDsTaskInstance> {
    List<TDsTaskInstance> getCurrentTaskInstanceList(String assessDate) ;
}
```

TdsTaskInstanceServiceImpl 实现类中实现方法

```
@Service
@DS("ds")
public class TDsTaskInstanceServiceImpl extends ServiceImpl<TDsTaskInstanceMapper,
TDsTaskInstance> implements TDsTaskInstanceService {

    public List<TDsTaskInstance> getCurrentTaskInstanceList(String assessDate){
        QueryWrapper<TDsTaskInstance> queryWrapper = new
QueryWrapper<TDsTaskInstance>().exists(
            "select 1 from " +
                " (select max(id) max_id " +
                "   from t_ds_task_instance" +
                "  where state = 7 and date_format(start_time , '%Y-%m-%d') = '" +
assessDate + "' group by name) tmax " +
                "where t_ds_task_instance.id = tmax.max_id");
        List<TDsTaskInstance> dsTaskInstanceList = list(queryWrapper);

        return dsTaskInstanceList ;
    }
}
```

4) 查询 DS 中的任务定义

TdsTaskDefinitionService 接口中添加方法

```
public interface TDsTaskDefinitionService extends IService<TDsTaskDefinition> {
    List<TDsTaskDefinition> getTaskDefinitionByInstances(List<TDsTaskInstance>
taskInstanceList) ;
}
```

TdsTaskDefinitionServiceImpl 类中实现方法

```
@Service
@DS("ds")
public class TDsTaskDefinitionServiceImpl extends ServiceImpl<TDsTaskDefinitionMapper,
TDsTaskDefinition> implements TDsTaskDefinitionService {

    public List<TDsTaskDefinition> getTaskDefinitionByInstances(List<TDsTaskInstance>
taskInstanceList){
        List<Long> taskCodeList =
taskInstanceList.stream().map(TDsTaskInstance::getTaskCode).collect(Collectors.toList());
```

```
QueryWrapper<TDsTaskDefinition> queryWrapper = new
QueryWrapper<TDsTaskDefinition>().in("code", taskCodeList);

List<TDsTaskDefinition> taskDefinitionList = list(queryWrapper);

//把 definition 中的脚本提取出 SQL
for (TDsTaskDefinition tDsTaskDefinition : taskDefinitionList) {
    String sql = extractSqlFromDefinition(tDsTaskDefinition);
    tDsTaskDefinition.setSql(sql);
}

return taskDefinitionList ;
}
/**
 * 从任务定义中提取 SQL
 */
private String extractSqlFromDefinition(TDsTaskDefinition tDsTaskDefinition){
    // 先提取脚本
    String taskParams = tDsTaskDefinition.getTaskParams() ;
    JSONObject jsonObj = JSON.parseObject(taskParams);
    String shellScript = jsonObj.getString("rawScript");
    // 在从脚本中提取 sql
    //开头:  with 开头 或者 insert 开头 , 有 with 则取 with, 没有则取 insert
    int sqlStartIndex = 0;
    int withIdx = shellScript.indexOf("with") ; //如果未找到, 返回-1
    if(withIdx >= 0 ){
        sqlStartIndex = withIdx ;
    }else{
        sqlStartIndex = shellScript.indexOf("insert");
    }
    //如果找不到 with 或者 insert, 说明不是 sql 任务, 不进行下面的切分
    if(sqlStartIndex == - 1 ){
        return null ;
    }

    //结尾:  ; 和" 都有的情况, 取; , 一个都没有取"
    int sqlEndIndex = 0 ;
    int fenhaoIdx = shellScript.indexOf(";", sqlStartIndex);
    if(fenhaoIdx >=0 ){
        sqlEndIndex = fenhaoIdx;
    }else{
        sqlEndIndex = shellScript.indexOf("\"" , sqlStartIndex) ;
    }

    //切
    String sql = shellScript.substring(sqlStartIndex, sqlEndIndex);

    return sql;
}
}
```

5) 修改考评处理类

```
public void mainAssess( String assessDate){
    // 0 幂等性处理 , 把考评档期的考评结果清理
    remove( new QueryWrapper<GovernanceAssessDetail>().eq("assess_date" , assessDate));
    // 1. 提取表元数据信息, 提取到一个 List<>
    // 1.1 元数据信息 + 辅助信息
    // 1.2 结合考评日期
    List<TableMetaInfo> tableMetaList =
tableMetaInfoService.getTableMetaList(assessDate);
    Map<String , TableMetaInfo> tableMetaInfoMap = new HashMap<>();
    for (TableMetaInfo tableMetaInfo : tableMetaList) {
```

```
        tableMetaInfoMap.put(tableMetaInfo.getSchemaName() + "." +
tableMetaInfo.getTableName() , tableMetaInfo) ;
    }
    // 2. 提取需要考评的指标, 提取到一个 List<>
    // 2.1 启用状态
    List<GovernanceMetric> governanceMetricList = governanceMetricService.list(
        new QueryWrapper<GovernanceMetric>()
            .eq("is_disabled", "0")
    );
    // 查询所有表的 ds 的任务定义 任务状态信息
    // 任务状态信息(当日) 传入 考评日期
    List<TDsTaskInstance> currentTaskInstanceList =
tdsTaskInstanceService.getCurrentTaskInstanceList(assessDate) ;
    //转换成 Map, 方便通过表名提取对应的定义和实例
    Map<String, TDsTaskInstance> currentTaskInstanceMap = new HashMap<>();
    Map<String, TDsTaskDefinition> currentTaskDefinitionMap = new HashMap<>();

    if(currentTaskInstanceList.size() > 0 ) {
        for (TDsTaskInstance tDsTaskInstance : currentTaskInstanceList) {
            currentTaskInstanceMap.put(tDsTaskInstance.getName(), tDsTaskInstance);
        }
        // 通过任务实例的 code, ds 的任务定义
        List<TDsTaskDefinition> currentTaskDefinitionList =
tdsTaskDefinitionService.getTaskDefinitionByInstances(currentTaskInstanceList);

        for (TDsTaskDefinition tDsTaskDefinition : currentTaskDefinitionList) {
            currentTaskDefinitionMap.put(tDsTaskDefinition.getName(), tDsTaskDefinition);
        }
    }
    List<CompletableFuture<GovernanceAssessDetail>> futureList = new
ArrayList<>( tableMetaList.size() * governanceMetricList.size() );
    long startMs = System.currentTimeMillis();
    // 3. 对每张表每个指标, 逐次进行考评
    List<GovernanceAssessDetail> governanceAssessDetailList = new
ArrayList<>( tableMetaList.size() * governanceMetricList.size());
    for (TableMetaInfo tableMetaInfo : tableMetaList) {
        for (GovernanceMetric governanceMetric : governanceMetricList) {
            if(governanceMetric.getSkipAssessTables() != null &&
governanceMetric.getSkipAssessTables().trim().length() > 0 ){
                String[] skipTableArr =
governanceMetric.getSkipAssessTables().split(",");
                //检查是否需要跳过检查的表
                boolean isSkip = false ;
                for (String skipTable: skipTableArr) {
                    if(tableMetaInfo.getTableName().equals(skipTable)){
                        isSkip = true ;
                        break ;
                    }
                }
                //是跳过表, 本次检查跳过
                if(isSkip){
                    continue;
                }
            }
            //某张表某个指标的考评
            String metricCode = governanceMetric.getMetricCode();
            Assessor assessor = springBeanProvider.getBeanByName(metricCode,
Assessor.class);

            AssessParam assessParam = new AssessParam();
            assessParam.setAssessDate(assessDate);
            assessParam.setTableMetaInfo(tableMetaInfo);
            assessParam.setGovernanceMetric(governanceMetric);
```

```

        assessParam.setTableMetaInfoList(tableMetaList);
//补充 DS 任务定义
assessParam.setTDsTaskDefinition( currentTaskDefinitionMap.get(tableMetaInfo.getSchemaName()
+ "." + tableMetaInfo.getTableName()));
//补充 DS 任务实例
assessParam.setTDsTaskInstance( currentTaskInstanceMap.get(tableMetaInfo.getSchemaName() +
"." + tableMetaInfo.getTableName()));

        GovernanceAssessDetail governanceAssessDetail =
assessor.metricAssess(assessParam);
        governanceAssessDetailList.add(governanceAssessDetail) ;
    }
}
// 4. 把考评结果写到 Mysql
saveBatch(governanceAssessDetailList) ;
}

```

11.5 指标：当日有任务报错

考评板块	考评指标	考评标准	需要信息
计算	计算中是否有报错	检查 DS 有报错 则给 0 分，其 余给 10 分	任务信息

```

@Component("TASK_FAILED")
public class TaskFailedAssessor extends Assessor {

    @Autowired
    TDsTaskInstanceService tDsTaskInstanceService;

    @Override
    protected void checkProblem(GovernanceAssessDetail governanceAssessDetail, AssessParam
assessParam) throws Exception {

        QueryWrapper<TDsTaskInstance> queryWrapper = new QueryWrapper<TDsTaskInstance>()
            .eq("date_format(start_time,'%Y-%m-%d')", assessParam.getAssessDate()) //当
            .eq("name", assessParam.getTableMetaInfo().getSchemaName() + "." +
assessParam.getTableMetaInfo().getTableName()) //对应库名.表名的任务
            .eq("state",6); //状态为失败

        List<TDsTaskInstance> failTaskInstanceList =
tDsTaskInstanceService.list(queryWrapper);

        if(failTaskInstanceList.size()>0){
            governanceAssessDetail.setAssessScore(BigDecimal.ZERO);
            governanceAssessDetail.setAssessProblem("存在失败任务");
        }

    }
}

```

11.6 指标：表产出时效监控

考评板块	考评指标	考评标准	需要信息
质量	表产出时效监控	当日产出时效，超过前{days}天产出 时效平均值{percent}% 则给 0 分，其余 10 分	任务信息

```
@Component("TIME_LINESS")
public class TimeLineAssessor extends Assessor {

    @Autowired
    TDsTaskInstanceService tDsTaskInstanceService;

    @Override
    protected void checkProblem(GovernanceAssessDetail governanceAssessDetail, AssessParam
    assessParam) throws Exception {
        // ODS 的排除

        if(assessParam.getTableMetaInfo().getTableMetaInfoExtra().getDwLevel().equals("ODS")){
            return ;
        }
        // 从参数中 taskInstance 提取 起止时间 得到秒数
        TDsTaskInstance tDsTaskInstance = assessParam.getTDsTaskInstance();
        if(tDsTaskInstance == null ) return ;

        Date submitTime = tDsTaskInstance.getSubmitTime();
        Date endTime = tDsTaskInstance.getEndTime();
        long durationSec = (endTime.getTime() - submitTime.getTime()) / 1000 ;

        // 从指标参数中提取 json  days  percent
        String metricParamsJson = assessParam.getGovernanceMetric().getMetricParamsJson();
        JSONObject jsonObject = JSON.parseObject(metricParamsJson);
        Integer days = jsonObject.getInteger("days");
        Integer percent = jsonObject.getInteger("percent");

        String assessDate = assessParam.getAssessDate();
        Date assessDateDt = DateUtils.parseDate(assessDate, "yyyy-MM-dd");
        Date beforeDaysDt = DateUtils.addDays(assessDateDt, 0 - days);
        String beforeDaysDtStr = DateFormatUtils.format(beforeDaysDt, "yyyy-MM-dd");

        // 进行查询, 查询前 days 日平均耗时, 可以查询列表, 用 java 汇总, 或者直接写 sql, 用 avg 求平
        均

        QueryWrapper<TDsTaskInstance> queryWrapper = new QueryWrapper<TDsTaskInstance>()
            .exists(
                " select 1 from (select max(id) max_id from t_ds_task_instance " +
                " where state = 7 and name ='" +
                governanceAssessDetail.getSchemaName() + "." + governanceAssessDetail.getTableName() + "' " +
                " and date_format(start_time , '%Y-%m-%d') >= '" +
                beforeDaysDtStr + "' " +
                " group by name, date_format(start_time, '%Y-%m-%d')) tmax " +
                " where t_ds_task_instance.id = tmax.max_id "
            );

        List<TDsTaskInstance> beforeTaskInstanceList =
        tDsTaskInstanceService.list(queryWrapper);
        long sumBeforeDaysDurationSec = 0 ;
        for (TDsTaskInstance dsTaskInstance : beforeTaskInstanceList) {
            long beforeDayDurationSec = (dsTaskInstance.getEndTime().getTime() -
            dsTaskInstance.getSubmitTime().getTime()) / 1000 ;
            sumBeforeDaysDurationSec += beforeDayDurationSec ;
        }
        // (当日秒数 - 平均值) / 平均值 > percent , 大于给 0 分,
        long avg = sumBeforeDaysDurationSec / beforeTaskInstanceList.size() ;
        long currentPercent = (durationSec - avg) * 100 / avg;

        governanceAssessDetail.setAssessComment("本次耗时: " + durationSec + " 秒, 平均耗时: "
        + avg + " 秒");
    }
}
```

```

        if(currentPercent > percent){
            governanceAssessDetail.setAssessScore(BigDecimal.ZERO);
            governanceAssessDetail.setAssessProblem("本次耗时超过平均值的百分比 : " +
currentPercent + "%");
        }
    }
}

```

11.7 指标：sql 是否简单加工

考评板块	考评指标	考评标准	需要信息
计算	简单加工	sql 语句没有任何 join\groupby\union\ 函数的复杂计算，且 where 过滤字段皆为分区字段，则视为简单加工，给 0 分，其余给 10 分	任务信息

1) 分析

- (1) 分析 sql
- (2) 检查 sql 的每个关键词 判断是否有 join\groupby\union 等关键词。
- (3) 提取整个 sql 涉及的来源表
- (4) 提取整个 sql 涉及过滤的字段
- (5) 根据来源表的元数据信息中的分区信息检查该过滤字段是否是分区字段。

2) Sql 解析工具类

用于把 SQL 解析为语法树

并且注入一个节点处理器，对语法树中的每个节点进行遍历。

```

public class SqlParser {

    public static void main(String[] args) throws ParseException, SemanticException {
        String sql = " select a, b ,c from user_info u where u.id = 123" ;
        SqlParser.prase( new TestDispatcher() , sql);
    }

    static class TestDispatcher implements Dispatcher{

        // 每到达一个节点, 要处理的事项
        @Override
        public Object dispatch(Node nd, Stack<Node> stack, Object... nodeOutputs) throws SemanticException {
            ASTNode astNode = (ASTNode)nd ;
            System.out.println( "type: " + astNode.getType() + " , token : " +
astNode.getToken().getText());
            return null;
        }
    }
}

```

更多 Java –大数据 –前端 –python 人工智能资料下载，可百度访问：尚硅谷官网

```

    }
}

public static void prase(Dispatcher dispatcher , String sql ) throws ParseException,
SemanticException {
    //1. 把 sql 转换为语法树 , 有工具完成, 在 hive 依赖中已经提供了
    ParseDriver parseDriver = new ParseDriver() ; // 用于把 sql 转为语法树
    ASTNode astNode = parseDriver.parse(sql);
    //2. 提供一个遍历器, 后序遍历
    while(astNode.getType() != HiveParser.TOK_QUERY){ //循环遍历, 直到找到第一个 query 节点,
循环退出。用 query 节点作为根节点
        astNode = (ASTNode) astNode.getChild(0);
    }
    //3. 自定义一个节点处理器 , 根据不同的需求在方法外部定义, 定义好后传递

    //4. 把处理器放到遍历器中
    GraphWalker graphWalker = new DefaultGraphWalker(dispatcher);

    //5. 让遍历器遍历语法树
    graphWalker.startWalking(Collections.singleton(astNode) , null );
}
}

```

3) 考评器

```

@Component("SIMPLE_PROCESS")
public class SimpleProcessAssessor extends Assessor {
    @Override
    protected void checkProblem(GovernanceAssessDetail governanceAssessDetail, AssessParam
assessParam) throws Exception {
        // ODS 的排除

        if(assessParam.getTableMetaInfo().getTableMetaInfoExtra().getDwLevel().equals("ODS")){
            return ;
        }
        // 排除没有定义的
        if(assessParam.getTDsTaskDefinition() == null ||
assessParam.getTDsTaskDefinition().getSql() == null ){
            return ;
        }

        //1. 获取 sql
        String sql = assessParam.getTDsTaskDefinition().getSql();
        //2. 创建一个自定义的节点处理器
        SimpleProcessDispatcher simpleProcessDispatcher = new SimpleProcessDispatcher();
        simpleProcessDispatcher.setDefaultSchemaName(governanceAssessDetail.getSchemaName());
        //3. 利用工具类对 sql 进行解析
        SqlParser.prase(simpleProcessDispatcher ,sql );
        //4. 从节点处理器提取相关信息
        // 4.1 涉及复杂计算的集合
        Set<String> complicateProcessSet = simpleProcessDispatcher.getComplicateProcessSet();
        // 4.2 被查询的表
        Set<String> refTablenameSet = simpleProcessDispatcher.getRefTablenameSet();
        // 4.3 被过滤的字段
        Set<String> filterFieldNameSet = simpleProcessDispatcher.getFilterFieldNameSet();
        //5. 根据相关信息判断是否为简单加工, 如果是 给 0 分 写问题描述 备注
        boolean isSimple = true ;
        if(complicateProcessSet.size() > 0 ){
            governanceAssessDetail.setAssessComment("涉及复杂处理: " + complicateProcessSet);
            isSimple = false ;
        }else{
            //判断 filterFieldNameSet 是否都是分区字段, 依据: refTableNameSet 表的元数据中的分区字段
            for (String tableNameWithSchema : refTablenameSet) {

```



```
//从被查询表中找分区字段集合
Map<String, TableMetaInfo> tableMetaInfoMap =
assessParam.getTableMetaInfoMap();
TableMetaInfo refTableMetaInfo = tableMetaInfoMap.get(tableNameWithSchema);
String partitionColNameJson = refTableMetaInfo.getPartitionColNameJson();
JSONArray partitionJsonArray = JSON.parseArray(partitionColNameJson);
//迭代过滤字段，用过滤字段和分区字段进行依次比较
for (String fieldName : filterFieldNameSet) {
    boolean isFieldPartitionCol = false;
    for (int i = 0; i < partitionJsonArray.size(); i++) {
        JSONObject partitionColJsonObj = partitionJsonArray.getJSONObject(i);
        String partitionColName = partitionColJsonObj.getString("name");
        //如果相同，该过滤字段为分区字段
        if(fieldName.equals(partitionColName)){
            isFieldPartitionCol = true;
            break;
        }
    }
    //只要不是分区字段，不是简单处理
    if(!isFieldPartitionCol){
        governanceAssessDetail.setAssessComment("该字段为非分区字段：" +
fieldName);

        isSimple = false;
        break;
    }else{
        isSimple = true;
    }
}
}

if(isSimple){
    governanceAssessDetail.setAssessScore(BigDecimal.ZERO);
    governanceAssessDetail.setAssessProblem("该Sql为简单加工");
}
}

class SimpleProcessDispatcher implements Dispatcher{

    //收集
    // 4.1 涉及复杂计算的集合
    @Getter
    Set<String> complicateProcessSet = new HashSet<>();
    // 4.2 被查询的表，带库名
    @Getter
    Set<String> refTablenameSet = new HashSet<>();

    @Setter
    String defaultSchemaName = null;

    // 4.3 被过滤的字段
    @Getter
    Set<String> filterFieldNameSet = new HashSet<>();

    //定义一个集合，哪些计算是复杂计算
    Set<Integer> complicateTokSet = Sets.newHashSet(
        HiveParser.TOK_JOIN, // join,包含通过 where 连接的情况
        HiveParser.TOK_GROUPBY, // group by
        HiveParser.TOK_LEFTOUTERJOIN, // left join
        HiveParser.TOK_RIGHTOUTERJOIN, //right join
        HiveParser.TOK_FULLOUTERJOIN, // full join
        HiveParser.TOK_FUNCTION, // count(1)
    );
}
```

```

        HiveParser.TOK_FUNCTIONDI, // count(distinct xx)
        HiveParser.TOK_FUNCTIONSTAR, // count(*)
        HiveParser.TOK_SELECTDI, // distinct
        HiveParser.TOK_UNIONALL // union
    );

    Set<String> operatorSet = Sets.newHashSet("=", ">", "<", ">=", "<=", "<>", "like");

    @Override
    public Object dispatch(Node nd, Stack<Node> stack, Object... nodeOutputs) throws
SemanticException {
        ASTNode astNode = (ASTNode) nd;
        //1. 任意一个节点 只要在复杂计算中 都要记录下来
        if (complicateTokSet.contains(astNode.getType())) {
            complicateProcessSet.add(astNode.getToken().getText());
        }
        //2. 收集所有 where 条件中的字段 TOK_WHERE, 找比较符号下面的. 或者 TOK_TABLE_OR_COL
        // 是一个条件判断符号, 而且是一个 where 下面的。
        if (operatorSet.contains(astNode.getToken().getText()) &&
astNode.getAncestor(HiveParser.TOK_WHERE) != null) {
            // 如果进入则为符号节点, 可以去两边的字段或者值
            for (Node fieldOrValueNode : astNode.getChildren()) { // 符号节点
                ASTNode fieldOrValueASTNode = (ASTNode) fieldOrValueNode;
                if (fieldOrValueASTNode.getType() == HiveParser.TOK_TABLE_OR_COL) { // 直
接就是字段

                    String filterFieldName = fieldOrValueASTNode.getChild(0).getText();
                    filterFieldNameSet.add(filterFieldName);
                } else if (fieldOrValueASTNode.getType() == HiveParser.DOT) { // 带表名的
字段

                    String filterFieldName = fieldOrValueASTNode.getChild(1).getText();
                    filterFieldNameSet.add(filterFieldName);
                }
            }
        }

        /* if (astNode.getType() == HiveParser.TOK_WHERE) {
            ArrayList<Node> childrenNodeList = astNode.getChildren();
            for (Node node : childrenNodeList) {
                ASTNode conditionNode = (ASTNode) node;
                for (Node fieldOrValueNode : conditionNode.getChildren()) {
                    ASTNode fieldOrValueASTNode = (ASTNode) fieldOrValueNode;
                    if (fieldOrValueASTNode.getType() == HiveParser.TOK_TABLE_OR_COL) { //
直接就是字段

                        String filterFieldName =
fieldOrValueASTNode.getChild(0).getText();
                        filterFieldNameSet.add(filterFieldName);
                    } else if (fieldOrValueASTNode.getType() == HiveParser.DOT) { // 带表
名的字段

                        String filterFieldName =
fieldOrValueASTNode.getChild(1).getText();
                        filterFieldNameSet.add(filterFieldName);
                    }
                }
            }
        } */
        //3. 收集所有被查询的表名 TOK_TABREF
        if (astNode.getType() == HiveParser.TOK_TABREF) {
            ASTNode tableNameNode = (ASTNode) astNode.getChild(0);
            if (tableNameNode.getChildren().size() == 1) { // 只有表名
                refTableNameSet.add(defaultSchemaName + "." +
tableNameNode.getChild(0).getText());
            } else {

```

```

        refTablenameSet.add( tableNameNode.getChild(0).getText() + "." +
tableNameNode.getChild(1).getText() );
    }
}
return null;
}
}

@Test
public void testDispatcher() throws ParseException, SemanticException {
    //String sql = " insert overwrite table dim_user_info select id ,n mae ,create_time
from ods_user_info where name = 'zhang3' and dt ='2023-05-02' " ;
    String sql =
        " insert overwrite table dim_user_info select id ,n mae ,create_time " +
        " from ods_user_info a , gmall.ods_user_detail b " +
        " where a.id = b.id and name = 'zhang3' and dt ='2023-05-02' " ;
    SimpleProcessDispatcher simpleProcessDispatcher = new SimpleProcessDispatcher();
    simpleProcessDispatcher.setDefaultSchemaName("gmall");
    SqlParser.prase(simpleProcessDispatcher,sql);

    Set<String> complicateProcessSet = simpleProcessDispatcher.getComplicateProcessSet();

    Set<String> refTablenameSet = simpleProcessDispatcher.getRefTablenameSet();

    Set<String> filterFieldNameSet = simpleProcessDispatcher.getFilterFieldNameSet();

    System.out.println(simpleProcessDispatcher);
}
}

```

11.8 指标：sql 是否包含 select*

考评板块	考评指标	考评标准	需要信息
计算	sql 是否包含 select*	sql 语句中包含 select* , 给 0 分，其余给 10 分	任务信息

1) 分析：

(1) 分析 sql

(2) 遍历 sql，查找 select*，虽然是 select* 但是实际上在 sql 语法中情况比较复杂，

比如

select a.* 包含 select *

select b,c,a.* 包含 select *

select a*b 不包含 select *

select count(*) 不包含 select *

可以用正则表达式解决，也可以用语法树解决。

以下为使用语法树遍历解决：

2) 考评器

```
@Component("SELECT_ALL")
public class SelectAllAssessor extends Assessor {

    @Override
    protected void checkProblem(GovernanceAssessDetail governanceAssessDetail, AssessParam
assessParam) throws Exception {

        if (assessParam.getTableMetaInfo().getTableMetaInfoExtra().getDwLevel().equals("ODS")
|| assessParam.getTDsTaskDefinition() == null) { //ods 层没有 sql 处理
            return;
        }

        //提取 sql 进行

        String sql = assessParam.getTDsTaskDefinition().getSql();
        governanceAssessDetail.setAssessComment(sql);

        SelectAllDispatcher selectAllDispatcher = new SelectAllDispatcher();
        SqlParser.parseSQL(sql, selectAllDispatcher);

        if (selectAllDispatcher.hasSelectAll()) {
            governanceAssessDetail.setAssessScore(BigDecimal.ZERO);
            governanceAssessDetail.setAssessProblem("存在 select * 语句");
        }

    }

    public class SelectAllDispatcher implements Dispatcher {

        @Getter
        boolean hasSelectAll = false;

        @Override
        public Object dispatch(Node nd, Stack<Node> stack, Object... nodeOutputs) throws
SemanticException {
            //检查该节点的处理内容
            ASTNode queryNode = (ASTNode) nd;
            //分析查询
            if (queryNode.getType() == HiveParser.TOK_ALLCOLREF) {
                hasSelectAll = true;
            }
            return null;
        }

    }

    @Test
    public void testSelectAll() throws ParseException, SemanticException {
        String sql = "insert overwrite table aa select 1 from (select a.* from order_info
a, order_detail b where a.id=b.oid) cc";
        SelectAllDispatcher selectAllDispatcher = new SelectAllDispatcher();
        SqlParser.parseSQL(sql, selectAllDispatcher);

        System.out.println("selectAllDispatcher = " + selectAllDispatcher);

    }

}
```

11.9 指标：数据倾斜检查

考评板块	考评指标	考评标准	需要信息
计算	计算中是否存在数据倾斜	检查是否有数据倾斜，如果某个 stage 的最大任务耗时超过平均任务耗时的 {percent}%, 只检查耗时超过 {stage_dur_seconds} 秒的 stage。 存在倾斜给 0 分，不存在给 10 分。	任务信息

1) 分析：

数据倾斜主要是指在多并行度操作时，同一个 stage 中某个 task 出现长尾效应，通常是因为数据分派不均造成的。

当然因为任务除了计算在启动调度上也会耗时，越短耗时的任务误差就越高，所以要设定一个 stage 耗时的阈值，尽量审查较长耗时的 stage。

那么如何获得 stage 及其 task 耗时呢，这就需要从历史日志中抓取。

但是如果光从历史日志的文本筛选非常繁琐，索性可以利用 spark 的历史服务来解析日志。历史服务器会把查询日志的操作，发布为 web 接口。可以通过浏览器或者 web 客户端工具查询。返回值均为 json 格式的数据，查询参数为 yarn_id 和 stage_id

2) 步骤：

- (1) 从任务信息中获得 yarn_id。
- (2) 通过 yarn_id 查询 stage 列表。
- (3) 逐个查询 stage 下的 task 耗时信息。
- (4) 统计平均耗时，以及最大 task 耗时对平均耗时的超出比例。
- (5) stage 耗时和最大 task 超出比例超过阈值的直接扣分。

11.9.1 配置并启动 spark 历史服务器

1) 首先确认 yarn 的 jobHistory 服务启动

```
latguigu@hadoop102 ~]$ jps
10593 MasterServer
16385 HistoryServer
7138 DataNode
7554 NodeManager
110758 Jps
9292 QuorumPeerMain
53900 RunJar
10990 AlertServer
7767 JobHistoryServer
11127 ApiApplicationServer
5970 NameNode
10842 LoggerServer
10716 WorkerServer
```

2) 修改 spark-env.sh

```
export SPARK_DIST_CLASSPATH=$(hadoop classpath)
export SPARK_HISTORY_OPTS="-Dspark.history.ui.port=18080
-Dspark.history.fs.logDirectory=hdfs://hadoop102:8020/spark-history
-Dspark.history.retainedApplications=1100"
```

3) 启动 spark 历史服务器

```
/opt/module/spark/sbin/start-history-server.sh
```

11.9.2 增加 Http 客户端工具类

1) 增加 okHttp 依赖

```
<dependency>
  <groupId>com.squareup.okhttp3</groupId>
  <artifactId>okhttp</artifactId>
</dependency>
```

2) Http 工具类

```
public class HttpUtil {
    private static OkHttpClient client = getInstance();

    public static OkHttpClient getInstance(){
        return new OkHttpClient();
    }

    public static String get(String url ){

        Request.Builder builder = new Request.Builder();
        Request request = builder
            .get()
            .url(url)
            .build();
        Call call = client.newCall(request);
        try {
            Response response = call.execute();
            return response.body().string();
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }

    public static void main(String[] args) {
        String json =
HttpUtil.get("http://hadoop102:18080/api/v1/applications/application_1682967712307_0047");
        System.out.println(json);
    }
}
```

```
}  
}
```

11.9.3 考评器代码

1) application.properties 中添加如下配置

```
spark.history.api.url=http://hadoop102:18080/api/v1/applications
```

2) 代码

```
@Component("DATA_SKEW")  
public class DataSkewAssessor extends Assessor {  
  
    @Value("${spark.history.api.url}")  
    String historyUrl ;  
    @Override  
    protected void checkProblem(GovernanceAssessDetail governanceAssessDetail, AssessParam  
assessParam) throws Exception {  
  
        if(assessParam.getTableMetaInfo().getTableMetaInfoExtra().getDwLevel().equals("ODS")){  
            return ;  
        }  
        if(assessParam.getTDsTaskDefinition() == null ){  
            throw new RuntimeException("缺少任务实例");  
        }  
  
        String metricParamsJson = assessParam.getGovernanceMetric().getMetricParamsJson();  
        JSONObject paramJsonObj = JSON.parseObject(metricParamsJson);  
        Integer percent = paramJsonObj.getInteger("percent");  
        Integer stageDurSeconds = paramJsonObj.getInteger("stage_dur_seconds");  
  
        //1. 先获得任务的{yarnId}  
        TDsTaskInstance tDsTaskInstance = assessParam.getTDsTaskInstance();  
        String yarnId = tDsTaskInstance.getAppLink();  
        if(yarnId == null || yarnId.length() == 0 ){  
            throw new RuntimeException("缺少 yarn_id");  
        }  
  
        //2. http://hadoop102:18080/api/v1/applications/{yarnId} 获取完成的{attemptId}  
        String completeAttemptId = getCompleteAttemptId(yarnId);  
  
        //3. http://hadoop102:18080/api/v1/applications/{yarnId}/{attemptId}/stages 有哪些  
        stages List<String> stageIdList  
        List<String> stagesIdList = getStageIdList(yarnId , completeAttemptId) ;  
  
        //4. http://hadoop102:18080/api/v1/applications/{yarnId}/{attemptId}/stages/{stageId}  
        // 提取每个 stage 的 taskDurationList  
        //5. 把 taskDurationList 取平均值  
        //6. 每个 taskDuration - avgDuration / avgDuration ==> taskPercent  
        //7. 用 taskPercent 和 参数中设置的阈值进行比较  
        List<Stage> stageList = calcStageList(yarnId , completeAttemptId , stagesIdList);  
  
        //8. 只要超过阈值, 存在数据倾斜, 给 0 分, 记录哪个 stage 出现的数据倾斜, 保存 comment  
        StringBuilder comment = new StringBuilder() ;  
        for (Stage stage : stageList) {  
            if(stage.getMaxTaskDuration() > stageDurSeconds &&  
stage.getMaxTaskDurationBeyondPercent() > percent ){  
                comment.append("stage_id : " + stage.getStageId() + " 超过平均耗时 : " +  
stage.getMaxTaskDurationBeyondPercent() +"%") ;  
            }  
        }  
        if (comment.length() > 0) {  
            governanceAssessDetail.setAssessScore(BigDecimal.ZERO);  
        }  
    }  
}
```

更多 Java-大数据-前端-python 人工智能资料下载, 可百度访问: 尚硅谷官网

```
governanceAssessDetail.setAssessProblem("存在数据倾斜");
    }
}

private List<Stage> calcStageList(String yarnId, String completeAttemptId, List<String>
stagesIdList) {
    List<Stage> stageList = new ArrayList<>();
    for (String stageId : stagesIdList) {
        String stageUrl = historyUrl + "/" + yarnId + "/" + completeAttemptId +
"/stages/" + stageId;
        String stageJson = HttpUtil.get(stageUrl);
        JSONArray jsonArray = JSON.parseArray(stageJson);
        for (int i = 0; i < jsonArray.size(); i++) {
            JSONObject stageJsonObj = jsonArray.getJSONObject(i);
            List<Integer> taskDurList = new ArrayList<>(); //每个 stage 所有任务的耗时
            if(stageJsonObj.getString("status").equals("COMPLETE")){
                JSONObject taskJsonObj = stageJsonObj.getJSONObject("tasks");

                for (Object taskObj : taskJsonObj.values()) {
                    JSONObject everyTaskJsonObj = (JSONObject) taskObj;
                    if(everyTaskJsonObj.getString("status").equals("SUCCESS")){
                        Integer duration = everyTaskJsonObj.getInteger("duration");
                        taskDurList.add(duration);
                    }
                }
                Stage stage = calcStateByTaskList(taskDurList);
                stage.setStageId(stageJsonObj.getString("stageId"));
                stageList.add(stage);
            }
        }
    }
    return stageList ;
}

private Stage calcStateByTaskList(List<Integer> taskDurList){

    //如果只有一个任务，本任务作为 stage 统计结果
    if(taskDurList.size() == 1){
        Stage stage = new Stage();
        stage.setMaxTaskDuration(taskDurList.get(0));
        stage.setAvgDuration(taskDurList.get(0));
        stage.setMaxTaskDurationBeyondPercent(0);
        return stage ;
    }

    Integer maxTaskDuration = 0 ;
    Integer sumDuration = 0 ;
    for (Integer taskDur : taskDurList) {
        maxTaskDuration = Math.max( maxTaskDuration , taskDur );
        sumDuration +=taskDur ;
    }
    //平均值不包含最大值
    Integer avgDuration = (sumDuration - maxTaskDuration) / (taskDurList.size() - 1) ;

    Integer maxTaskDurationBeyondPercent = (maxTaskDuration - avgDuration) * 100 /
avgDuration ;

    Stage stage = new Stage();
    stage.setMaxTaskDuration(maxTaskDuration);
    stage.setAvgDuration(avgDuration);
    stage.setMaxTaskDurationBeyondPercent(maxTaskDurationBeyondPercent);
    return stage ;
}
```



```
private List<String> getStageIdList(String yarnId, String completeAttemptId) {
    String stagesUrl = historyUrl + "/" + yarnId + "/" + completeAttemptId + "/stages" ;
    String stagesJson = HttpUtil.get(stagesUrl);
    JSONArray stagesJsonArray = JSON.parseArray(stagesJson);
    List<String> stageIdList = stagesJsonArray.stream().map(json -> ((JSONObject)
json).getString("stageId")).collect(Collectors.toList());
    return stageIdList ;
}

private String getCompleteAttemptId( String yarnId){
    String attemptId = historyUrl + "/" + yarnId ;
    String attemptJson = HttpUtil.get(attemptId);
    JSONObject appJson = JSON.parseObject(attemptJson);
    JSONArray attemptJsonArr = appJson.getJSONArray("attempts");
    String completeAttemptId = null ;
    for (int i = 0; i < attemptJsonArr.size(); i++) {
        JSONObject attemptJsonObj = attemptJsonArr.getJSONObject(i);
        if (attemptJsonObj.getBoolean("completed")) {
            completeAttemptId = attemptJsonObj.getString("attemptId");
        }
    }
    return completeAttemptId ;
}

@Data
class Stage{
    private String stageId ;
    //Integer StageDuration ;
    Integer maxTaskDuration ;
    Integer avgDuration ;
    Integer maxTaskDurationBeyondPercent ;

}
}
```

第 12 章 对指标考评过程的优化

由于所有指标考评任务之间是独立的，没有任何依赖，那么可以利用多线程并行执行这些考评任务。

由于需要所有考评任务执行完成以后，统一保存，并进入下一个环节。所以每个任务执行完毕后要做集结，等待其他任务完成。

这就不是简单的多线程，需要通过异步编排实现逐个提交->并行运行->统一集结。

修改后的考评任务代码

```
//声明连接池
ThreadPoolExecutor threadPoolExecutor =
    new ThreadPoolExecutor(50 , 100 , 60 ,TimeUnit.SECONDS , new LinkedBlockingDeque<>(1000));
public void mainAssess( String assessDate){
    // 0 幂等性处理 , 把考评档期的考评结果清理
    remove( new QueryWrapper<GovernanceAssessDetail>().eq("assess_date" , assessDate));

    // 1. 提取表元数据信息, 提取到一个 List<>
    // 1.1 元数据信息 + 辅助信息
```

更多 Java-大数据-前端-python 人工智能资料下载，可百度访问：[尚硅谷官网](http://www.shang硅谷.com)

```
// 1.2 结合考评日期
List<TableMetaInfo> tableMetaList =
tableMetaInfoService.getTableMetaList(assessDate);
Map<String, TableMetaInfo> tableMetaInfoMap = new HashMap<>();
for (TableMetaInfo tableMetaInfo : tableMetaList) {
    tableMetaInfoMap.put(tableMetaInfo.getSchemaName() + "." +
tableMetaInfo.getTableName(), tableMetaInfo);
}

// 2. 提取需要考评的指标, 提取到一个 List<>
// 2.1 启用状态
List<GovernanceMetric> governanceMetricList = governanceMetricService.list(
    new QueryWrapper<GovernanceMetric>()
        .eq("is_disabled", "0")
);

// 查询所有表的 ds 的任务定义 任务状态信息
// 任务状态信息(当日) 传入 考评日期
List<TDsTaskInstance> currentTaskInstanceList =
tdsTaskInstanceService.getCurrentTaskInstanceList(assessDate);
// 转换成 Map, 方便通过表名提取对应的定义和实例
Map<String, TDsTaskInstance> currentTaskInstanceMap = new HashMap<>();
Map<String, TDsTaskDefinition> currentTaskDefinitionMap = new HashMap<>();

if(currentTaskInstanceList.size() > 0) {
    for (TDsTaskInstance tDsTaskInstance : currentTaskInstanceList) {
        currentTaskInstanceMap.put(tDsTaskInstance.getName(), tDsTaskInstance);
    }
    // 通过任务实例的 code, ds 的任务定义
    List<TDsTaskDefinition> currentTaskDefinitionList =
tdsTaskDefinitionService.getTaskDefinitionByInstances(currentTaskInstanceList);

    for (TDsTaskDefinition tDsTaskDefinition : currentTaskDefinitionList) {
        currentTaskDefinitionMap.put(tDsTaskDefinition.getName(), tDsTaskDefinition);
    }
}

List<CompletableFuture<GovernanceAssessDetail>> futureList = new
ArrayList<>(tableMetaList.size() * governanceMetricList.size());
long startMs = System.currentTimeMillis();
// 3. 对每张表每个指标, 逐次进行考评
List<GovernanceAssessDetail> governanceAssessDetailList = new
ArrayList<>(tableMetaList.size() * governanceMetricList.size());
for (TableMetaInfo tableMetaInfo : tableMetaList) {
    for (GovernanceMetric governanceMetric : governanceMetricList) {
        if(governanceMetric.getSkipAssessTables() != null &&
governanceMetric.getSkipAssessTables().trim().length() > 0){
            String[] skipTableArr =
governanceMetric.getSkipAssessTables().split(",");
            //检查是否需要跳过检查的表
            boolean isSkip = false;
            for (String skipTable: skipTableArr) {
                if(tableMetaInfo.getTableName().equals(skipTable)){
                    isSkip = true;
                    break;
                }
            }
            //是跳过表, 本次检查跳过
            if(isSkip){
                continue;
            }
        }
    }
}
```

```

        //某张表某个指标的考评
        String metricCode = governanceMetric.getMetricCode();
        Assessor assessor = springBeanProvider.getBeanByName(metricCode,
Assessor.class);

        AssessParam assessParam = new AssessParam();
        assessParam.setAssessDate(assessDate);
        assessParam.setTableMetaInfo(tableMetaInfo);
        assessParam.setGovernanceMetric(governanceMetric);
        //assessParam.setTableMetaInfoList(tableMetaList);
        assessParam.setTableMetaInfoMap( tableMetaInfoMap );

        assessParam.setTDsTaskDefinition( currentTaskDefinitionMap.get(tableMetaInfo.getSchemaName()
+ "." + tableMetaInfo.getTableInfo()));

        assessParam.setTDsTaskInstance( currentTaskInstanceMap.get(tableMetaInfo.getSchemaName() +
"." + tableMetaInfo.getTableInfo()));

        // 此处为核心计算逻辑， 可以考虑放到异步线程进行计算
        //GovernanceAssessDetail governanceAssessDetail =
assessor.metricAssess(assessParam);
        //governanceAssessDetailList.add(governanceAssessDetail) ;

        // 异步改造
        CompletableFuture<GovernanceAssessDetail> future =
CompletableFuture.supplyAsync(
            () -> {
                GovernanceAssessDetail governanceAssessDetail =
assessor.metricAssess(assessParam);
                return governanceAssessDetail;
            }
            ,
            threadPoolExecutor
        );

        futureList.add(future) ;

    }
}

// 异步计算， 最后集结， 统一计算
governanceAssessDetailList = futureList.stream().map(future ->
future.join()).collect(Collectors.toList());

long durationMs = System.currentTimeMillis() - startMs ;
System.out.println("所有考评一共消耗 " + durationMs + "毫秒");

// 4. 把考评结果写到 Mysql
saveBatch(governanceAssessDetailList) ;
}

```

第 13 章 任务四：数据治理考评的结果核算为分数

需要根据考评结果明细，核算为三种分数：

- (1) 每张表的分数
- (2) 每个技术负责人的分数
- (3) 全局分数

更多 Java –大数据 –前端 –python 人工智能资料下载，可百度访问：[尚硅谷官网](http://www.shangguigu.com)

主要的计算方式就是利用 sql 的 group by 进行计算。

计算核算到表时，要考虑考评指标的治理类型，不同治理类型对应不同的权重。要把分数乘以权重计算该治理类型的分数。

13.1 计算每张表的考评分

13.1.1 各表的考评结果表

```
create table if not exists dga.governance_assess_table
(
    id                bigint primary key auto_increment comment 'id',
    assess_date       varchar(10)    null comment '考评日期',
    table_name        varchar(200)   null comment '表名',
    schema_name       varchar(200)   null comment '库名',
    tec_owner         varchar(200)   null comment '技术负责人',
    score_spec_avg    decimal(10, 2) null comment '规范分数',
    score_storage_avg decimal(10, 2) null comment '存储分数',
    score_calc_avg    decimal(10, 2) null comment '计算分数',
    score_quality_avg decimal(10, 2) null comment '质量分数',
    score_security_avg decimal(10, 2) null comment '安全分数',
    score_on_type_weight decimal(10, 2) null comment '五维权重后分数',
    problem_num       bigint         null comment '问题项个数',
    create_time       datetime       null comment '创建日期'
)
comment '表治理考评情况';
```

13.1.2 根据表生成对应的代码

package		说明
bean	GovernanceAssessTable	各表考评结果实体类
mapper	GovernanceAssessTableMapper	各表考评结果接口
service	GovernanceAssessTableService	各表考评结果接口
service.impl	GovernanceAssessTableServiceImpl	各表考评结果实现类

13.1.3 思路

- (1) 先利用 sql 进行聚合，把每个表的五维的平均分数。(Mapper 层实现)
- (2) 查询治理类型的权重。
- (3) 结合权重计算，该表的总考评分数。

13.1.4 Mapper 层求各表五维平均分数

```
@Mapper
@DS("dga")
public interface GovernanceAssessTableMapper extends BaseMapper<GovernanceAssessTable> {

    @Select("SELECT \n" +
        "    assess_date ,\n" +
        "    TABLE_NAME,\n" +
        "    SCHEMA_NAME,\n" +
        "    tec_owner,\n" +
```

```
        "    AVG(IF(governance_type = 'SPEC', assess_score , NULL )) score_spec_avg ,\n" +
        "    AVG(IF(governance_type = 'STORAGE', assess_score , NULL ))\n" +
        "score_storage_avg ,\n" +
        "    AVG(IF(governance_type = 'CALC', assess_score , NULL )) score_calc_avg,\n" +
        "    AVG(IF(governance_type = 'QUALITY', assess_score , NULL ))\n" +
        "score_quality_avg,\n" +
        "    AVG(IF(governance_type = 'SECURITY', assess_score , NULL ))\n" +
        "score_security_avg,\n" +
        "    SUM(IF(assess_score<10,1,0)) problem_num " +
        "FROM governance_assess_detail  \n" +
        "where assess_date=#{assessDate} \n" +
        "GROUP BY TABLE_NAME, assess_date ,SCHEMA_NAME, tec_owner")
    public List<GovernanceAssessTable> selectGovernanceAssessTableByDetail( String
assessDate );
}
```

13.1.5 Service 层结合权重求总分

GovernanceAssessTableService 接口增加方法

```
public interface GovernanceAssessTableService extends IService<GovernanceAssessTable> {
    public void calcGovernanceAssessTable(String assessDate);
}
```

GovernanceAssessTableServiceImpl 实现方法

```
@Service
@DS("dga")
public class GovernanceAssessTableServiceImpl extends
ServiceImpl<GovernanceAssessTableMapper, GovernanceAssessTable> implements
GovernanceAssessTableService {

    @Autowired
    GovernanceTypeService governanceTypeService ;

    @Override
    public void calcGovernanceAssessTable(String assessDate) {
        remove(new QueryWrapper<GovernanceAssessTable>().eq("assess_date" , assessDate));

        List<GovernanceAssessTable> governanceAssessTables =
            baseMapper.selectGovernanceAssessTableByDetail(assessDate);

        List<GovernanceType> governanceTypes = governanceTypeService.list();
        Map<String, BigDecimal> typeWeightMap = new HashMap<>();
        for (GovernanceType governanceType : governanceTypes) {
            typeWeightMap.put( governanceType.getTypeCode() ,
governanceType.getTypeWeight());
        }

        for (GovernanceAssessTable governanceAssessTable : governanceAssessTables) {
            BigDecimal specScoreWithWeight =
governanceAssessTable.getScoreSpecAvg().multiply(typeWeightMap.get("SPEC")).divide(BigDecimal.TEN, 1 , RoundingMode.HALF_UP);
            BigDecimal storageScoreWithWeight =
governanceAssessTable.getScoreStorageAvg().multiply(typeWeightMap.get("STORAGE")).divide(BigDecimal.TEN, 1 , RoundingMode.HALF_UP);
            BigDecimal calcScoreWithWeight =
governanceAssessTable.getScoreCalcAvg().multiply(typeWeightMap.get("CALC")).divide(BigDecimal.TEN, 1 , RoundingMode.HALF_UP);
            BigDecimal qualityScoreWithWeight =
governanceAssessTable.getScoreQualityAvg().multiply(typeWeightMap.get("QUALITY")).divide(BigDecimal.TEN, 1 , RoundingMode.HALF_UP);
```

更多 Java –大数据 –前端 –python 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```

        BigDecimal securityScoreWithWeight =
governanceAssessTable.getScoreSecurityAvg().multiply(typeWeightMap.get("SECURITY")).divide(B
igDecimal.TEN, 1, RoundingMode.HALF_UP);

        BigDecimal total =
specScoreWithWeight.add(storageScoreWithWeight).add(calcScoreWithWeight).add(qualityScoreWit
hWeight).add(securityScoreWithWeight);

        governanceAssessTable.setScoreOnTypeWeight(total);
        governanceAssessTable.setCreateTime(new Date());
    }

    saveBatch(governanceAssessTables);
}
}

```

13.2 计算每个技术负责人的考评分

13.2.1 各个技术负责人的考评结果表

```

create table if not exists dga.governance_assess_tec_owner
(
    id          bigint      primary key auto_increment comment 'id',
    assess_date varchar(10)  null comment '考评日期',
    tec_owner   varchar(200) null comment '技术负责人',
    score_spec  decimal(10, 2) null comment '规范分数',
    score_storage decimal(10, 2) null comment '存储分数',
    score_calc  decimal(10, 2) null comment '计算分数',
    score_quality decimal(10, 2) null comment '质量分数',
    score_security decimal(10, 2) null comment '安全分数',
    score       decimal(10, 2) null comment '分数',
    table_num   bigint       null comment '涉及表',
    problem_num bigint       null comment '问题项个数',
    create_time datetime      null comment '创建时间'
)
comment '技术负责人治理考评表';

```

13.2.2 根据表生成对应的代码

package		说明
bean	GovernanceAssessTecOwner	技术负责人考评结果实体类
mapper	GovernanceAssessTecOwnerMapper	技术负责人考评结果接口
service	GovernanceAssessTecOwnerService	技术负责人考评结果接口
service.impl	GovernanceAssessTecOwnerServiceImpl	技术负责人考评结果实现类

13.2.3 思路

- (1) 利用 sql 进行聚合，计算每个技术负责人的五维平均分数及总分(Mapper 层实现)
- (2) 保存统计结果 (Service 层实现)

13.2.4 Mapper 层求各个技术负责人五维平均分数

```

@Mapper
@DS("dga")

```

更多 Java-大数据-前端-python 人工智能资料下载，可百度访问：尚硅谷官网

```
public interface GovernanceAssessTecOwnerMapper extends BaseMapper<GovernanceAssessTecOwner>
{
    @Select("SELECT \n" +
        "    assess_date ,\n" +
        "    tec_owner ,\n" +
        "    AVG(gt.score_spec_avg * 10) score_spec ,\n" +
        "    AVG(gt.score_storage_avg * 10) score_storage ,\n" +
        "    AVG(gt.score_calc_avg * 10) score_calc ,\n" +
        "    AVG(gt.score_quality_avg * 10) score_quality ,\n" +
        "    AVG(gt.score_security_avg * 10) score_security ,\n" +
        "    AVG(gt.score_on_type_weight) score, \n" +
        "    COUNT(*) table_num,\n" +
        "    SUM(problem_num) problem_num,\n" +
        "    NOW() create_time \n" +
        "    \n" +
        "FROM governance_assess_table gt\n" +
        "WHERE assess_date = #{assessDate}\n" +
        "GROUP BY assess_date ,tec_owner")
    public List<GovernanceAssessTecOwner> getAssessTecOwner(String assessDate) ;
}
```

13.2.1 Service 层结合权重求总分

GovernanceAssessTecOwnerService 接口增加方法

```
public interface GovernanceAssessTecOwnerService extends IService<GovernanceAssessTecOwner>
{
    public void calcAssessTecOwner(String assessDate);
}
```

GovernanceAssessTecOwnerServiceImpl 实现方法

```
@Service
@DS("dga")
public class GovernanceAssessTecOwnerServiceImpl extends
    ServiceImpl<GovernanceAssessTecOwnerMapper, GovernanceAssessTecOwner> implements
    GovernanceAssessTecOwnerService {

    @Override
    public void calcAssessTecOwner(String assessDate) {
        remove(new QueryWrapper<GovernanceAssessTecOwner>().eq("assess_date" , assessDate));

        List<GovernanceAssessTecOwner> assessTecOwnerList =
baseMapper.getAssessTecOwner(assessDate);
        saveBatch(assessTecOwnerList);
    }
}
```

13.3 计算全局的考评分

13.3.1 各个全局的考评结果表

```
create table if not exists dga.governance_assess_global
(
    id          bigint primary key auto_increment comment 'id',
    assess_date varchar(10)    null comment '考评日期',
    score_spec   decimal(10, 2) null comment '规范分数',
    score_storage decimal(10, 2) null comment '存储分数',
    score_calc   decimal(10, 2) null comment '计算分数',
    score_quality decimal(10, 2) null comment '质量分数',
    score_security decimal(10, 2) null comment '安全分数',
)
```

更多 Java –大数据 –前端 –python 人工智能资料下载，可百度访问：尚硅谷官网

```

    score          decimal(10, 2) null comment '分数',
    table_num       bigint          null comment '涉及表',
    problem_num     bigint          null comment '问题项个数',
    create_time     datetime        null comment '创建时间'
)
comment '治理总考评表';

```

13.3.2 根据表生成对应的代码

package		说明
bean	GovernanceAssessGlobal	全局考评结果实体类
mapper	GovernanceAssessGlobalMapper	全局考评结果接口
service	GovernanceAssessGlobalService	全局考评结果接口
service.impl	GovernanceAssessGlobalServiceImpl	全局考评结果实现类

13.3.3 思路

- (1) 先用 sql 进行聚合，计算全局的五维平均分数及总分(Mapper 层实现)
- (2) 保存统计结果 (Service 层实现)

13.3.4 Mapper 层求各表五维平均分数

```

@Mapper
@DS("dga")
public interface GovernanceAssessGlobalMapper extends BaseMapper<GovernanceAssessGlobal> {

    @Select("SELECT \n" +
        "    assess_date ,\n" +
        "    AVG(gt.score_spec_avg * 10) score_spec ,\n" +
        "    AVG(gt.score_storage_avg * 10) score_storage ,\n" +
        "    AVG(gt.score_calc_avg * 10) score_calc ,\n" +
        "    AVG(gt.score_quality_avg * 10) score_quality ,\n" +
        "    AVG(gt.score_security_avg * 10) score_security ,\n" +
        "    AVG(gt.score_on_type_weight) score, \n" +
        "    COUNT(*) table_num,\n" +
        "    SUM(problem_num) problem_num,\n" +
        "    NOW() create_time \n" +
        "    \n" +
        "FROM governance_assess_table gt\n" +
        "WHERE assess_date = #{assessDate}\n"
    )
    public List<GovernanceAssessGlobal> getAssessGlobal(String assessDate);
}

```

13.3.5 Service 层结合权重求总分

GovernanceAssessGlobalService 接口增加方法

```

public interface GovernanceAssessGlobalService extends IService<GovernanceAssessGlobal> {

    public void calcAssessGlobal(String assessDate);
}

```

GovernanceAssessGlobalServiceImpl 实现方法

```

@Service
@DS("dga")

```

更多 Java –大数据 –前端 –python 人工智能资料下载，可百度访问：尚硅谷官网


```
public class GovernanceAssessGlobalServiceImpl extends
ServiceImpl<GovernanceAssessGlobalMapper, GovernanceAssessGlobal> implements
GovernanceAssessGlobalService {

    @Override
    public void calcAssessGlobal(String assessDate) {
        remove(new QueryWrapper<GovernanceAssessGlobal>().eq("assess_date" , assessDate)) ;

        List<GovernanceAssessGlobal> assessGlobal = baseMapper.getAssessGlobal(assessDate);
        saveBatch(assessGlobal);
    }
}
```

13.4 把考评任务串联起来统一调度

把整个考评过程用一个方法串联起来，包括：

- (1) 清理当日考评情况
- (2) 考评指标明细生成
- (3) 统计各表治理考评情况
- (4) 统计计算负责人治理考评情况
- (5) 统计全局治理考评情况

增加接口 **MainAssessService**

```
public interface MainAssessService {

    public void mainAssess(String assessDate) ;

}
```

增加 **MainAssessServiceImpl**

```
@Service
public class MainAssessServiceImpl implements MainAssessService {

    @Value("${assess.schema}")
    private String assessSchema ;

    @Autowired
    TableMetaInfoService tableMetaInfoService ;

    @Autowired
    GovernanceAssessDetailService governanceAssessDetailService ;

    @Autowired
    GovernanceAssessTableService governanceAssessTableService ;

    @Autowired
    GovernanceAssessTecOwnerService governanceAssessTecOwnerService ;

    @Autowired
    GovernanceAssessGlobalService governanceAssessGlobalService ;

    @Override
    public void mainAssess(String assessDate) throws Exception {
        //1. 元数据采集
        tableMetaInfoService.initTableMeta( assessSchema , assessDate);
        //2. 考评
```

更多 Java –大数据 –前端 –python 人工智能资料下载，可百度访问：[尚硅谷官网](http://www.shangguigu.com)

```
governanceAssessDetailService.mainAssess (assessDate);  
//3. 核算分数  
governanceAssessTableService.calcGovernanceAssessTable (assessDate);  
governanceAssessTecOwnerService.calcAssessTecOwner (assessDate);  
governanceAssessGlobalService.calcAssessGlobal (assessDate);  
}  
}
```

application.properties 中添加配置

```
assess.schema=gmall
```

13.5 定时调度计算任务

13.5.1 springtask 介绍

springtask 是 springboot 框架中自带的时间调度程序。

优点：简单易用配置少。

缺点：不能做分布式调度。

13.5.1.1 如何启用

在 springboot 启动程序上增加注解 @EnableScheduling。

```
@SpringBootApplication  
@EnableScheduling  
public class DgaApplication {  
  
    public static void main(String[] args) {  
        SpringApplication.run(DgaApplication.class, args);  
    }  
  
}
```

建立一个任务类，类上面增加@Component,在方法上增加表达式。

```
@Component  
public class MarketingTask {  
  
    @Scheduled(cron = "0/10 * * * * *")  
    public void text1(){  
        System.out.println("text1 测试"+System.currentTimeMillis());  
    }  
  
}
```

13.5.1.2 如何配置

@Scheduled 可用的参数

- cron 表达式 标识 "秒 分 时 天 月 星期几"。
- fixedDelay 标识 每次触发的固定间隔毫秒数，注意间隔为是前一次的结束时间到后一次的开始时间点的间隔。
- fixedRate 标识 每次触发的固定间隔毫秒数，注意间隔为是前一次的开始时间到后一次的开始时间点的间隔。如果后一次开始时前一次的任务还没做完，则后一次的处理

会被阻塞，直到前一任务完成。

- `initialDelay` 标识每次启动时初始的延迟，可以配合 `fixedDelay` 或者 `fixedRate` 使用。

```
@Component
public class DemoTask {

    @Scheduled(cron = "0/10 * * * * *")
    public void text1(){
        System.out.println("text1 测试"+DateFormatUtils.format(new Date(),"mm:ss"));
    }

    @Scheduled(fixedDelay = 5000)
    public void text2(){
        System.out.println("text2 测试"+ DateFormatUtils.format(new Date(),"mm:ss"));
    }

    @Scheduled(fixedRate = 5000)
    public void text3(){
        System.out.println("text3 测试"+DateFormatUtils.format(new Date(),"mm:ss"));
    }

    @Scheduled(initialDelay =3000, fixedRate = 5000)
    public void text4(){
        System.out.println("text4 测试"+DateFormatUtils.format(new Date(),"mm:ss"));
    }

}
```

考评项目的调度

```
@Component
public class AssessScheduleTask {

    @Autowired
    TableMetaInfoService tableMetaInfoService;

    @Autowired
    MainAssessService mainAssessService;

    @Value("${assess.schemas}")
    String schemas=null;

    @Scheduled(cron = "0/10 * * * * *")
    public void checkProblem(){
        String[] schemaArr = schemas.split(",");

        String assessDate = DateFormatUtils.format(new Date(), "yyyy-MM-dd");

        for (String schema : schemaArr) {

            tableMetaInfoService.initTableMeta(schema,assessDate);
        }
        mainAssessService.mainAssess(assessDate);
    }

}
```

第 14 章 任务五：为可视化治理考评提供数据接口

14.1 效果图



14.2 接口要求

考评的数据接口主要的目的是为可视化界面提供数据支持。

主要的开发都在 Controller 层。

接口名称	接口路径	请求方式
全局分数	/governance/globalScore	GET
问题列表	/governance/problemList/{governType}/{pageNo}/{pageSize}	GET
五维问题个数	/governance/problemNum	GET
用户评分排行榜	/governance/rankList	GET
手动执行评估	/governance/assess/{date}	POST

14.3 全局分数接口

接口名称	接口路径	请求方式
全局分数	/governance/globalScore	GET
返回值结构示例	{ "assessDate":"2023-04-01", "sumScore":90, "scoreList":[20,40,34,55,66] }	

```
@RestController
@RequestMapping("/governance")
public class GovernanceAssessController {

    @Autowired
    GovernanceAssessGlobalService governanceAssessGlobalService;

    @Autowired
    GovernanceAssessDetailService governanceAssessDetailService ;

    @Autowired
    GovernanceAssessTecOwnerService governanceAssessTecOwnerService ;
```

```
@GetMapping("globalScore")
public String getGlobalScore() {
    //取最新的全局总分
    GovernanceAssessGlobal governanceAssessGlobal
        = governanceAssessGlobalService.getOne(new
    QueryWrapper<GovernanceAssessGlobal>().orderByDesc("assess_date").last("limit 1"));
    JSONObject jsonObject = new JSONObject();
    jsonObject.put("assessDate", governanceAssessGlobal.getAssessDate());
    jsonObject.put("sumScore", governanceAssessGlobal.getScore());
    List<BigDecimal> scoreList = new ArrayList<>();
    scoreList.add(governanceAssessGlobal.getScoreSpec());
    scoreList.add(governanceAssessGlobal.getScoreStorage());
    scoreList.add(governanceAssessGlobal.getScoreCalc());
    scoreList.add(governanceAssessGlobal.getScoreQuality());
    scoreList.add(governanceAssessGlobal.getScoreSecurity());
    jsonObject.put("scoreList", scoreList);

    return jsonObject.toJSONString();
}
```

14.4 问题列表接口

接口名称	接口路径	请求方式
问题列表	/governance/problemList/{governType}/{pageNo}/{pageSize}	GET
返回值结构示例	[{"assessComment":"","assessDate":"2023-05-01","assessProblem":"缺少技术 OWNER","assessScore":0.00,"commentLog":"","createTime":1682954933000, "governanceType":"SPEC","governanceUrl":"/table_meta/table_meta/detail?tableId=1803","id":21947,"isAssessException":0,"metricId":1,"metricName":" 是否有技术 Owner","schemaName":"gmail","tableName":"ads_page_path"}, {"assessComment":"","assessDate":"2023-05-01","assessProblem":"缺少业务 OWNER","assessScore":0.00,"commentLog":"","createTime":1682954933000, "governanceType":"SPEC","governanceUrl":"/table_meta/table_meta/detail?tableId=1803","id":21948,"isAssessException":0,"metricId":2,"metricName":" 是否有业务 Owner","schemaName":"gmail","tableName":"ads_page_path"}, {"assessComment":"","assessDate":"2023-05-01","assessProblem":"缺少技术 OWNER","assessScore":0.00,"commentLog":"","createTime":1682954933000, "governanceType":"SPEC","governanceUrl":"/table_meta/table_meta/detail?tableId=1804","id":21964,"isAssessException":0,"metricId":1,"metricName":" 是否有技术 Owner","schemaName":"gmail","tableName":"ads_user_change"}, {"assessComment":"","assessDate":"2023-05-01","assessProblem":"缺少业务	

	OWNER","assessScore":0.00,"commentLog":"","createTime":1682954933000,"governanceType":"SPEC","governanceUrl":"/table_meta/table_meta/detail?tableId=1804","id":21965,"isAssessException":0,"metricId":2,"metricName":"是否有业务
	Owner","schemaName":"gmall","tableName":"ads_user_change"},{"assessComment":"","assessDate":"2023-05-01","assessProblem":"缺少技术
	OWNER","assessScore":0.00,"commentLog":"","createTime":1682954933000,"governanceType":"SPEC","governanceUrl":"/table_meta/table_meta/detail?tableId=1805","id":21981,"isAssessException":0,"metricId":1,"metricName":"是否有技术
	Owner","schemaName":"gmall","tableName":"ads_user_retention"]}]

在 GovernanceAssessDetailService 增加方法

```
public interface GovernanceAssessDetailService extends IService<GovernanceAssessDetail> {

    public void assessAllTableMetrics(String assessDate);

    public List<GovernanceAssessDetail> getLastProblemListByType(String governType ,Integer pageNo ,Integer pageSize );

}
```

在 GovernanceAssessDetailServiceImpl 增加方法

```
public List<GovernanceAssessDetail> getLastProblemListByType(String governType ,Integer pageNo ,Integer pageSize ){

    //根据每页大小和页码计算显示行号
    int from = (pageNo-1)* pageSize;

    QueryWrapper<GovernanceAssessDetail> queryWrapper = new
    QueryWrapper<GovernanceAssessDetail>()
        .eq("governance_type", governType)
        .lt("assess_score", 10)
        .inSql("assess_date","select max(assess_date) from governance_assess_detail")
        .last("limit " + from + "," + pageSize);
    List<GovernanceAssessDetail> governanceAssessDetailList = list(queryWrapper);
    return governanceAssessDetailList;

}
```

在 GovernanceAssessController 中增加方法

```
@GetMapping("/problemList/{governType}/{pageNo}/{pageSize}")
public String getProblemList(@PathVariable("governType")String
governType ,@PathVariable("pageNo") Integer pageNo ,@PathVariable("pageSize") Integer
pageSize ){

    List<GovernanceAssessDetail> governanceAssessDetailList =
    governanceAssessDetailService.getLastProblemListByType(governType, pageNo, pageSize);

    return JSONObject.toJSONString(governanceAssessDetailList) ;

}
```

14.5 接口: 各个治理类型问题个数

接口名称	接口路径	请求方式
全局分数	/governance/problemNum	GET
返回值结构示例	{ "SPEC":1, "STORAGE":4,"CALC":12,"QUALITY":34,"SECURITY":12}	

在 GovernanceAssessDetailService 增加方法

```
public interface GovernanceAssessDetailService extends IService<GovernanceAssessDetail> {

    public void assessAllTableMetrics (String assessDate);

    public List<GovernanceAssessDetail> getLastProblemListByType (String governType ,Integer pageNo ,Integer pageSize );
    //增加方法
    public List<Map<String, Object>> getLastProblemNum();

}
```

在 GovernanceAssessDetailServiceImpl 增加方法

```
//根据治理类型进行分组聚合 所有问题求个数
public List<Map<String, Object>> getLastProblemNum(){
    QueryWrapper<GovernanceAssessDetail> queryWrapper = new
    QueryWrapper<GovernanceAssessDetail>()
        .select("governance_type","count(*) ct")
        .lt("assess_score", 10)
        .inSql("assess_date","select max(assess_date) from governance_assess_detail")
        .groupBy("governance_type");

    List<Map<String, Object>> mapList = listMaps(queryWrapper);
    return mapList;
}
```

在 GovernanceAssessController 中增加方法

```
@GetMapping("/problemNum")
public String getProblemNum(){
    //求各个治理类型的问题个数
    List<Map<String, Object>> lastProblemNumMapList =
    governanceAssessDetailService.getLastProblemNum();

    //根据接口要求调整结构
    Map<String,Long> problemMap=new HashMap();
    for (Map<String, Object> map : lastProblemNumMapList) {
        String governanceType = (String)map.get("governance_type");
        Long problemNum = (Long)map.get("ct");
        problemMap.put(governanceType,problemNum);
    }
    return JSONObject.toJSONString(problemMap) ;
}
```

14.6 接口：分组人员排行榜

接口名称	接口路径	请求方式
排行榜	/governance/rankList	GET
返回值结构示例	[{"tecOwner":"zhang3","score":99}, {"tecOwner":"li4","score":98}, {"tecOwner": "wang5","score":97}]	

--	--

在 GovernanceAssessController 中增加方法

```
@GetMapping("rankList")
public String getRankList(){
    QueryWrapper<GovernanceAssessTecOwner> queryWrapper = new
    QueryWrapper<GovernanceAssessTecOwner>()
        .select("tec_owner as tecOwner", "score")
        .inSql("assess_date", "select max(assess_date) from
governance_assess_tec_owner")
        .orderByDesc("score");
    List<Map<String, Object>> mapList =
governanceAssessTecOwnerService.listMaps(queryWrapper);

    return JSON.toJSONString(mapList) ;
}
```

14.7 接口：手动触发评估

接口名称	接口路径	请求方式
手动执行评估	/governance/assess/{date}	POST
返回值结构示例	success	

在 GovernanceAssessController 中增加方法

```
@PostMapping("/assess/{date}")
public String getAssess(@PathVariable("date")String assessDate){
    mainAssessService.mainAssess(assessDate);
    return "success";
}
```

14.8 最终接口发布效果图

